

APEX TERMINAL

Two-Year Master Plan — Expanded Edition

104 Weeks | Detailed Technical Breakdown | AI Autopilot Architecture

104 Weeks	6 Strategies	Dual LLM	Full Stack
52 Wks Y1 + 52 Wks Y2	All Option Types	Groq + Gemini	React + FastAPI

Table of Contents

Wk 001	[Y1-Q1] Project Scaffolding & Monorepo Setup
Wk 002	[Y1-Q1] Database Schema & Core Data Models
Wk 003	[Y1-Q1] Backend API Layer & WebSocket Infrastructure
Wk 004	[Y1-Q1] React Frontend Shell & Live Dashboard
Wk 005	[Y1-Q1] Alpaca & Tradier Integration — Live Paper Trading
Wk 006	[Y1-Q1] Options Chain Engine & Strategy Templates
Wk 007	[Y1-Q1] Groq LLM Integration — Fast Candidate Ranking
Wk 008	[Y1-Q1] Gemini LLM Integration — Deep Validation Layer
Wk 009	[Y1-Q1] Entry Scoring Algorithm & Risk Validation Layer
Wk 010	[Y1-Q1] Exit Logic Automation & Position Monitor Loop
Wk 011	[Y1-Q1] n8n Scheduler & Full Autonomous Daily Loop
Wk 012	[Y1-Q1] News Sentiment Engine & Event Classification
Wk 013	[Y1-Q1] Kill Switch, Kill Switch Recovery & Incident System
Wk 014	[Y1-Q2] Walk-Forward Backtesting Engine
Wk 015	[Y1-Q2] Monte Carlo Simulation & Risk-of-Ruin Analysis
Wk 016	[Y1-Q2] Real-Time Greeks Dashboard & Portfolio Exposure
Wk 017	[Y1-Q2] Trade Execution Hardening & Order Management
Wk 018	[Y1-Q2] Performance Attribution & Reporting Engine
Wk 019	[Y1-Q2] Symbol Universe Management & Market Regime Detection
Wk 020	[Y1-Q2] Comprehensive Test Suite — 1,000+ Tests
Wk 021	[Y1-Q2] Options Volatility Surface & IV Analytics
Wk 022	[Y1-Q2] Earnings Calendar & Event-Driven Risk Gateway
Wk 023	[Y1-Q2] Alerting System & Notification Hub
Wk 024	[Y1-Q2] Data Integrity Pipeline & Quality Gates
Wk 025	[Y1-Q2] Frontend Polish: Dark Theme, Animations & UX
Wk 026	[Y1-Q2] Q2 Wrap-Up: Integration Testing, Hardening & v1.0 Tag
Wk 027	[Y1-Q3] Advanced LLM Prompt Engineering & Chain-of-Thought
Wk 028	[Y1-Q3] Portfolio Correlation & Sector Exposure Management
Wk 029	[Y1-Q3] Advanced Exit Logic: Trailing Stops & Gamma Scalping
Wk 030	[Y1-Q3] n8n Workflow Orchestration (Docker Setup)
Wk 031	[Y1-Q3] Advanced Market Data: Tick Data & Level 2 Quotes
Wk 032	[Y1-Q3] ML Feature Engineering & Predictive Signals
Wk 033	[Y1-Q3] Real-Time P&L; Dashboard & Risk Metrics Display
Wk 034	[Y1-Q3] Options Strategy Optimizer & Parameter Tuning
Wk 035	[Y1-Q3] Multi-Account & Paper-to-Live Transition Framework
Wk 036	[Y1-Q3] Advanced Charting: Multi-Frame Analysis & Strategy Overlay
Wk 037	[Y1-Q3] Compliance Logging & Regulatory Audit Trail
Wk 038	[Y1-Q3] Q3 Integration: Live Paper Trading Month
Wk 039	[Y1-Q3] Sentiment Analysis 2.0: Social + Institutional Flow
Wk 040	[Y1-Q4] Portfolio Heat Map & Sector Dashboard
Wk 041	[Y1-Q4] Advanced Risk Metrics: VaR, CVaR & Stress Testing Dashboard
Wk 042	[Y1-Q4] SEC Filing & Earnings Call Transcript Analysis
Wk 043	[Y1-Q4] Options Term Structure & Roll Strategy Engine
Wk 044	[Y1-Q4] System Monitoring & Observability Stack
Wk 045	[Y1-Q4] Multi-Strategy Portfolio: All 6 Strategy Types Live
Wk 046	[Y1-Q4] AI-Powered Market Briefing & Natural Language Reports
Wk 047	[Y1-Q4] API Rate Limiting, Caching & CDN Architecture
Wk 048	[Y1-Q4] Automated Strategy Research & Alpha Discovery
Wk 049	[Y1-Q4] Position Sizing Engine: Kelly Criterion & Volatility Scaling
Wk 050	[Y1-Q4] Full Security Hardening & Penetration Test
Wk 051	[Y1-Q4] Documentation Suite & Interactive API Reference
Wk 052	[Y1-Q4] Year 1 Wrap-Up: Performance Review & Year 2 Planning
Wk 053	[Y2-Q1] Autonomous AI Agent Architecture (LangChain)
Wk 054	[Y2-Q1] Multi-Agent System: Researcher + Validator + Executor
Wk 055	[Y2-Q1] RAG-Powered Market Knowledge Base (ChromaDB)
Wk 056	[Y2-Q1] Real-Time News Arbitrage & Event-Driven Alpha
Wk 057	[Y2-Q1] Options Flow Intelligence & Dark Pool Analytics
Wk 058	[Y2-Q1] Fine-Tuning LLM on Own Trade History (LoRA PEFT)
Wk 059	[Y2-Q1] Regime-Adaptive RL Allocation Agent (PPO)
Wk 060	[Y2-Q1] Go-Live: First Real Money Trades
Wk 061	[Y2-Q1] Scale-Up to 5 Contracts & Kelly Recalibration
Wk 062	[Y2-Q1] Multi-Broker Integration & Automatic Failover
Wk 063	[Y2-Q1] Futures & Index Hedging Integration
Wk 064	[Y2-Q1] Portfolio Rebalancing Engine & Capital Efficiency
Wk 065	[Y2-Q1] Institutional Reporting & FINRA Compliance
Wk 066	[Y2-Q2] Tax Lot Accounting & Wash Sale Compliance
Wk 067	[Y2-Q2] Crypto Correlation Signal Integration
Wk 068	[Y2-Q2] Macro Economic Indicator Dashboard

Wk 069 [Y2-Q2] IV Skew Trading Strategies
Wk 070 [Y2-Q2] Live Earnings Play Strategies
Wk 071 [Y2-Q2] Portfolio Insurance & Tail Risk Hedging
Wk 072 [Y2-Q2] AI Agent Self-Evaluation & Improvement Loop
Wk 073 [Y2-Q2] Multi-Timeframe Regime Analysis
Wk 074 [Y2-Q2] Client Reporting API & Partner Integration
Wk 075 [Y2-Q2] Advanced Backtesting: Stress Period Deep-Dives
Wk 076 [Y2-Q2] Volatility Forecasting with GARCH Models
Wk 077 [Y2-Q2] Options Market Making Simulation
Wk 078 [Y2-Q2] Q3 Performance Review & Y2H2 Planning
Wk 079 [Y2-Q3] Mobile App: React Native Dashboard
Wk 080 [Y2-Q3] Web3 Integration & On-Chain Trade Audit Trail
Wk 081 [Y2-Q3] Quantitative Research Platform & Jupyter Integration
Wk 082 [Y2-Q3] API Marketplace: Sell Data Feeds & Signals to Third Parties
Wk 083 [Y2-Q3] Enterprise Multi-Tenant Architecture for Multiple Users
Wk 084 [Y2-Q3] Black-Litterman Portfolio Optimization Model
Wk 085 [Y2-Q3] Regulatory Filing Automation (Form 4, 13F Preparation)
Wk 086 [Y2-Q3] AI-Powered Strategy Narrative Auto-Generator
Wk 087 [Y2-Q3] Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Forex)
Wk 088 [Y2-Q3] Distributed Computing: Ray Cluster for Parallel Backtests
Wk 089 [Y2-Q3] Automated A/B Testing Framework for Strategy Variants
Wk 090 [Y2-Q3] Production Hardening: Kubernetes, Helm Charts, Auto-Scaling
Wk 091 [Y2-Q3] Community Platform: Strategy Sharing Marketplace
Wk 092 [Y2-Q4] Beta Program: External User Onboarding (10 Beta Users)
Wk 093 [Y2-Q4] Public API v2.0 with GraphQL & WebSocket Subscriptions
Wk 094 [Y2-Q4] Advanced Options Analytics: Smile Arbitrage & Density Extraction
Wk 095 [Y2-Q4] Institutional Data Integration: Bloomberg B-PIPE Alternative
Wk 096 [Y2-Q4] Automated Earnings Straddle & Strangle Strategy Suite
Wk 097 [Y2-Q4] Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments
Wk 098 [Y2-Q4] Performance Attribution Engine: Brinson-Hood-Beebower Model
Wk 099 [Y2-Q4] Predictive Analytics Dashboard: ML-Powered Forward Metrics
Wk 100 [Y2-Q4] Two-Year Milestone: 10,000 Paper Trades Analysis & Insights
Wk 101 [Y2-Q4] Open Source Launch: Apache 2.0 License + GitHub Stars Campaign
Wk 102 [Y2-Q4] Premium Tier: SaaS Model Design & Stripe Billing Integration
Wk 103 [Y2-Q4] Final Integration Testing & Two-Year Performance Certification
Wk 104 [Y2-Q4] Two-Year Completion: Open Source Launch, 4-Year Roadmap & Celebration

Week 1: Project Scaffolding & Monorepo Setup

Lay the bedrock: toolchain, repo structure, CI/CD, coding standards, and dev environment.

Goal Zero-to-Running	Stack React+FastAPI	CI GitHub Actions	Test Target 100% bootstrap
-------------------------	------------------------	----------------------	-------------------------------

GOALS

- > Initialize monorepo with frontend (Vite+React+TS) and backend (FastAPI+Python 3.11) workspaces
- > Configure ESLint, Prettier, Black, isort, and pre-commit hooks across the full stack
- > Set up GitHub Actions CI pipeline: lint → unit test → build on every PR
- > Create Docker Compose dev environment (frontend:5173, backend:8000, SQLite volume)
- > Establish branch strategy: main, develop, feature/*, hotfix/*
- > Write CONTRIBUTING.md, architecture.md, and initial README with setup instructions

TASKS

- > Run: `npx create-vite@latest frontend --template react-ts`
- > Run: `cd frontend && npm i zustand @tanstack/react-query lightweight-charts`
- > Run: `pip install fastapi uvicorn sqlalchemy alembic pytest pytest-asyncio`
- > Create `phase1/main.py` with `/health` endpoint returning `{status:'ok',version:'0.1.0'}`
- > Create `docker-compose.dev.yml` with hot-reload volumes for both services
- > Write `.github/workflows/ci.yml`: on push/PR → lint + test + build
- > Set up SQLite with Alembic migrations; create initial schema migration
- > Create Makefile targets: `make dev`, `make test`, `make lint`, `make build`

COMMANDS & FILES

- | | |
|--|---|
| <ul style="list-style-type: none">-> <code>git init && git checkout -b main</code>-> <code>python3 -m venv venv && source venv/bin/activate</code>-> <code>pip install fastapi uvicorn sqlalchemy alembic pydantic</code>-> <code>npx create-vite@latest frontend -- --template react-ts</code>-> <code>alembic init alembic</code>-> <code>docker run -d --name redis -p 6379:6379 redis:alpine</code>-> <code>uvicorn apps.api.main:app --reload --port 8000</code> | <ul style="list-style-type: none">-> <code>apps/api/main.py</code> — FastAPI app entry point-> <code>apps/api/database.py</code> — SQLAlchemy engine + session factory-> <code>apps/api/models/</code> — SQLAlchemy ORM models directory-> <code>apps/api/schemas/</code> — Pydantic request/response schemas-> <code>.env.example</code> — template for all required env vars-> <code>.gitignore</code> — exclude <code>venv</code>, <code>.env</code>, <code>*.db</code>, <code>__pycache__</code>-> <code>docker-compose.yml</code> — Redis + Postgres services-> Makefile — shortcuts: <code>make dev</code>, <code>make test</code>, <code>make lint</code> |
|--|---|

API ENDPOINTS TO BUILD

- > GET `/health` — liveness probe returning `{status: ok, ts: ...}`
- > GET `/api/v1/status` — system info: db connected, redis connected, version

TESTS TO WRITE

- > `test_health.py`: assert GET `/health` returns 200 + `{status: ok}`
- > `test_db.py`: assert SQLAlchemy can create/drop all tables
- > `test_env.py`: assert all required env vars present on startup

ENV VARS & DEPENDENCIES

- | | |
|---|---|
| <ul style="list-style-type: none">-> <code>DATABASE_URL=sqlite:///apex.db</code>-> <code>REDIS_URL=redis://localhost:6379/0</code>-> <code>SECRET_KEY=</code>-> <code>ENV=development</code> | <ul style="list-style-type: none">-> <code>fastapi>=0.104</code>-> <code>uvicorn[standard]</code>-> <code>sqlalchemy>=2.0</code>-> <code>alembic</code>-> <code>pydantic>=2.0</code>-> <code>python-dotenv</code>-> <code>redis</code>-> <code>pytest</code>-> <code>httpx</code> (test client) |
|---|---|

DELIVERABLES

- > Monorepo runs with single 'make dev' command
- > CI passes green on first commit
- > Health endpoint returns valid JSON
- > Pre-commit hooks block bad code from entering repo
- > README has copy-paste setup instructions verified on a clean machine

AUTOPILOT LOGIC

- > No autopilot features yet — this week is pure infrastructure
- > Design autopilot config schema (JSON/YAML) that will govern all future AI decisions
- > Stub out AutopilotEngine class with run_cycle() method that returns a noop RunArtifact
- > Wire kill_switch boolean to config — even in week 1, the kill switch exists and is ON

PITFALLS & HOW TO AVOID

- > Don't commit .env — add it to .gitignore on Day 1
- > SQLite is fine for dev; plan Postgres migration path now
- > Use Pydantic v2 settings not raw os.getenv() for env validation
- > Set PYTHONPATH=. in Makefile to avoid import errors in tests

SUCCESS METRICS

- > GET /health returns 200 in < 50ms
- > Alembic migrations run clean with zero errors
- > All 3 tests pass in CI
- > Docker Compose brings up Redis + API in < 10s

RISK

- > Risk: Dev environment drift between team members → mitigate with Docker Compose
- > Risk: CI flakiness on first setup → keep CI pipeline minimal (lint + smoke test only)
- > Risk: Wrong Python version → pin to 3.11 in runtime.txt and .python-version

DAY-BY-DAY BREAKDOWN

- > Mon: Initialize repos, install toolchains, verify Python/Node versions
- > Tue: Configure linters, pre-commit hooks, and write CONTRIBUTING.md
- > Wed: Create FastAPI skeleton + SQLite + Alembic; verify /health endpoint
- > Thu: Set up React+Vite+TS; create App.tsx with placeholder dashboard
- > Fri: Wire up GitHub Actions CI; Docker Compose; tag v0.1.0 milestone

Week 2: Database Schema & Core Data Models

Design production-grade SQLite schema for trades, positions, options, runs, and audit logs.

Tables 8 core	Migrations Alembic	Test Coverage >90%	Uptime Local dev
------------------	-----------------------	-----------------------	---------------------

GOALS

- > Design and implement all core database tables: trades, positions, options_cache, autopilot_runs, bars, alerts, incidents, config
- > Write Alembic migration for every table with proper indices and foreign keys
- > Create SQLAlchemy ORM models and Pydantic schemas for API I/O
- > Implement repository pattern: one class per domain (TradeRepo, PositionRepo, RunRepo)
- > Write 30+ unit tests for all repository CRUD operations using pytest + SQLite in-memory
- > Document schema decisions in docs/schema.md with ER diagram (ASCII)

TASKS

- > Create models.py with SQLAlchemy declarative_base() and all 8 table classes
- > Define trades table: id, symbol, strategy_type, direction, qty, entry_price, exit_price, pnl, status, opened_at, closed_at
- > Define autopilot_runs table: run_id UUID, timestamp, duration_ms, success, phase, candidates_count, orders_placed, error_json
- > Define positions table: position_id, symbol, qty, avg_cost, current_price, unrealized_pnl, strategy_type, broker_ref
- > Define options_cache table: symbol, expiration, strike, option_type, bid, ask, iv, delta, gamma, theta, vega, cached_at
- > Define bars table: symbol, tf, ts, open, high, low, close, volume — with composite index on (symbol, tf, ts)
- > Create alembic/versions/001_initial_schema.py migration
- > Write repos/trade_repo.py with create, get, list, update_status, get_stats methods
- > Write repos/run_repo.py with save_run, list_runs, get_run methods
- > Write tests/test_repos.py with 30+ parametrized CRUD tests

COMMANDS & FILES

- | | |
|--|--|
| -> alembic revision --autogenerate -m 'initial schema' | -> apps/api/models/trade.py — Trade ORM model |
| -> alembic upgrade head | -> apps/api/models/position.py — Position ORM model |
| -> python3 -c 'from apps.api.database import engine; from apps.api.models import Base; Base.metadata.create_all(engine)' | -> apps/api/models/run_artifact.py — RunArtifact (autopilot audit log) |
| -> sqlite3 apex.db '.schema' head -100 | -> apps/api/models/symbol.py — Universe Symbol model |
| -> python3 scripts/seed_data.py # insert test symbols | -> apps/api/models/alert.py — Alert model |
| | -> alembic/versions/001_initial.py — first migration |
| | -> scripts/seed_data.py — insert 50 test symbols + mock trades |

API ENDPOINTS TO BUILD

- > GET /api/v1/trades — paginated trade history
- > GET /api/v1/trades/{id} — single trade detail
- > GET /api/v1/positions — open positions
- > GET /api/v1/universe — symbol list with metadata

TESTS TO WRITE

- > test_models.py: create Trade record, assert DB roundtrip
- > test_migrations.py: alembic upgrade head runs clean
- > test_schema_integrity.py: all expected columns exist with correct types
- > test_seed.py: seed script inserts 50 symbols without errors

ENV VARS & DEPENDENCIES

- | | |
|---|--------------------------|
| -> DATABASE_URL=sqlite:///apex.db (upgrade to postgres later) | -> sqlalchemy>=2.0 |
| | -> alembic |
| | -> faker (for seed data) |

DELIVERABLES

- > alembic upgrade head runs cleanly from scratch
- > All 8 tables created with proper constraints and indices
- > 30+ repo tests pass with 100% coverage on repo layer
- > Schema documented in docs/schema.md

AUTOPILOT LOGIC

- > RunArtifact dataclass mirrors autopilot_runs table exactly — parity is enforced by test
- > audit_decision() function stub: accepts RunArtifact, writes to DB, returns run_id
- > Config table seeded with default: max_positions=4, max_risk_per_trade=50, kill_switch=true
- > Implement get_config() / set_config() helpers used by all future autopilot code

PITFALLS & HOW TO AVOID

- > Name FK columns as _id not just id to avoid joins confusion
- > Add created_at/updated_at to every model using server_default=func.now()
- > Store monetary values as INTEGER cents, not FLOAT — avoids rounding bugs
- > Index symbol + date columns you'll query in hot paths

SUCCESS METRICS

- > All models migrate in < 1s
- > Seed script completes in < 2s
- > Trade roundtrip (insert+query) < 5ms
- > Schema has indexes on trades.symbol, trades.created_at

RISK

- > Risk: Schema changes mid-development are costly → design schema fully upfront
- > Risk: SQLite locking under concurrent writes → use WAL mode (PRAGMA journal_mode=WAL)
- > Risk: Forgetting indices → add index on every foreign key and every filter column

DAY-BY-DAY BREAKDOWN

- > Mon: Design full ER diagram on paper; enumerate all columns and types
- > Tue: Write SQLAlchemy models and Alembic migration
- > Wed: Implement repository classes with full CRUD
- > Thu: Write all unit tests; achieve >90% coverage on DB layer
- > Fri: Document schema, run migration on fresh DB, tag Week 2 complete

Week 3: Backend API Layer & WebSocket Infrastructure

Build the full REST API surface and real-time WebSocket event bus powering the live dashboard.

Endpoints 25+	WS Events 5 types	Latency <50ms P99	Test Suite Integration
------------------	----------------------	----------------------	---------------------------

GOALS

- > Implement all 25+ REST endpoints across autopilot, portfolio, options, and charting domains
- > Build WebSocket event bus: clients subscribe to run_update, position_update, alert, log channels
- > Add API versioning (/api/v1/) and structured error responses with error codes
- > Implement request validation via Pydantic v2 with detailed error messages
- > Add rate limiting (slowapi) and CORS configuration for frontend dev server
- > Write integration tests for every endpoint using httpx async client

TASKS

- > Create routers: autopilot_router, portfolio_router, options_router, chart_router, admin_router
- > POST /api/v1/autopilot/cycle — trigger a cycle, return RunArtifact
- > GET /api/v1/autopilot/status — returns engine state, kill_switch, last_run_id
- > GET /api/v1/autopilot/positions — open positions with real-time P&L;
- > GET /api/v1/autopilot/runs?limit=20 — paginated run history
- > GET /api/v1/autopilot/config — current config JSON
- > PATCH /api/v1/autopilot/config — update config fields
- > POST /api/v1/autopilot/kill — activate kill switch
- > GET /api/v1/options/chain?symbol=AAPL&expiration;=2025-02-21 — options chain
- > GET /api/v1/bars/{symbol}?tf=1D&limit;=200 — OHLCV bars
- > Implement WebSocket /ws endpoint: broadcast events to all connected clients
- > Add ConnectionManager class: track clients, handle disconnect, broadcast JSON events
- > Write tests/test_api.py with 40+ async httpx tests — mock DB calls with pytest-mock

COMMANDS & FILES

- | | |
|---|---|
| -> pip install alpaca-py | -> services/broker/alpaca_client.py — Alpaca paper API wrapper |
| -> python3 -c 'from alpaca.trading.client import TradingClient; c=TradingClient(KEY,SECRET,paper=True); print(c.get_account())' | -> services/broker/interface.py — abstract BrokerInterface base class |
| -> python3 scripts/test_market_data.py AAPL | -> services/broker/mock_broker.py — deterministic mock for tests |
| -> python3 -m pytest tests/test_broker.py -v | -> services/market_data/alpaca_feed.py — real-time bars + quotes |
| | -> scripts/test_market_data.py — CLI: fetch last 10 bars for a symbol |
| | -> tests/test_broker.py — broker interface contract tests |

API ENDPOINTS TO BUILD

- > GET /api/v1/broker/account — buying power, equity, cash
- > GET /api/v1/broker/orders — recent order history
- > POST /api/v1/broker/orders — place new order (paper only)
- > DELETE /api/v1/broker/orders/{id} — cancel order

TESTS TO WRITE

- > test_broker_account.py: mock broker returns correct buying_power
- > test_order_placement.py: place limit order, assert order_id returned
- > test_market_data.py: fetch AAPL bars, assert OHLCV fields present
- > test_broker_interface.py: MockBroker satisfies BrokerInterface contract

ENV VARS & DEPENDENCIES

-> ALPACA_API_KEY=your_paper_key	-> alpaca-py>=0.8
-> ALPACA_SECRET_KEY=your_paper_secret	-> websockets
-> ALPACA_PAPER=true	-> aiohttp
-> ALPACA_BASE_URL=https://paper-api.alpaca.markets	

DELIVERABLES

- > All 25+ endpoints return correct responses with proper HTTP status codes
- > WebSocket connects and receives live events when autopilot cycle runs
- > Integration test suite passes in <30 seconds
- > API documented at /docs (Swagger) and /redoc
- > Rate limiting prevents >100 req/min per IP

AUTOPILOT LOGIC

- > POST /api/v1/autopilot/cycle is the beating heart — it calls engine.run_cycle()
- > All cycle events (phase transitions, trade decisions, errors) emitted via WebSocket
- > cycle response includes: run_id, duration_ms, candidates_considered, orders_placed, no_action_reasons
- > Kill switch endpoint returns 200 immediately; engine checks it at every phase boundary

PITFALLS & HOW TO AVOID

- > Never hardcode API keys — load from .env only
- > Paper vs live endpoint URLs are different — always check ALPACA_PAPER flag
- > Alpaca rate limit: 200 requests/minute — add retry with exponential backoff
- > Market hours check before any order — reject outside 9:30-16:00 ET

SUCCESS METRICS

- > Account balance fetched in < 500ms
- > Paper order placed and confirmed in < 2s
- > MockBroker used in 100% of unit tests (no live calls in CI)
- > Market data fetch returns ≥ 10 bars for liquid symbols

RISK

- > Risk: WebSocket memory leak on client disconnect → implement proper cleanup in ConnectionManager
- > Risk: Blocking DB calls in async context → use asyncio.to_thread() for all SQLAlchemy calls
- > Risk: CORS misconfiguration blocking frontend → test with actual Vite dev server origin

DAY-BY-DAY BREAKDOWN

- > Mon: Create router files; implement all autopilot endpoints with stub responses
- > Tue: Implement options and chart endpoints; wire to real DB repos
- > Wed: Build WebSocket ConnectionManager; integrate event broadcasting into cycle
- > Thu: Write all integration tests; fix any contract mismatches found
- > Fri: Add rate limiting, CORS, docs verification; tag Week 3 complete

Week 4: React Frontend Shell & Live Dashboard

Build the core UI: charting shell, WebSocket provider, Zustand state, and the unified dashboard.

Components 20+	WS Connected Yes	Render FPS ≥60	Unit Tests 22 pass
-------------------	---------------------	-------------------	-----------------------

GOALS

- > Build full React app shell: router, layout, sidebar navigation, theme provider
- > Implement Zustand stores for: autopilot state, positions, orders, config, events
- > Create WebSocket context provider that auto-reconnects and dispatches events to stores
- > Build UnifiedDashboardView with Supergraph chart, AIPanel, positions widget, orders widget
- > Add lightweight-charts candlestick chart with real API data from /api/v1/bars/{symbol}
- > Achieve 22 passing Vitest unit tests on all store logic and utility functions

TASKS

- > npm install zustand @tanstack/react-query lightweight-charts react-router-dom
- > Create stores/autopilotStore.ts: status, lastRun, killSwitch, cycle() action
- > Create stores/positionsStore.ts: positions[], totalUnrealized, fetchPositions() action
- > Create providers/WebSocketProvider.tsx: connects to /ws, routes events to stores
- > Create features/layout/AppShell.tsx: sidebar + top nav + outlet
- > Create features/chart/SupergraphModule.tsx: lightweight-charts with candles + trade markers
- > Create features/autopilot/AIPanel.tsx: start/stop button, status, last run summary
- > Create features/trading/tiles/PositionsTile.tsx: table with live P&L; updates
- > Write 22 Vitest tests across stores and utility functions
- > Configure Vite proxy /api → http://localhost:8000 for dev

COMMANDS & FILES

- | | |
|---|--|
| -> cd frontend && npm install && npm run dev | -> frontend/src/App.tsx — root router with 5 main routes |
| -> npm install react-router-dom zustand @tanstack/react-query axios | -> frontend/src/store/useAppStore.ts — Zustand global state |
| -> npm install lightweight-charts recharts | -> frontend/src/lib/api.ts — Axios client with base URL + auth interceptor |
| -> npm run build && npm run preview | -> frontend/src/pages/Dashboard.tsx — main overview page |
| | -> frontend/src/pages/Positions.tsx — live positions table |
| | -> frontend/src/components/NavSidebar.tsx — left nav with route links |
| | -> frontend/src/components/StatusBar.tsx — top bar: market status, P&L, time |

API ENDPOINTS TO BUILD

- > All frontend calls go through api.ts client — no direct fetch() calls

TESTS TO WRITE

- > npm run test: Vitest unit tests for store slices
- > test_app_store.ts: setPositions updates positions slice correctly
- > test_api_client.ts: Axios interceptor attaches Authorization header
- > playwright/test_navigation.ts: all 5 nav links render without 404

ENV VARS & DEPENDENCIES

- | | |
|--|----------------------------|
| -> VITE_API_BASE_URL=http://localhost:8000 | -> react-router-dom@6 |
| -> VITE_WS_URL=ws://localhost:8000/ws | -> zustand@4 |
| | -> @tanstack/react-query@5 |
| | -> axios |
| | -> lightweight-charts@4 |

-> recharts
-> lucide-react

DELIVERABLES

- > Dashboard renders at localhost:5173 with real data from backend
- > Candlestick chart loads AAPL bars on mount
- > AIPanel shows autopilot status; clicking Run Cycle calls POST /cycle
- > WebSocket events update positions in real time without polling
- > 22 Vitest unit tests pass in <5 seconds

AUTOPILOT LOGIC

- > AIPanel is the user's entire interaction with autopilot — it must be clear and confidence-inspiring
- > Last Run Summary card shows: timestamp, candidates_considered, orders_placed, duration_ms
- > Kill switch button is RED and prominent — one click, confirmed with modal
- > Status indicator: ● Running (green), ● Idle (grey), ● Killed (red) — visible on every page

PITFALLS & HOW TO AVOID

- > Don't mix useState + Zustand for same data — pick one source of truth
- > React Query handles caching — don't duplicate in Zustand
- > Set up path aliases in vite.config.ts: @/ maps to src/
- > Use React Error Boundary at page level to prevent full app crash

SUCCESS METRICS

- > npm run dev starts in < 3s
- > npm run build completes in < 30s with no errors
- > All 5 pages render without console errors
- > Lighthouse performance score ≥ 80 on Dashboard

RISK

- > Risk: lightweight-charts version mismatch with React 19 → pin to ^4.1.0 and test
- > Risk: WebSocket reconnect storm on server restart → implement exponential backoff (1s,2s,4s,8s,max30s)
- > Risk: Zustand store getting stale across navigation → use subscribeWithSelector middleware

DAY-BY-DAY BREAKDOWN

- > Mon: Scaffold React app; install deps; create router and AppShell layout
- > Tue: Build Zustand stores and WebSocket provider; wire to backend
- > Wed: Build SupergraphModule with lightweight-charts; load real bar data
- > Thu: Build AIPanel, PositionsTile; RunArtifact display card
- > Fri: Write 22 unit tests; verify full stack E2E; tag Week 4 complete

Week 5: Alpaca & Tradier Integration — Live Paper Trading

Connect to real broker and data APIs; achieve live paper trade execution end-to-end.

Broker Alpaca Paper	Data Tradier Options	Exec Latency <2s	Auth API Key + Env
------------------------	-------------------------	---------------------	-----------------------

GOALS

- > Implement AlpacaBroker class: submit_order, cancel_order, get_positions, get_account, get_orders
- > Implement TradierOptionsProvider: get_chain, get_quotes, get_expirations with 60s cache
- > Add keys.env pattern with validation at startup — missing keys = hard fail with message
- > Write paper trade execution test: place a paper order on Alpaca and verify fill
- > Implement retry logic with exponential backoff for all external API calls
- > Health-check endpoints verify broker connectivity; red status if any API is unreachable

TASKS

- > pip install alpaca-trade-api requests-cache
- > Create services/execution/alpaca_broker.py with full interface
- > Create services/options/tradier_provider.py with caching decorator
- > Implement validate_env() at app startup: check all required env vars
- > Write integration test: call alpaca.get_account() and assert equity > 0
- > Write integration test: call tradier.get_chain('AAPL') and assert len > 0
- > Add /health endpoint to check: DB ✓, Alpaca ✓, Tradier ✓, LLM ✓
- > Wire broker health into autopilot engine: abort cycle if broker not reachable

COMMANDS & FILES

- | | |
|--|--|
| -> pip install tradier pandas numpy | -> services/options/chain_fetcher.py — fetch full options chain from Tradier |
| -> python3 scripts/fetch_options_chain.py AAPL 2024-03-15 | -> services/options/iv_calculator.py — Black-Scholes IV solver (Newton-Raphson) |
| -> python3 scripts/calc_iv.py AAPL # implied vol calculation | -> services/options/greeks_calculator.py — Delta, Gamma, Theta, Vega, Rho |
| -> python3 -m pytest tests/test_options_engine.py -v | -> services/options/spread_builder.py — build vertical/IC/calendar spreads |
| | -> scripts/fetch_options_chain.py — CLI to fetch + print chain for symbol/expiry |
| | -> scripts/calc_iv.py — CLI to compute IV for current chain |
| | -> tests/test_options_engine.py — contract tests for greeks and spread math |

API ENDPOINTS TO BUILD

- > GET /api/v1/options/chain/{symbol} — current chain with Greeks
- > GET /api/v1/options/iv/{symbol} — IV rank + percentile
- > GET /api/v1/options/spreads/{symbol} — pre-built spread candidates

TESTS TO WRITE

- > test_bs_greeks.py: Delta of ATM call ≈ 0.50 within tolerance
- > test_iv_solver.py: IV solver converges for known price/strike pairs
- > test_spread_builder.py: iron condor has 4 legs, net credit > 0
- > test_chain_fetcher.py: mock Tradier returns parsed OptionChain object

ENV VARS & DEPENDENCIES

- | | |
|---|--------------------------------|
| -> TRADIER_TOKEN=your_sandbox_token | -> tradier (requests wrapper) |
| -> TRADIER_BASE_URL=https://sandbox.tradier.com | -> numpy |
| | -> scipy (for IV root-finding) |
| | -> pandas |

DELIVERABLES

- > make test-integration passes with real API keys in environment
- > Paper order placed and visible in Alpaca dashboard
- > Tradier options chain returned with Greeks in <500ms
- > /health shows all green services
- > Startup fails loudly with clear error if keys are missing

AUTOPILOT LOGIC

- > Autopilot now has a real execution surface — `paper_broker.py` wraps `AlpacaBroker` for simulation
- > `PaperBroker` tracks virtual positions in SQLite when Alpaca paper isn't available
- > `BrokerVerification` dataclass: `is_alpaca_live`, `is_tradier_live`, `account_equity`, `buying_power`
- > Engine aborts if `BrokerVerification` fails — never trade without confirmed connectivity

PITFALLS & HOW TO AVOID

- > Black-Scholes assumes European options — use binomial for American calls on dividends
- > IV can be negative with bad inputs — clamp solver to `[0.001, 20.0]` range
- > Options chains have thousands of rows — filter to 0.1-0.9 delta strikes only
- > Tradier sandbox has real chains but no live fills — perfect for testing

SUCCESS METRICS

- > IV solver converges in < 10ms per contract
- > Full chain for SPY fetched and parsed in < 2s
- > Greeks accurate to 3 decimal places vs reference model
- > Spread builder generates ≥ 3 IC candidates for SPY

RISK

- > Risk: API keys committed to git → `keys.env` in `.gitignore` from day 1, pre-commit hook blocks secrets
- > Risk: Tradier rate limit (120/min) hit during option chain scan → token bucket implementation
- > Risk: Alpaca paper API throttled → add jitter to retry delays

DAY-BY-DAY BREAKDOWN

- > Mon: Implement `AlpacaBroker`; write `account/position/order` methods
- > Tue: Implement `TradierProvider` with caching; verify options chain
- > Wed: Integrate both into `/health`; add startup validation
- > Thu: Write integration tests with real APIs; verify paper order
- > Fri: Wire broker health into cycle; document API key setup

Week 6: Options Chain Engine & Strategy Templates

Build the full options scanning engine: chain fetching, Greeks math, and the 4 strategy templates.

Strategies 4 templates	Greeks All 4	Chain Speed <500ms	Candidates 50+ per scan
---------------------------	-----------------	-----------------------	----------------------------

GOALS

- > Implement full options chain processor: normalize Tradier response into internal OptionContract model
- > Implement Black-Scholes Greeks verification: compute delta/gamma/theta/vega independently and diff against Tradier
- > Build 4 strategy templates: PutCreditSpread, CallCreditSpread, IronCondor, DebitSpread
- > Implement StrategyMatcher: given a chain, generate all valid spread candidates matching template parameters
- > Build liquidity filter: reject any spread where bid_ask_spread_pct > 10%
- > Add IV Rank calculator: compare current IV to 52-week IV range

TASKS

- > pip install numpy scipy
- > Create models/option_contract.py: symbol, expiration, strike, option_type, bid, ask, iv, delta, gamma, theta, vega
- > Create services/options/greeks.py: black_scholes_price(), compute_delta(), compute_gamma(), compute_theta(), compute_vega()
- > Create strategies/put_credit_spread.py: generate candidates given chain, target_delta, min_dte, max_dte
- > Create strategies/iron_condor.py: symmetric wings around ATM
- > Create strategies/call_credit_spread.py and debit_spread.py
- > Create services/options/iv_rank.py: fetch 52w high/low IV, compute current rank 0-100
- > Write test_strategies.py: for each template, generate candidates from mock chain and assert validity

COMMANDS & FILES

- | | |
|---|---|
| -> pip install langchain-groq langchain-google-genai langchain-core | -> services/llm/groq_client.py — Groq Llama3 async client |
| -> python3 scripts/test_llm.py # test Groq + Gemini round-trip | -> services/llm/gemini_client.py — Google Gemini async client |
| -> python3 scripts/score_candidates.py --dry-run --symbols AAPL,SPY,QQQ | -> services/llm/consensus_scorer.py — dual LLM scoring + agreement logic |
| | -> services/llm/prompts/trade_score_prompt.txt — structured scoring prompt |
| | -> services/llm/prompts/risk_assessment_prompt.txt — risk evaluation prompt |
| | -> scripts/test_llm.py — send test prompt to both LLMs, print responses |
| | -> scripts/score_candidates.py — CLI to score symbol list |

API ENDPOINTS TO BUILD

- > POST /api/v1/llm/score — score a trade candidate (returns 0-100 + rationale)
- > GET /api/v1/llm/health — check both LLMs are responding

TESTS TO WRITE

- > test_groq_client.py: mock API returns score, assert parsed correctly
- > test_consensus.py: if both LLMs agree score > 70 → APPROVE
- > test_disagreement.py: if LLMs disagree by > 30pts → requires higher threshold
- > test_prompt_format.py: prompt template renders with all required fields

ENV VARS & DEPENDENCIES

- | | |
|-----------------------------------|---------------------------|
| -> GROQ_API_KEY=your_groq_key | -> langchain-groq |
| -> GEMINI_API_KEY=your_gemini_key | -> langchain-google-genai |
| -> LLM_TIMEOUT_SECONDS=30 | -> langchain-core |
| -> LLM_MIN_CONSENSUS_SCORE=70 | -> tiktoken |

DELIVERABLES

- > Given AAPL chain, generate 50+ spread candidates in <1s
- > All Greeks verified against Black-Scholes within ± 0.05 tolerance
- > IV Rank computed for 10 symbols without error
- > Strategy candidates include: symbol, legs[2-4], max_profit, max_loss, pop, dte, iv_rank
- > All strategy unit tests pass

AUTOPILOT LOGIC

- > CANDIDATE_GENERATION phase now has real content: iterate universe → fetch chain → match strategies
- > CandidateRecord dataclass has all fields autopilot needs: score, pop, iv_rank, max_loss, max_profit, dte
- > Autopilot only considers candidates where PoP ≥ 0.50 and liquidity_score ≥ 0.5
- > All strategy parameters configurable via autopilot config: target_delta, min_dte, max_dte per strategy type

PITFALLS & HOW TO AVOID

- > LLM responses are non-deterministic — always parse JSON, never regex
- > Set temperature=0.1 for scoring tasks to reduce variance
- > Groq rate limit: 30 req/min on free tier — add backoff + queue
- > Always validate LLM score is 0-100 integer — reject malformed responses

SUCCESS METRICS

- > Both LLMs respond in < 5s
- > Consensus score computed in < 10s total
- > LLM disagreement rate < 30% on same candidate set
- > Score correlation between Groq + Gemini > 0.75

RISK

- > Risk: Black-Scholes mismatch for deep ITM/OTM options → acceptable; log mismatches >0.15 delta diff
- > Risk: Stale chains used for execution → enforce max_chain_age=60s before submission

DAY-BY-DAY BREAKDOWN

- > Mon: Build OptionContract model; normalize Tradier chain response
- > Tue: Implement Black-Scholes Greeks; write verification tests
- > Wed: Build PutCreditSpread and IronCondor templates
- > Thu: Build CallCreditSpread and DebitSpread; implement IV Rank
- > Fri: Write all strategy unit tests; integrate into engine CANDIDATE_GENERATION phase

Week 7: Groq LLM Integration — Fast Candidate Ranking

Integrate Groq API as Stage 1 of the dual-LLM pipeline for rapid options candidate ranking.

LLM Groq llama3-70b	Latency <3s	Prompt Structured JSON	Fallback Deterministic
------------------------	----------------	---------------------------	---------------------------

GOALS

- > Integrate Groq API: groq-python client with structured JSON output mode
- > Build GroqProvider class with rank_candidates() method accepting CandidateRecord list
- > Design the Groq prompt: system context, market regime, portfolio state, candidate structured data
- > Parse and validate Groq response via Pydantic: selected_ids, confidence, explanation
- > Implement deterministic fallback: sort by composite_score desc, take top 5
- > Handle Groq failures gracefully: timeout (10s), rate limit, invalid JSON — all fall to deterministic

TASKS

- > pip install groq
- > Create services/llm/groq_provider.py with api_key from env, timeout=10s
- > Build GROQ_SYSTEM_PROMPT: role=options trading AI, constraints, output format
- > Build rank_candidates(candidates, context) → sends structured JSON, parses LLMResponse
- > Implement LLMResponse Pydantic model: selected_ids: List[str], confidence: float, explanation: str
- > Add validator: all selected_ids must exist in input candidates (hallucination guard)
- > Add validator: confidence must be ≥ 0.60, else treat as uncertain
- > Write unit test mocking Groq API: assert selected_ids validated, fallback triggered on bad response
- > Add llm_calls table to DB: provider, model, prompt_tokens, completion_tokens, latency_ms, success

COMMANDS & FILES

- | | |
|---|---|
| -> python3 scripts/run_risk_checks.py --symbol AAPL --qty 2 | -> services/risk/risk_manager.py — master risk gate (all checks in one class) |
| -> python3 scripts/compute_var.py --portfolio-file positions.json | -> services/risk/position_sizer.py — Kelly fraction + max position size |
| -> python3 -m pytest tests/test_risk_manager.py -v --tb=short | -> services/risk/portfolio_monitor.py — live P&L; + drawdown tracking |
| | -> services/risk/kill_switch.py — hard stop: liquidate all on trigger |
| | -> services/risk/gates/daily_loss_gate.py — reject if daily loss > limit |
| | -> services/risk/gates/position_count_gate.py — reject if too many open positions |
| | -> services/risk/gates/correlation_gate.py — reject correlated duplicate |
| | -> config/risk_params.yaml — all risk thresholds in one editable file |

API ENDPOINTS TO BUILD

- > GET /api/v1/risk/status — current risk metrics: drawdown, daily P&L, position count
- > POST /api/v1/risk/kill-switch — trigger emergency liquidation
- > GET /api/v1/risk/gates/{trade_id} — show which gates passed/failed for a trade

TESTS TO WRITE

- > test_daily_loss_gate.py: gate rejects after \$500 daily loss
- > test_max_positions.py: gate rejects 6th position when max=5
- > test_kill_switch.py: kill switch marks all positions for close
- > test_kelly_fraction.py: Kelly returns 0 for win_rate < 0.50

ENV VARS & DEPENDENCIES

- | | |
|---------------------------|-------------------------------|
| -> MAX_DAILY_LOSS_USD=500 | -> pydantic (gate validation) |
|---------------------------|-------------------------------|

-> MAX_OPEN_POSITIONS=5
-> MAX_PORTFOLIO_DRAWDOWN_PCT=10
-> POSITION_RISK_PCT=2

-> numpy (Kelly math)
-> pyyaml (config)

DELIVERABLES

- > GroqProvider returns valid selections for 20 mock candidates in <3s
- > Hallucination guard rejects IDs not in input — test case proves this
- > Deterministic fallback activates when Groq fails — test case proves this
- > All LLM calls logged to DB with token counts and latency
- > Sensitive prompt data never logged to stdout

AUTOPILOT LOGIC

- > SELECTION phase now: CANDIDATE_GENERATION → [50 candidates] → Groq → [top 8]
- > Groq context includes: market_regime, vix_level, portfolio.equity, portfolio.total_risk, candidates[]
- > Each candidate sent to Groq as: {id, symbol, template, pop, iv_rank, max_loss, dte, liquidity_score}
- > Groq explains its picks in natural language — explanation stored in RunArtifact for dashboard display

PITFALLS & HOW TO AVOID

- > Kill switch must work even if DB is down — write to a local file flag too
- > Risk params must be hot-reloadable — use watchdog to reload risk_params.yaml
- > Correlation gate: compare proposed trade to ALL open positions, not just same symbol
- > Daily loss resets at 4:00 PM ET — don't accidentally reset mid-day

SUCCESS METRICS

- > All 7 risk gates evaluated in < 100ms total
- > Kill switch completes within 5s of trigger
- > Risk gate pass/fail logged with reason for every trade attempt
- > Zero trades executed when any hard gate fails

RISK

- > Risk: Groq API down → deterministic fallback makes autopilot independent of LLM availability
- > Risk: LLM picks hallucinated tickers → ID validation catches this before any order is considered
- > Risk: Prompt injection via news data → sanitize all external text before including in prompt

DAY-BY-DAY BREAKDOWN

- > Mon: Install groq; build GroqProvider skeleton; test API connection
- > Tue: Design and iterate on system prompt for quality rankings
- > Wed: Implement Pydantic response validation; build all guard layers
- > Thu: Wire into engine SELECTION phase; run live test with real candidates
- > Fri: Write unit and integration tests; document LLM module

Week 8: Gemini LLM Integration — Deep Validation Layer

Add Gemini as Stage 2 validator: cross-checks Groq picks, provides deeper analysis, reduces false positives.

LLM Gemini 1.5 Flash	Agreement Gate Intersection≥1	Fallback Groq-only	Dual Cost Tracked
-------------------------	----------------------------------	-----------------------	----------------------

GOALS

- > Integrate Gemini API (google-generativeai): GeminiProvider.rank_candidates() method
- > Design Gemini prompt: receives Groq's top picks + Groq's explanation as additional context
- > Implement cross-validation logic: find intersection of Groq and Gemini selections
- > Handle all failure modes: both fail → deterministic, one fails → use other's output
- > Add dual-LLM agreement metric to RunArtifact: groq_picks, gemini_picks, agreed_picks, final_picks
- > Track combined LLM cost per cycle; alert if cost > \$0.01 per cycle

TASKS

- > pip install google-generativeai
- > Create services/llm/gemini_provider.py mirroring GroqProvider interface
- > Build Gemini prompt: 'Validate these options trades selected by another AI...'
- > Implement HybridSelector.select(): Groq → top8, Gemini → validate, intersect → final 2-3
- > Implement consensus_select(groq_ids, gemini_ids) → agreed set or deterministic if empty
- > Add to RunArtifact: llm_agreement: LLMAgreement(groq, gemini, agreed, final_source)
- > Write integration test: mock both LLMs agree on 2 candidates → assert those 2 returned
- > Write integration test: LLMs disagree → assert deterministic fallback fires

COMMANDS & FILES

- > python3 scripts/backtest_simple.py --strategy credit_spread --start 2022-01-01 --end 2023-01-01
 - > python3 scripts/generate_report.py --run-id latest
 - > python3 -m pytest tests/test_backtest.py -v
- > services/backtest/engine.py — vectorized backtest over historical chain data
 - > services/backtest/strategy_runner.py — run single strategy over date range
 - > services/backtest/metrics.py — Sharpe, win rate, max drawdown, profit factor
 - > services/backtest/report_builder.py — generate backtest report dict
 - > data/historical/ — downloaded OHLCV + options chain parquet files
 - > scripts/backtest_simple.py — CLI for single strategy backtest
 - > scripts/download_historical.py — bulk download historical data

API ENDPOINTS TO BUILD

- > POST /api/v1/backtest/run — start a backtest run (returns run_id)
- > GET /api/v1/backtest/{run_id} — fetch backtest results + metrics
- > GET /api/v1/backtest/history — list all backtest runs

TESTS TO WRITE

- > test_metrics.py: Sharpe formula correct on known return series
- > test_max_drawdown.py: correctly identifies worst drawdown period
- > test_backtest_engine.py: runs 1-year without error, returns metrics dict
- > test_no_lookahead.py: strategy never sees future bar data

ENV VARS & DEPENDENCIES

- > BACKTEST_DATA_DIR=./data/historical
 - > BACKTEST_DEFAULT_CAPITAL=10000
- > pandas
 - > numpy
 - > pyarrow (parquet)
 - > matplotlib (equity curve)

DELIVERABLES

- > Full dual-LLM pipeline operational: Groq ranks → Gemini validates → intersection selected
- > Agreement metric visible in RunArtifact and displayed on RunsAuditView
- > All four failure modes tested and working (both-ok, groq-only, gemini-only, both-fail)
- > LLM cost tracked per cycle in DB
- > Dashboard shows which LLM combo was used for each run

AUTOPILOT LOGIC

- > Pipeline: 50 candidates → Groq fast rank (top 8) → Gemini deep validate → 2-3 final picks
- > If LLMs disagree entirely: log warning, use deterministic fallback, flag run for human review
- > 'LLM-confirmed' badge appears on trades that passed both LLMs
- > llm_log table stores full prompt+response for every call — critical for audit and debugging

PITFALLS & HOW TO AVOID

- > Lookahead bias: signals must use only data available at bar close
- > Options data gaps (weekends, holidays) — handle with forward-fill carefully
- > Never use adjusted close for options backtest — use raw prices
- > Slippage model: assume 10% of bid-ask spread as execution cost minimum

SUCCESS METRICS

- > 1-year backtest completes in < 30s
- > Sharpe ratio > 0.8 on baseline credit spread strategy
- > Win rate > 55% on paper trades
- > Max drawdown < 15% of starting capital

RISK

- > Risk: Both LLMs agree on a bad trade → human audit sample (10% of runs reviewed weekly)
- > Risk: Gemini slower than expected → set 15s timeout; if slow, use Groq-only result
- > Risk: API costs spiral → cap at \$5/day; disable LLMs and use deterministic if exceeded

DAY-BY-DAY BREAKDOWN

- > Mon: Install google-generativeai; build GeminiProvider skeleton
- > Tue: Design Gemini prompt with Groq context injection
- > Wed: Build HybridSelector with consensus logic; test all four paths
- > Thu: Integrate into engine; add agreement metrics to RunArtifact
- > Fri: Write all tests; review LLM call logs; verify cost tracking

Week 9: Entry Scoring Algorithm & Risk Validation Layer

Implement the 7-factor entry scoring system (0-100) and all 12 risk validation gates.

Score Factors 7 weighted	Min Score 65/100	Risk Gates 12	Validation Pre-order
-----------------------------	---------------------	------------------	-------------------------

GOALS

- > Implement EntryScorer: compute composite 0-100 score from 7 weighted factors
- > Implement all 12 ValidationGates: risk_budget, max_positions, max_per_underlying, liquidity, spread_width, earnings_blackout, news_sentiment, regime_mismatch, dte_bounds, delta_bounds, cluster_concentration, symbol_filter
- > Wire scoring into engine SELECTION phase: candidates ranked by score, min 65 to proceed
- > Wire validation into VALIDATION phase: all 12 gates checked before any execution
- > Build rejection ledger: every rejected candidate logged with reason
- > Add composite score to RunArtifact and display in candidate explorer on dashboard

TASKS

- > Create services/autopilot/scoring.py: EntryScorer.calculate(candidate, sentiment, technicals, llm_conf)
- > Implement all 7 factors with exact weights from MASTER_PLAN.md
- > Create services/autopilot/validator.py: Validator.validate(candidate, portfolio, config, calendar, news)
- > Implement each gate as a method returning (passed: bool, gate: ValidationGate, reason: str)
- > Create RejectionRecord dataclass: candidate_id, gate, reason, timestamp
- > Persist all RejectionRecords to rejection_log DB table
- > Write 50+ unit tests for scoring: test each factor boundary condition
- > Write 12 unit tests for validator: one per gate, test pass and fail case

COMMANDS & FILES

- | | |
|---|--|
| -> pip install yfinance vaderSentiment newsapi-python | -> services/news/news_fetcher.py — NewsAPI + Yahoo Finance RSS reader |
| -> python3 scripts/fetch_news.py AAPL --last 24h | -> services/news/sentiment_analyzer.py — VADER + FinBERT sentiment scoring |
| -> python3 scripts/analyze_sentiment.py --symbol AAPL | -> services/universe/scanner.py — filter symbols by IV rank, volume, spread |
| -> python3 scripts/universe_scanner.py --min-iv-rank 50 | -> services/universe/universe_loader.py — load S&P; 500 symbol list from CSV |
| | -> data/universe/sp500.csv — static symbol list with sector metadata |
| | -> scripts/fetch_news.py — CLI: fetch recent headlines for a symbol |
| | -> scripts/universe_scanner.py — CLI: run full universe scan |

API ENDPOINTS TO BUILD

- > GET /api/v1/universe/scan — run live universe scan, return top 20 candidates
- > GET /api/v1/news/{symbol} — recent headlines + sentiment scores
- > GET /api/v1/universe/symbols — full symbol list with sector + metadata

TESTS TO WRITE

- > test_sentiment.py: positive earnings headline scores > 0.3 (VADER)
- > test_scanner.py: scanner returns symbols with IV rank > 50
- > test_universe_loader.py: SP500 loads 500 symbols correctly
- > test_news_fetcher.py: mock NewsAPI returns parsed Article objects

ENV VARS & DEPENDENCIES

- | | |
|----------------------------------|-------------------|
| -> NEWS_API_KEY=your_newsapi_key | -> yfinance |
| -> UNIVERSE_MIN_IV_RANK=40 | -> vaderSentiment |
| -> UNIVERSE_MIN_VOLUME=500000 | -> newsapi-python |

-> SENTIMENT_THRESHOLD=0.1

-> requests

-> beautifulsoup4

DELIVERABLES

- > EntryScorer returns correct score for 10 diverse test candidates
- > All factor boundaries produce correct scores (edge cases tested)
- > All 12 gates pass/fail correctly in unit tests
- > Rejection log populated after each cycle
- > Full scoring visible in dashboard candidate explorer view

AUTOPILOT LOGIC

- > Step-by-step scoring breakdown shown in AIPanel: which factors contributed how many points
- > Validation gate failure shown in RunArtifact.no_action_reasons with human-readable explanation
- > Score threshold configurable (default 65); adjust via config API without restart
- > Backtesting module later will replay scoring on historical candidates to verify edge

PITFALLS & HOW TO AVOID

- > NewsAPI free tier: 100 req/day — cache aggressively, avoid per-symbol calls
- > VADER is general-purpose — supplement with FinBERT for financial text
- > IV rank needs 52-week IV history — build and store this, don't compute live
- > Universe scan over 500 symbols: parallelize with asyncio.gather, not sequential

SUCCESS METRICS

- > Universe scan of 500 symbols completes in < 60s
- > Sentiment score computed in < 200ms per article
- > Top 20 candidates filtered correctly by IV rank + volume criteria
- > News cache hit rate > 80% in production

RISK

- > Risk: Over-tuning score weights → document weights with rationale; change only with backtest evidence
- > Risk: Earnings blackout bypass → calendar.days_until_earnings uses multiple data sources
- > Risk: Cluster concentration not enforced → count positions by sector; reject if >2 in same sector

DAY-BY-DAY BREAKDOWN

- > Mon: Design and document all 7 scoring factors with test cases
- > Tue: Implement EntryScorer; write 25 scoring unit tests
- > Wed: Implement Validator with all 12 gates
- > Thu: Write 12 gate unit tests; wire into engine phases
- > Fri: Add rejection log DB; expose scoring in dashboard; tag Week 9

Week 10: Exit Logic Automation & Position Monitor Loop

Fully automate position monitoring: 5-minute exit checks, stop-loss, profit targets, DTE exits.

Exit Triggers 5 types	Poll Interval 5 min	Latency <10s	Audit Every exit logged
--------------------------	------------------------	-----------------	----------------------------

GOALS

- > Implement `PositionMonitor.run_pass()`: evaluates all open positions against 5 exit triggers every 5 minutes
- > Implement all 5 exit triggers with exact priority order from `MASTER_PLAN.md`
- > Handle the 'roll' case: when strike defense decision is 'roll', generate replacement candidate automatically
- > Build `ExitRecord` with full audit trail: `position_id`, `trigger`, `entry_price`, `exit_price`, `pnl`, `held_days`
- > Wire monitoring pass into the daily scheduler (next week); for now, call via API endpoint
- > Test every exit trigger path with mock positions and market data

TASKS

- > Create `services/autopilot/monitoring.py`: `PositionMonitor` class
- > Implement `ExitTrigger` enum: `STOP_LOSS`, `NEWS_SHOCK`, `STRIKE_DEFENSE`, `DTE_THRESHOLD`, `PROFIT_TARGET`
- > `evaluate_exit(position, current_price, news, calendar)` → (`should_exit`, `reason`, `urgency`)
- > Evaluate stop-loss: `current_cost > 2x entry_credit` → `STOP_LOSS` urgent
- > Evaluate DTE: `dte < 7` → `DTE_THRESHOLD`
- > Evaluate profit: `profit >= 0.5 * max_profit` → `PROFIT_TARGET`
- > Evaluate news: `has_major_negative(symbol, 4h)` → `NEWS_SHOCK` urgent
- > Evaluate strike: underlying within 2% of `short_strike` → `ai_strike_defense()`
- > Write 25+ tests covering all triggers and edge cases
- > Create POST `/api/v1/monitoring/run` endpoint to trigger manually

COMMANDS & FILES

- | | |
|---|--|
| <ul style="list-style-type: none"> -> <code>python3 scripts/run_autopilot_dry_run.py</code> -> <code>python3 scripts/test_entry_score.py --symbol SPY</code> -> <code>python3 -m pytest tests/test_entry_logic.py -v</code> | <ul style="list-style-type: none"> -> <code>services/autopilot/pipeline.py</code> — master orchestrator: <code>scan→score→validate→execute</code> -> <code>services/autopilot/entry_scorer.py</code> — composite score (IV rank, sentiment, greeks, LLM) -> <code>services/autopilot/run_artifact.py</code> — log every cycle to DB with full audit trail -> <code>services/autopilot/scheduler.py</code> — <code>APScheduler</code>: pre-market scan at 9:15 AM ET -> <code>scripts/run_autopilot_dry_run.py</code> — single full cycle without order execution -> <code>scripts/test_entry_score.py</code> — CLI to score one symbol through the full pipeline |
|---|--|

API ENDPOINTS TO BUILD

- > POST `/api/v1/autopilot/run` — trigger single autopilot cycle (dry-run or live)
- > GET `/api/v1/autopilot/status` — current cycle status, last run time, next run time
- > GET `/api/v1/autopilot/runs` — history of all run artifacts
- > GET `/api/v1/autopilot/runs/{id}` — detailed run artifact with scores + decisions

TESTS TO WRITE

- > `test_pipeline_dry_run.py`: full pipeline completes without error in < 60s
- > `test_entry_score_weights.py`: score components sum to weighted total correctly
- > `test_run_artifact.py`: `RunArtifact` saved to DB with all required fields
- > `test_scheduler.py`: scheduler fires at correct ET time (mocked)

ENV VARS & DEPENDENCIES

- | | |
|--|--|
| <ul style="list-style-type: none"> -> <code>AUTOPILOT_MODE=dry_run</code> # or live -> <code>AUTOPILOT_RUN_TIME=09:15</code> | <ul style="list-style-type: none"> -> <code>apscheduler>=3.10</code> -> <code>pytz</code> |
|--|--|

-> ENTRY_SCORE_THRESHOLD=70

-> structlog (structured logging)

DELIVERABLES

- > All 5 exit triggers fire correctly on mock positions
- > ExitRecords persisted to exits table in DB
- > P&L; calculation accurate for credit spreads: $(\text{entry_credit} - \text{close_price}) \times \text{qty} \times 100$
- > Urgent exits logged at WARNING level; normal exits at INFO
- > Manual trigger endpoint works from dashboard

AUTOPILOT LOGIC

- > MONITORING phase runs FIRST in every cycle — exits before entries, always
- > Emergency exits (STOP_LOSS, NEWS_SHOCK) trigger instant execution regardless of market hours
- > Roll decision: AI Strike Defense queries LLM with position context + news sentiment
- > all_exit_actions[] in RunArtifact includes every position evaluated, not just exited ones

PITFALLS & HOW TO AVOID

- > Dry-run mode MUST be identical path to live — only differ at order submission
- > Run artifact must be saved BEFORE order execution, not after
- > APScheduler jobs survive app restart only if using persistent job store
- > Market holiday calendar must be checked before scheduling runs

SUCCESS METRICS

- > Full dry-run cycle completes in < 120s
- > RunArtifact logged for 100% of cycles
- > Entry score breakdown visible in RunArtifact JSON
- > Scheduler fires within 30s of target time

RISK

- > Risk: Exit order fails at broker → retry 3x with 5s backoff; escalate to kill switch if still failing
- > Risk: Stale position P&L; used for stop-loss check → always fetch fresh quote before evaluating
- > Risk: Monitor runs during market close → check market_is_open() before executing exits

DAY-BY-DAY BREAKDOWN

- > Mon: Design ExitTrigger priority system; write specs for each trigger
- > Tue: Implement PositionMonitor with stop-loss and profit target
- > Wed: Implement DTE, news shock, and strike defense triggers
- > Thu: Write 25+ unit tests; wire into engine MONITORING phase
- > Fri: Add manual trigger endpoint; test E2E with paper positions

Week 11: n8n Scheduler & Full Autonomous Daily Loop

Implement the full 4:00 AM → 4:05 PM daily schedule via n8n workflows; achieve true autonomous operation.

Schedule 5 workflows	Coverage 24hr loop	Docker Required	Autonomy 99% target
-------------------------	-----------------------	--------------------	------------------------

GOALS

- > Configure n8n with Docker (or APScheduler as fallback if Docker unavailable)
- > Implement 5 scheduled workflows: pre-market-prep, market-open, intraday-scan, position-monitor, eod-report
- > Build market hours awareness: is_market_open(), minutes_to_close(), next_open() functions
- > Implement APScheduler-based scheduler as WSL Docker fallback (runs in Python process)
- > Verify full daily loop runs autonomously: pre-market→open→scan→monitor→close→report
- > Add circuit breaker: if 3+ consecutive cycles fail, auto-activate kill switch and send alert

TASKS

- > pip install apscheduler
- > Create services/automation/scheduler.py with APScheduler AsyncIOScheduler
- > Add job: pre_market_prep() at 8:00 AM ET — warm data, check connectivity
- > Add job: first_scan() at 10:00 AM ET — full candidate scan + execution
- > Add job: intraday_scan() every 30 min 10:30→3:00 PM ET — continuous scanning
- > Add job: monitor_positions() every 5 min during market hours — exit checks
- > Add job: eod_report() at 4:05 PM ET — generate daily P&L; summary
- > Implement market_hours.py: NYSE calendar, is_market_open(), skip holidays
- > Add consecutive-failure counter with kill-switch auto-activation at threshold 3
- > Write n8n/workflows/*.json for Docker environment as alternative

COMMANDS & FILES

- | | |
|--|---|
| -> pip install prometheus_client grafana-client | -> services/monitoring/metrics.py — Prometheus Counter, Gauge, Histogram definitions |
| -> docker run -d -p 9090:9090 prom/prometheus | -> services/monitoring/health_checker.py — checks DB, Redis, broker, LLM connectivity |
| -> docker run -d -p 3000:3000 grafana/grafana | -> config/prometheus.yml — scrape config targeting localhost:8000/metrics |
| -> python3 scripts/push_test_metrics.py | -> config/grafana/dashboards/apex_overview.json — pre-built Grafana dashboard JSON |
| -> curl http://localhost:8000/metrics grep apex_ | -> scripts/push_test_metrics.py — generate fake metrics to verify Grafana rendering |

API ENDPOINTS TO BUILD

- > GET /metrics — Prometheus scrape endpoint (standard format)
- > GET /api/v1/health/deep — all service health checks with latency
- > GET /api/v1/health/live — liveness probe (k8s compatible)
- > GET /api/v1/health/ready — readiness probe (k8s compatible)

TESTS TO WRITE

- > test_metrics_endpoint.py: GET /metrics returns 200 + apex_trades_total
- > test_health_deep.py: all services healthy returns {status: all_ok}
- > test_counter_increments.py: trade counter increments after successful execution

ENV VARS & DEPENDENCIES

- | | |
|--------------------------------------|----------------------------|
| -> PROMETHEUS_ENABLED=true | -> prometheus_client>=0.19 |
| -> GRAFANA_URL=http://localhost:3000 | -> structlog |
| | -> psutil (system metrics) |

DELIVERABLES

- > APScheduler runs full daily loop autonomously in a single Python process
- > Market hours awareness: no scans on weekends or holidays
- > Circuit breaker activates kill switch after 3 consecutive failures
- > Daily log shows every scheduled job execution with timestamp and outcome
- > n8n workflows defined as JSON backup for Docker environments

AUTOPILOT LOGIC

- > This week completes the autonomous loop — the system now runs completely without human input
- > Scheduler logs each job start/end with: job_name, scheduled_time, actual_time, duration_ms, success
- > Pre-market prep includes: connectivity check, data warm-up, config reload from DB
- > EOD report includes: daily P&L, total trades, win rate, account equity, tomorrow schedule preview

PITFALLS & HOW TO AVOID

- > Prometheus metrics must be registered at module level not inside functions
- > Label cardinality: don't use trade_id as a label — only symbol, strategy, status
- > Grafana panels need at least 24h of data to show meaningful graphs
- > Health check endpoint must be unauthenticated for load balancer probing

SUCCESS METRICS

- > GET /metrics responds in < 50ms
- > Grafana dashboard shows live trade count, P&L, and risk metrics
- > Health check catches Redis disconnect within 10s
- > All apex_ custom metrics visible in Prometheus UI

RISK

- > Risk: Scheduler drift over time → use UTC internally, display in ET; verify with NYSE calendar library
- > Risk: Job overlaps (monitor fires during scan) → use job coalescing: skip if previous run still active
- > Risk: System clock is wrong in WSL → use ntpdate or chrony; verify clock on scheduler start

DAY-BY-DAY BREAKDOWN

- > Mon: Research APScheduler vs n8n; implement market_hours.py with NYSE calendar
- > Tue: Build scheduler.py with all 5 jobs and timezone handling
- > Wed: Implement circuit breaker; test failure escalation
- > Thu: Run full simulated day loop; verify all jobs fire at correct times
- > Fri: Write n8n JSON workflows; document autonomous operation; tag Week 11

Week 12: News Sentiment Engine & Event Classification

Build the FinBERT-powered sentiment pipeline with real-time news scoring and event classification.

Model FinBERT	Sources 5 feeds	Update Rate 5 min	Events Classified 3 tiers
------------------	--------------------	----------------------	------------------------------

GOALS

- > Integrate FinBERT (ProsusAI/finbert) for real-time financial sentiment analysis
- > Build multi-source news aggregator: Finnhub, SEC 8-K filings, and fallback to Yahoo Finance RSS
- > Implement composite sentiment formula with source credibility and recency decay
- > Build event classifier: HIGH (earnings/FDA/M&A;), MEDIUM (analyst/launch), LOW (macro commentary)
- > Wire sentiment into entry scoring (Factor 3) and exit logic (news_shock trigger)
- > Cache sentiment scores per symbol with 5-minute TTL to avoid redundant FinBERT calls

TASKS

- > pip install transformers torch finnhub-python feedparser
- > Create services/autopilot/news_provider.py: fetch_headlines(symbol, hours=24)
- > Create services/autopilot/news_sentiment.py: SentimentEngine with FinBERT
- > Implement score_headline(headline, source, published_at) → float (-1 to +1)
- > Implement aggregate_sentiment(symbol, hours=24) → CompositeScore with breakdown
- > Implement classify_event(symbol) → EventClassification: HIGH/MEDIUM/LOW
- > Add sentiment cache: Redis-compatible dict with TTL (use cachetools.TTLCache)
- > Wire sentiment into validator: reject if HIGH event within 2 days of expiration
- > Write tests: mock headlines, assert sentiment scores in expected range

COMMANDS & FILES

- | | |
|---|--|
| <ul style="list-style-type: none"> -> pip install python-jose[cryptography] passlib[bcrypt] python-multipart -> python3 scripts/create_admin_user.py -> python3 scripts/generate_jwt_secret.py -> curl -X POST http://localhost:8000/auth/login -d 'username=admin&password=test' | <ul style="list-style-type: none"> -> apps/api/auth/jwt_handler.py — JWT encode/decode with expiry -> apps/api/auth/dependencies.py — FastAPI Depends(get_current_user) -> apps/api/auth/router.py — POST /auth/login, POST /auth/refresh, POST /auth/logout -> apps/api/models/user.py — User model with hashed_password -> scripts/create_admin_user.py — CLI to create initial admin user -> scripts/generate_jwt_secret.py — print 32-byte hex secret -> frontend/src/hooks/useAuth.ts — auth state hook + token refresh logic |
|---|--|

API ENDPOINTS TO BUILD

- > POST /auth/login — returns {access_token, refresh_token, expires_in}
- > POST /auth/refresh — exchange refresh token for new access token
- > POST /auth/logout — blacklist token in Redis
- > GET /auth/me — current user info

TESTS TO WRITE

- > test_login.py: valid credentials return JWT access token
- > test_invalid_login.py: wrong password returns 401
- > test_protected_route.py: GET /api/v1/trades requires valid JWT
- > test_token_refresh.py: refresh token returns new access token

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none"> -> JWT_SECRET_KEY=<32 byte hex> -> JWT_ALGORITHM=HS256 | <ul style="list-style-type: none"> -> python-jose[cryptography] -> passlib[bcrypt] |
|---|--|

-> ACCESS_TOKEN_EXPIRE_MINUTES=60
-> REFRESH_TOKEN_EXPIRE_DAYS=30

-> python-multipart

DELIVERABLES

- > FinBERT running and scoring headlines in <500ms each
- > Composite sentiment returned for 10 test symbols
- > Event classification correctly identifies earnings as HIGH
- > Sentiment cache reduces FinBERT calls by 90% in steady state
- > News shock exit trigger fires when HIGH event detected

AUTOPILOT LOGIC

- > Sentiment is now a real-time signal, not a static assumption
- > SentimentSnapshot stored in RunArtifact: per-symbol sentiment scores at time of cycle
- > Pre-market prep fetches and caches sentiment for all universe symbols — hot cache for scan
- > Factor 3 in entry score uses cached sentiment → score reflects current market narrative

PITFALLS & HOW TO AVOID

- > Store refresh tokens in Redis with TTL — can then invalidate on logout
- > Never put sensitive data in JWT payload — just user_id + role
- > Set SameSite=Strict + HttpOnly on cookie if storing tokens in cookies
- > CORS must be configured to allow only your frontend origin

SUCCESS METRICS

- > Login endpoint responds in < 300ms
- > JWT validation adds < 2ms overhead per request
- > 100% of /api/v1/* routes require valid JWT (verified by test)
- > Refresh token rotation working (old token invalidated after refresh)

RISK

- > Risk: FinBERT first load is slow (model download) → pre-load at startup; warm model in pre-market prep
- > Risk: Finnhub rate limit → free tier 60 calls/min; implement symbol batching
- > Risk: Sentiment model wrong on domain-specific jargon → test on 100 known headlines; accept ~80% accuracy

DAY-BY-DAY BREAKDOWN

- > Mon: Install transformers; test FinBERT on 20 sample headlines manually
- > Tue: Build news_provider.py with Finnhub + RSS fallback; normalize output
- > Wed: Build SentimentEngine with composite formula and recency decay
- > Thu: Implement event classifier; wire into validator and exit logic
- > Fri: Write tests; benchmark sentiment cycle time; tag Week 12

Week 13: Kill Switch, Kill Switch Recovery & Incident System

Build a bulletproof emergency stop system with auto-recovery protocols and full incident tracking.

Kill Switch <1s response	Auto-Recovery 3 levels	Incident Log Full trail	Alerts Email+Slack
-----------------------------	---------------------------	----------------------------	-----------------------

GOALS

- > Harden kill switch: one API call → all scheduled jobs paused, no new orders, all WS clients notified
- > Build 3-tier auto-recovery: Level 1 (retry once), Level 2 (restart service), Level 3 (kill switch + alert)
- > Implement incident tracking: every error captured as IncidentRecord with full stack trace
- > Build alert system: Email (SMTP) and Slack webhook notifications for critical events
- > Write runbook: step-by-step procedures for every known failure mode
- > Load test recovery: simulate broker failure, data feed failure, LLM failure — verify correct response

TASKS

- > Add kill_switch_activated_at column to config; broadcast ws event on activation
- > Implement /api/v1/admin/kill (POST) and /api/v1/admin/resume (POST)
- > Create services/incidents/incident_tracker.py: capture_incident(error, context, severity)
- > Create IncidentRecord: id, timestamp, severity (FATAL/HIGH/MEDIUM), error_type, stack_trace, resolved_at
- > Create services/alerts/alert_service.py: send_email(subject, body), send_slack(message)
- > Implement recovery pipeline: on cycle error → Level1 retry → Level2 service restart → Level3 kill
- > Write scripts/runbook.md with procedures for every failure mode
- > Test: simulate 3 consecutive failures → assert kill switch activates and alert sent

COMMANDS & FILES

- | | |
|---|---|
| -> python3 -m pytest tests/ -v --cov=services --cov-report=html | -> tests/integration/test_full_pipeline.py — end-to-end pipeline integration test |
| -> python3 scripts/run_q1_integration_test.py | -> tests/integration/test_api_endpoints.py — all API endpoints smoke test |
| -> python3 scripts/run_full_dry_run.py --verbose | -> scripts/run_q1_integration_test.py — Q1 completion checklist runner |
| -> open coverage/index.html | -> docs/Q1_RETROSPECTIVE.md — what was built, gaps, lessons learned |
| | -> .github/workflows/ci.yml — GitHub Actions: lint + test on every PR |

API ENDPOINTS TO BUILD

- > All Q1 endpoints verified: /health, /broker/*, /options/*, /autopilot/*, /auth/*

TESTS TO WRITE

- > test_full_pipeline.py: scan → score → risk check → dry-run order in < 120s
- > test_coverage.py: overall test coverage ≥ 60%
- > test_no_secrets.py: no .env values leak in test outputs
- > playwright/test_frontend_smoke.py: all 5 pages load without JS errors

ENV VARS & DEPENDENCIES

- | | |
|---------------------------|-------------------|
| -> CI=true | -> pytest-cov |
| -> TEST_ENV=true | -> pytest-asyncio |
| -> AUTOPILOT_MODE=dry_run | -> playwright |
| | -> httpx |

DELIVERABLES

- > Kill switch activates in <1 second when called via API

- > WebSocket broadcasts 'kill_switch_activated' event to all dashboard clients
- > Incident tracker captures every exception with stack trace
- > Email alert sent on FATAL incident (test with real SMTP)
- > Auto-recovery correctly escalates through 3 levels
- > Runbook complete and accurate

AUTOPILOT LOGIC

- > Now the system can be trusted to run overnight — it fails safely and loudly
- > Kill switch check is the FIRST thing every cycle does — even before data fetching
- > 'Incident dashboard' view in frontend: list of all incidents, severity badges, resolved/open status
- > Weekly incident summary emailed every Sunday: incident count, MTTR, resolution rate

PITFALLS & HOW TO AVOID

- > CI must use MockBroker — never run live or paper API calls in CI
- > Fix failing tests before marking Q1 complete — no deferred debt
- > Document every gap discovered in Q1_RETROSPECTIVE.md for Q2 planning

SUCCESS METRICS

- > Test coverage $\geq 60\%$ on services/
- > All integration tests pass in $< 5\text{min}$
- > CI pipeline green on main branch
- > Full dry-run cycle completes in $< 2\text{min}$ with realistic data

RISK

- > Risk: Kill switch bypass due to async race condition → use a DB flag, not just in-memory
- > Risk: Alert spam during prolonged outage → implement alert deduplication (same type: max 1/hour)
- > Risk: Email credentials in env → always use env vars; never hardcode; test with non-production SMTP

DAY-BY-DAY BREAKDOWN

- > Mon: Harden kill switch with DB persistence; broadcast WS event on activation
- > Tue: Build incident tracker; integrate into all exception handlers
- > Wed: Build alert service (SMTP + Slack webhook)
- > Thu: Implement 3-tier auto-recovery; test all escalation paths
- > Fri: Write runbook; run quarterly simulation; tag Q1 COMPLETE — celebrate!

Week 14: Walk-Forward Backtesting Engine

Build a rigorous walk-forward backtesting infrastructure to validate every strategy before live deployment.

Methodology Walk-Forward	OOS Ratio >70%	Scenarios 3 regimes	Speed <5 min full run
-----------------------------	-------------------	------------------------	--------------------------

GOALS

- > Implement WalkForwardEngine: sliding 2-year train / 1-year test windows across 2018-2024
- > Simulate trade execution with realistic costs: \$0.65/contract + 0.5% slippage
- > Compute OOS performance metrics: win rate, Sharpe, Sortino, max drawdown per window
- > Aggregate results across all windows to get TRUE expected performance
- > Build strategy comparison: run multiple strategy configs and rank by OOS Sharpe
- > Store backtest results in DB; expose via /api/v1/backtest/results endpoint

TASKS

- > Create services/backtest/engine.py: WalkForwardEngine class
- > Implement fetch_historical_options(): load from DB or CSV fixture for testing
- > Simulate entry: record credit received, max_profit, max_loss, dte at entry
- > Simulate exit: iterate daily; fire exit triggers in same priority order as live engine
- > Compute per-window stats: ProfitFactor, Sharpe, MaxDD using quantstats or manual
- > Build BacktestResult dataclass: windows[], aggregate_sharpe, aggregate_win_rate, oos_ratio
- > POST /api/v1/backtest/run → async task that runs backtest and stores result
- > GET /api/v1/backtest/results → list results with filter by strategy type

COMMANDS & FILES

- | | |
|---|--|
| -> python3 scripts/run_walk_forward.py --windows 12 | -> services/backtest/walk_forward.py |
| --window-size 90d | |
| -> python3 scripts/plot_wf_results.py --run-id latest | -> services/backtest/window_splitter.py |
| -> python3 -m pytest tests/test_walk_forward.py -v | -> services/backtest/overfitting_detector.py |
| | -> scripts/run_walk_forward.py |

API ENDPOINTS TO BUILD

- > POST /api/v1/backtest/walk-forward — run WFO with N windows
- > GET /api/v1/backtest/walk-forward/{id}/windows — per-window metrics

TESTS TO WRITE

- > test_wf_no_lookahead.py: in-sample never overlaps out-of-sample
- > test_wf_consistency.py: OOS Sharpe within 20% of IS Sharpe (no overfit)

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|------------------------------------|
| -> BACKTEST_WF_WINDOWS=12 | -> pandas |
| -> BACKTEST_WF_WINDOW_DAYS=90 | -> numpy |
| | -> optuna (parameter optimization) |

DELIVERABLES

- > WalkForwardEngine produces valid OOS results for PutCreditSpread strategy
- > Results stored in DB and retrievable via API
- > Sharpe and max drawdown computed correctly (verified against known dataset)
- > Backtest completes in <5 minutes for 3-year history

AUTOPILOT LOGIC

- > Autopilot MUST pass backtest gates before activating on any new strategy: Sharpe>1.0, MaxDD<20%
- > BacktestView in frontend: interactive chart of equity curve + per-window metrics table
- > Nightly backtest job (Week 94 concept, previewed here): run backtest on last 30 days live trades

PITFALLS & HOW TO AVOID

- > Walk-forward windows must never share data — strict date boundaries
- > Report both IS and OOS metrics — reporting only IS is misleading

SUCCESS METRICS

- > WFO runs 12 windows in < 5min
- > OOS Sharpe > 0.6 across all windows
- > Overfitting score (IS/OOS ratio) < 1.5

RISK

- > Risk: Look-ahead bias — use strict date boundary; NEVER use data from test window in training
- > Risk: Survivorship bias — include delisted symbols from historical data fixture
- > Risk: Overfitting from too many parameter combos → test max 3 parameters at a time

DAY-BY-DAY BREAKDOWN

- > Mon: Design walk-forward methodology; create historical data fixtures
- > Tue: Build WalkForwardEngine core; implement simulation loop
- > Wed: Implement all performance metrics computation
- > Thu: Build API endpoints; test with real PutCreditSpread strategy
- > Fri: Wire into frontend BacktestView; document methodology

Week 15: Monte Carlo Simulation & Risk-of-Ruin Analysis

Run 1M+ outcome simulations to stress-test the strategy and quantify tail risk before risking real capital.

Simulations 1,000,000	Scenarios 7 stress	RoR Target <1%	Speed <60s
--------------------------	-----------------------	-------------------	---------------

GOALS

- > Implement MonteCarloSimulator with 1M trade-sequence shuffles
- > Compute: expected return, 5th/95th percentile returns, max drawdown at 95th pct, risk-of-ruin
- > Build 7 stress scenarios from MASTER_PLAN.md: Flash Crash, COVID 2020, Black Monday, etc.
- > Stress test requires ALL scenarios to show drawdown < survival limit
- > Monte Carlo results shown in frontend with distribution histogram
- > Alert if risk-of-ruin > 1% — autopilot blocked until resolved

TASKS

- > pip install numpy scipy matplotlib
- > Create services/backtest/monte_carlo.py: MonteCarloSimulator
- > Implement trade_shuffling(): shuffle historical trades, simulate equity curve 1M times
- > Use numpy vectorization for speed: np.random.choice on returns array, cumsum for equity
- > Stress scenarios: modify return distribution to simulate extreme conditions
- > POST /api/v1/backtest/monte-carlo → runs simulation, returns distribution stats
- > Add MC results block to ReportsView: histogram + risk-of-ruin gauge

COMMANDS & FILES

- | | |
|--|------------------------------------|
| -> python3 scripts/monte_carlo.py --trades 500 --simulations 10000 | -> services/risk/monte_carlo.py |
| -> python3 scripts/plot_mc_histogram.py | -> services/risk/var_calculator.py |
| | -> scripts/monte_carlo.py |
| | -> scripts/plot_mc_histogram.py |

API ENDPOINTS TO BUILD

- > POST /api/v1/risk/monte-carlo — run MC simulation on portfolio
- > GET /api/v1/risk/var — portfolio VaR at 95% and 99% confidence

TESTS TO WRITE

- > test_monte_carlo.py: 10,000 sim results follow expected distribution
- > test_var_95.py: VaR(95) equals 5th percentile of MC loss distribution

ENV VARS & DEPENDENCIES

- | | |
|-------------------------|---------------|
| -> MC_SIMULATIONS=10000 | -> numpy |
| -> VAR_CONFIDENCE=0.95 | -> scipy |
| | -> matplotlib |

DELIVERABLES

- > 1M simulations complete in <60 seconds on modern CPU
- > Risk-of-ruin correctly computed as $P(\text{equity} < 0)$ across simulations
- > All 7 stress scenarios produce results within survival limits for base strategy
- > Frontend shows simulation results with histogram and key percentile markers

AUTOPILOT LOGIC

- > MC simulation runs monthly (first Sunday of month) automatically
- > If RoR > 1%: autopilot shows WARNING badge and requires human confirmation to continue
- > Stress scenario results stored and compared month-over-month to detect strategy degradation

PITFALLS & HOW TO AVOID

- > MC assumes IID returns — real trades are not IID; document this caveat
- > 10,000 sims can be slow — use numpy vectorization, not Python loops

SUCCESS METRICS

- > 10,000 MC sims complete in < 10s
- > VaR(95) reported for portfolio
- > Ruin probability < 5% at max drawdown scenario

RISK

- > Risk: 1M simulations too slow in Python → use numpy vectorized bootstrap instead of loop
- > Risk: MC results look good but use wrong distribution → verify returns are IID before bootstrapping

DAY-BY-DAY BREAKDOWN

- > Mon: Design MC methodology; build array-based bootstrap
- > Tue: Implement 7 stress scenarios with modified distributions
- > Wed: Build API endpoint; performance test for 1M speed
- > Thu: Build frontend histogram chart for MC results
- > Fri: Integration test; verify RoR blocking behavior

Week 16: Real-Time Greeks Dashboard & Portfolio Exposure

Build live portfolio Greeks monitoring with automatic exposure management and position balancing alerts.

Greeks 4 live	Update Rate 30s	Limits Auto-enforced	Capacity Display Yes
------------------	--------------------	-------------------------	-------------------------

GOALS

- > Build GreeksDashboard: real-time delta, theta, vega, gamma for entire portfolio
- > Implement portfolio limits enforcement: delta ± 0.20 , theta positive, vega $\pm \$500$, gamma $< \$100/1\%$
- > Show capacity indicator: how much more risk can be added within limits
- > Alert when any portfolio Greek approaches a limit ($>80\%$ of limit = WARN, $>100\%$ = BLOCK)
- > Build `check_portfolio_balance()` function called before every new trade validation
- > Greeks update every 30 seconds during market hours using latest option quotes

TASKS

- > Create `services/options/greeks_aggregator.py`: compute portfolio-level Greeks from positions
- > GET `/api/v1/portfolio/greeks` → returns {delta, theta, vega, gamma, limits, capacity, alerts}
- > Build `GreeksTile.tsx`: live table with per-position Greeks and portfolio totals
- > Implement `portfolio_delta_capacity()`: remaining delta budget after current positions
- > Add `check_portfolio_balance(candidate)` to validator: returns False if adding would breach limit
- > Wire 30-second Greeks refresh into frontend WebSocket or polling

COMMANDS & FILES

- | | |
|--|--|
| -> <code>python3 scripts/fetch_greeks_dashboard.py</code> | -> <code>frontend/src/pages/GreeksDashboard.tsx</code> |
| -> <code>npm run dev # verify GreeksDashboard.tsx renders</code> | -> <code>services/greeks/portfolio_greeks.py</code> |
| | -> <code>services/greeks/greek_aggregator.py</code> |
| | -> <code>scripts/fetch_greeks_dashboard.py</code> |

API ENDPOINTS TO BUILD

- > GET `/api/v1/greeks/portfolio` — aggregated portfolio greeks
- > GET `/api/v1/greeks/{position_id}` — per-position greek detail

TESTS TO WRITE

- > `test_greek_aggregator.py`: portfolio delta sum correct for 3 positions
- > `test_greeks_api.py`: endpoint returns delta, gamma, theta, vega, rho

ENV VARS & DEPENDENCIES

- | | |
|--|----------------------------------|
| -> <code>GREEKS_REFRESH_INTERVAL_SECONDS=60</code> | -> <code>numpy</code> |
| | -> <code>scipy</code> |
| | -> <code>react (frontend)</code> |

DELIVERABLES

- > Portfolio Greeks update every 30 seconds with real option quotes
- > Validator rejects new trades that would breach any portfolio Greek limit
- > Dashboard shows green/yellow/red indicators for each Greek near its limit
- > Capacity bar shows remaining delta/vega budget at a glance

AUTOPILOT LOGIC

- > Portfolio Greeks are a hard constraint — autopilot cannot breach them, period
- > Delta imbalance detection: if portfolio delta > 0.15 , autopilot prefers bearish strategies for balance
- > Theta decay is the engine's core profit mechanism — positive theta displayed prominently

PITFALLS & HOW TO AVOID

- > Aggregate theta is positive for credit positions — UI must show sign correctly

-> Vega units are per 1% IV change — document clearly in UI tooltip

SUCCESS METRICS

- > Portfolio greeks computed in < 500ms
- > Dashboard refreshes every 60s
- > Delta neutral flag triggers when $|\text{portfolio_delta}| < 0.10$

RISK

- > Risk: Stale quotes for Greeks → enforce quote age <120s before computing portfolio Greeks
- > Risk: Greeks computation slow for 10+ positions → cache individual Greeks; only recompute on quote update

DAY-BY-DAY BREAKDOWN

- > Mon: Build greeks_aggregator.py with position-weighted sums
- > Tue: API endpoint; limit enforcement in validator
- > Wed: GreeksTile.tsx with live update
- > Thu: Capacity indicators; warning thresholds
- > Fri: Integration test with 5 open positions

Week 17: Trade Execution Hardening & Order Management

Harden the execution layer: multi-leg order submission, fill confirmation, partial fills, and order state machine.

Order Types 3 multi-leg	Fill Confirm <5s	Partial Fills Handled	Audit Complete
----------------------------	---------------------	--------------------------	-------------------

GOALS

- > Submit multi-leg spread orders as single combo orders to Alpaca (reduces slippage vs legging)
- > Implement order state machine: PENDING → WORKING → FILLED / PARTIALLY_FILLED / CANCELLED
- > Handle partial fills: store partial position, monitor for completion, retry or cancel remainder
- > Confirm fills by polling order status with 5-second timeout; escalate to incident if no fill
- > Build OrderRecord with full execution audit: order_id, legs[], fill_price, slippage, broker_ref
- > Test execution with real paper trades on Alpaca; verify split fills handled correctly

TASKS

- > Refactor alpaca_broker.py: submit_spread_order(legs: list[Leg]) → handles combo vs sequential
- > Implement OrderStateMachine: transitions, timeouts, escalation to incident if stuck
- > Poll alpaca.get_order(order_id) every 2s until FILLED or timeout (30s)
- > Handle partial fills: update position with filled qty; schedule retry or cancel remainder
- > Create orders table columns: fill_price, slippage_pct, legs_json, partial_fill_qty
- > Write execution test suite: 10 paper orders including partial fill simulation

COMMANDS & FILES

- | | |
|---|---------------------------------------|
| -> uvicorn --workers 4 --loop uvloop | -> services/execution/order_router.py |
| -> python3 scripts/test_order_latency.py | -> services/execution/fill_tracker.py |
| -> websocat ws://localhost:8000/ws/orders | -> apps/api/websocket/order_stream.py |

API ENDPOINTS TO BUILD

- > POST /api/v1/orders — place with slippage tracking
- > WS /ws/orders — real-time order fill stream

TESTS TO WRITE

- > test_fill_latency.py: fill confirmation < 2s
- > test_partial_fill.py: partial fill handled

ENV VARS & DEPENDENCIES

- | | |
|-----------------------------------|-----------------|
| -> EXECUTION_MAX_SLIPPAGE_PCT=0.5 | -> uvloop>=0.19 |
| -> ORDER_TIMEOUT_SECONDS=30 | -> websockets |

DELIVERABLES

- > Multi-leg spread orders placed as single Alpaca combo order
- > Order state machine transitions correctly through all states
- > Partial fills handled: position updated with correct filled quantity
- > Execution audit trail complete in DB for every order attempt

AUTOPILOT LOGIC

- > Execution phase now has real production-grade error handling
- > Failed execution (no fill in 30s) → cancel order → log incident → try next candidate
- > Fill price vs expected credit tracked as slippage_pct → feed into future entry scoring

PITFALLS & HOW TO AVOID

- > Mid-price orders only for options — market orders too wide on spreads
- > Partial fills must be tracked — don't assume full fill immediately

SUCCESS METRICS

- > Order fill latency < 2s p95

-> Slippage < 0.5% of mid price

RISK

- > Risk: Combo order not supported by Alpaca paper → fallback to sequential leg execution with retry
- > Risk: Order stuck in WORKING state → hard timeout at 60s; cancel and escalate to incident

DAY-BY-DAY BREAKDOWN

- > Mon: Redesign execution layer for combo orders
- > Tue: Order state machine with all transitions
- > Wed: Partial fill handling; retry logic
- > Thu: Write 10-paper-order test suite
- > Fri: Slippage tracking; wire into entry scoring feedback loop

Week 18: Performance Attribution & Reporting Engine

Build automated daily/weekly/monthly P&L; reports with performance attribution by strategy, symbol, and regime.

Reports 3 frequencies	Attribution 3 dimensions	Delivery Email+Dashboard	Format PDF+JSON
--------------------------	-----------------------------	-----------------------------	--------------------

GOALS

- > Daily EOD report: total P&L; open positions summary, win rate, top winner/loser
- > Weekly report: 7-day P&L; chart, strategy breakdown, sector exposure, Greeks recap
- > Monthly report: full performance attribution, Monte Carlo update, strategy ranking, improvement recommendations
- > Attribution by: strategy_type, underlying symbol, IV regime, market regime
- > PDF report generation with reportlab; email delivery via SMTP
- > All reports stored in DB and accessible via /api/v1/reports endpoint

TASKS

- > pip install reportlab
- > Create services/autopilot/reporting.py: ReportEngine class
- > Implement generate_daily_report(): query DB for today's trades, compute stats, format PDF
- > Implement generate_weekly_report(): 7-day equity curve, per-strategy breakdown
- > Implement generate_monthly_report(): full analytics including Monte Carlo update
- > POST /api/v1/reports/generate?period=daily|weekly|monthly
- > GET /api/v1/reports?period=weekly&limit=10
- > Build ReportsView.tsx: list of reports with preview and download button

COMMANDS & FILES

- | | |
|---|---------------------------------------|
| -> python3 scripts/generate_daily_report.py --date today | -> services/reporting/daily_report.py |
| -> python3 scripts/email_report.py --recipient me@example.com | -> services/reporting/pdf_builder.py |
| | -> services/reporting/csv_exporter.py |
| | -> scripts/email_report.py |

API ENDPOINTS TO BUILD

- > GET /api/v1/reports/daily/{date} — daily P&L; PDF
- > GET /api/v1/reports/export/csv?start=&end;= — CSV export

TESTS TO WRITE

- > test_report_generation.py: PDF generates without error
- > test_csv_export.py: CSV has correct column headers and row count

ENV VARS & DEPENDENCIES

- | | |
|--|---------------------|
| -> SMTP_HOST=smtp.gmail.com | -> reportlab |
| -> SMTP_PORT=587 | -> jinja2 |
| -> REPORT_EMAIL_RECIPIENT=me@example.com | -> smtplib (stdlib) |

DELIVERABLES

- > Daily PDF report generated at 4:05 PM automatically
- > Weekly report emailed every Sunday morning
- > Attribution correctly identifies which strategies and symbols drove P&L;
- > ReportsView shows last 10 reports of each frequency

AUTOPILOT LOGIC

- > Reports are how the user KNOWS the autopilot is working — CRITICAL feature
- > Monthly report includes 'strategy health' section: is backtest edge degrading? Recommend changes
- > Report includes LLM agreement rate: what % of cycles had both LLMs agree

PITFALLS & HOW TO AVOID

- > PDF generation can OOM on large logs — paginate, don't load all in memory
- > SMTP credentials in env only — never hardcoded

SUCCESS METRICS

- > Daily report PDF generated by 4:30 PM ET
- > CSV export < 1s for 1 year of trades

RISK

- > Risk: ReportEngine crashes if no trades that day → handle empty periods gracefully (0-trade report)
- > Risk: PDF too large for email → compress with zlib; limit to last 90 days in monthly

DAY-BY-DAY BREAKDOWN

- > Mon: Design report structure; implement daily report
- > Tue: Weekly report with equity curve chart
- > Wed: Monthly report with Monte Carlo update
- > Thu: Email delivery; PDF formatting polish
- > Fri: ReportsView in frontend; schedule daily/weekly jobs

Week 19: Symbol Universe Management & Market Regime Detection

Build intelligent universe selection and market regime classifier that adapts strategy mix to conditions.

Universe 500 symbols	Filter Speed <2s	Regimes 4 types	Adaptation Automatic
-------------------------	---------------------	--------------------	-------------------------

GOALS

- > Build symbol universe: 500 large-cap equities + top-20 ETFs, liquid options, filtered for quality
- > Universe filter criteria: avg_volume>1M, market_cap>\$1B, options_volume>500/day, iv_liquidity>0.3
- > Implement MarketRegimeDetector: classify current regime as Bull/Bear/Neutral/HighVol
- > Map regime → strategy allocation: Bull→more calls, Bear→more puts, HighVol→more iron condors
- > Pre-market universe scan: filter 500 → ~150 qualified symbols for the day
- > Weekly universe refresh: add newly qualified symbols, remove those that no longer qualify

TASKS

- > Create services/autopilot/universe.py: UniverseManager
- > Load initial 500 symbols from S&P; 500 + Russell 1000 list (CSV in data/)
- > Filter universe daily: volume, market cap, options liquidity checks
- > MarketRegimeDetector: use VIX level + SPY 50dma position + VIX 30d trend
- > Regime → strategy_allocation dict: override default allocation weights per regime
- > GET /api/v1/universe → qualified symbols for today with filter stats
- > POST /api/v1/universe/refresh → trigger manual universe refresh

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week19_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week19*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week19_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week19_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 500-symbol universe filtered down to qualified set in <2 seconds

- > Regime correctly classified as Bull/Bear/Neutral/HighVol for test dates
- > Strategy allocation automatically shifts based on regime
- > Universe refresh runs every Monday pre-market

AUTOPILOT LOGIC

- > Autopilot skips symbols that don't pass universe filter — no exceptions
- > Regime displayed prominently in AIPanel: '■ Regime: BULL · Strategy Mix: 60% Put Credit'
- > Universe filter results logged to DB: how many symbols qualify each day

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: All symbols filtered out on low-liquidity day → minimum universe floor of 20 symbols
- > Risk: Regime detection wrong during rapid transitions → add 3-day confirmation window before switching

DAY-BY-DAY BREAKDOWN

- > Mon: Load 500-symbol universe; implement volume/liquidity filters
- > Tue: MarketRegimeDetector with VIX + SPY signals
- > Wed: Regime → strategy allocation mapping
- > Thu: Pre-market universe scan job; API endpoints
- > Fri: Test regime detection on historical dates; document

Week 20: Comprehensive Test Suite — 1,000+ Tests

Reach full test coverage: 1,033+ backend tests across unit, integration, and E2E levels.

Total Tests 1,033+	Coverage >85%	Runtime <5 min	Flakiness Zero
-----------------------	------------------	-------------------	-------------------

GOALS

- > Reach 1,033+ passing pytest tests across all backend services
- > Achieve >85% code coverage on all service modules
- > Zero flaky tests: any test that fails randomly is fixed or removed
- > Playwright E2E tests for critical user journeys: run autopilot cycle, view positions, kill switch
- > 22 Vitest unit tests all passing with <5s runtime
- > CI pipeline runs full test suite on every PR in <10 minutes

TASKS

- > Audit existing tests; identify coverage gaps using pytest-cov
- > Write missing unit tests for: scoring, validator, monitoring, universe, regime
- > Write integration tests for: full cycle with real DB, broker mocks, LLM mocks
- > Write Playwright E2E: login→dashboard→run cycle→view run artifact→kill switch
- > Fix all flaky tests: identify timing dependencies; use deterministic mocks
- > Optimize CI: parallelize test suites; cache pip and npm dependencies
- > Add test coverage badge to README

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week20_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week20*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week20_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week20_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 1,033+ tests all passing in CI
- > >85% coverage report generated by pytest-cov
- > Playwright E2E passes headlessly in CI

- > CI pipeline completes in <10 minutes
- > Zero flaky test failures in last 10 CI runs

AUTOPILOT LOGIC

- > All autopilot cycle phases individually tested with comprehensive mocks
- > Regression test: run full dry_run cycle → assert RunArtifact structure matches schema exactly
- > Property-based testing (hypothesis): entry scorer always returns 0-100 for any valid input

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration tests require real API keys → use request recording (VCR) for CI
- > Risk: Test DB state bleeding between tests → use transaction rollback fixture per test

DAY-BY-DAY BREAKDOWN

- > Mon: pytest-cov audit; identify top 20 coverage gaps
- > Tue: Write 200+ missing unit tests
- > Wed: Write integration tests with VCR cassettes
- > Thu: Playwright E2E tests; CI parallelization
- > Fri: Zero flaky tests verification; update coverage badge

Week 21: Options Volatility Surface & IV Analytics

Build 3D implied volatility surface visualization and IV analytics for smarter trade selection.

Surface 3D vol surface	IV Rank 52-week	IV Percentile vs 1yr	Update 15 min
---------------------------	--------------------	-------------------------	------------------

GOALS

- > Implement implied volatility surface: IV vs strike × expiration as a 3D matrix
- > Visualize vol surface as a 3D chart in VolSurfaceTile.tsx (using Three.js or Plotly)
- > Implement IV Percentile (IVP): current IV vs 1-year historical distribution (better than IV Rank)
- > Use IV surface shape to detect vol skew opportunities: steep put skew → prefer put spreads
- > Alert when vol surface inverts (front month IV > back month) — unusual signal
- > Store IV surface snapshots daily for trend analysis

TASKS

- > Extend TradierProvider: build IV matrix from full chain across expirations
- > Create services/options/vol_surface.py: build_surface(symbol) → 2D numpy array
- > GET /api/v1/options/vol-surface?symbol=AAPL → JSON matrix for charting
- > Build VolSurfaceTile.tsx with Plotly.js surface chart
- > Implement compute_iv_percentile(symbol, current_iv, lookback_days=252)
- > Store daily IV snapshots in iv_history table: symbol, date, atm_iv, skew_ratio
- > Wire IVP into entry scorer: use IVP instead of IV Rank (90-day lookback preferred)

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week21_feature.py --dry-run-> python3 -m pytest tests/test_week21*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week21_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week21_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > 3D vol surface renders correctly for AAPL and SPY
- > IVP computed for universe symbols using 1-year history
- > Vol surface inversion alert fires correctly for test case

-> IV history table populated daily via scheduler

AUTOPILOT LOGIC

- > Entry scorer now uses IVP (superior to IV Rank for regime-awareness)
- > Vol skew analysis: steep put skew adds 3 bonus points to put credit spread score
- > Dashboard shows 'vol rank' badge on each position showing IVP at entry time

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Vol surface sparse for less-liquid symbols → handle missing strikes with linear interpolation
- > Risk: IVP needs 252 days of history → cold-start period uses IV Rank as fallback

DAY-BY-DAY BREAKDOWN

- > Mon: Build IV matrix from option chain; compute surface
- > Tue: Vol surface API endpoint
- > Wed: VolSurfaceTile.tsx with Plotly 3D
- > Thu: IVP calculation with 252-day history
- > Fri: Daily snapshot scheduler; skew detector

Week 22: Earnings Calendar & Event-Driven Risk Gateway

Integrate earnings calendar with automated blackout zones and event-driven risk adjustments.

Sources 2 calendars	Blackout 2d pre-earnings	Coverage Full S&P500;	Accuracy >99%
------------------------	-----------------------------	--------------------------	------------------

GOALS

- > Integrate Tradier earnings calendar + Nasdaq earnings API as dual sources
- > Implement earnings_blackout_gate: reject any trade if earnings within 2 trading days
- > Build event timeline: show upcoming earnings for all positions on dashboard
- > Send pre-market alert listing today's and tomorrow's earnings for held symbols
- > Implement post-earnings analysis: capture IV crush, price move, P&L; impact

TASKS

- > Create services/data/earnings_calendar.py: EarningsCalendar with dual-source validation
- > GET /api/v1/calendar/earnings?symbols=AAPL,MSFT → earnings dates
- > Add earnings_blackout_gate to ValidationGate enum; implement in validator
- > CalendarTile.tsx: heatmap of upcoming earnings for watchlist
- > Alserter: pre-market check — if any position has earnings today: send URGENT alert
- > Store IV before and after earnings in iv_events table for post-analysis

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week22_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week22*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week22_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week22_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Earnings dates returned for all 500 universe symbols
- > Blackout gate blocks trades within 2 days of earnings (tested)
- > Pre-market alert sends list of at-risk positions
- > Post-earnings IV crush captured in DB

AUTOPILOT LOGIC

- > Earnings blackout is a HARD STOP — cannot be disabled via config
- > Pre-earnings position review: autopilot evaluates whether to close 3 days before, not just 2
- > earnings_risk_score added to position record: countdown days × IV exposure

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Earnings date wrong (company reschedules) → dual-source validation + manual override API
- > Risk: After-hours earnings miss a day boundary → use 'confirmed_date' field only; ignore unconfirmed

DAY-BY-DAY BREAKDOWN

- > Mon: Earnings calendar integration (dual source)
- > Tue: Blackout gate in validator
- > Wed: CalendarTile.tsx
- > Thu: Pre-market earnings alert job
- > Fri: Post-earnings IV analysis; event DB table

Week 23: Alerting System & Notification Hub

Build a comprehensive multi-channel alerting system for price alerts, risk warnings, and system events.

Channels	Alert Types	Delivery	Dedup
Email+Slack+WS	10+	<30s	1hr window

GOALS

- > Build AlertsEngine: 10+ alert types managed centrally
- > Price alerts: user-set price targets → notify when triggered
- > Risk alerts: Greeks near limit, drawdown approaching max, consecutive losses
- > System alerts: cycle failures, broker disconnected, budget exceeded
- > Implement deduplication: same alert type per symbol → max 1 per hour
- > AlertsView.tsx: manage active alerts, history, acknowledgement

TASKS

- > Create services/alerts/alerts_engine.py: AlertsEngine with alert_type router
- > Define AlertType enum: PRICE_TARGET, GREEK_LIMIT, DRAWDOWN_WARNING, EARNINGS_RISK, CYCLE_FAILURE, KILL_SWITCH
- > Implement check_price_alerts() in intraday scan loop
- > Implement check_risk_alerts() in Greeks monitoring
- > POST /api/v1/alerts → create alert | GET /api/v1/alerts → list | DELETE /api/v1/alerts/{id}
- > AlertsView.tsx with create-alert form, active-alerts table, alert history

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week23_feature.py --dry-run-> python3 -m pytest tests/test_week23*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week23_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week23_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > 10+ alert types configured and firing correctly in tests
- > Price alert fires when AAPL crosses target price (paper test)
- > Deduplication prevents alert spam

-> AlertsView shows all active and historical alerts

AUTOPILOT LOGIC

- > Autopilot events → alert pipeline: every significant engine event creates an alert
- > Alert confidence: 'high-confidence' alerts (both LLMs + high score) get ✓ badge
- > Alert history shown in RunsAuditView: what alerts fired during each cycle

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Alert channel down (Slack webhook expired) → queue alerts in DB; retry with backoff
- > Risk: Alert spam on volatile day → exponential backoff + dedup window

DAY-BY-DAY BREAKDOWN

- > Mon: AlertsEngine core; 10 alert types
- > Tue: Price alert checker in scan loop
- > Wed: Risk alerts (Greeks, drawdown)
- > Thu: AlertsView.tsx
- > Fri: Deduplication; E2E test all channels

Week 24: Data Integrity Pipeline & Quality Gates

Add multi-source data verification, staleness detection, and automated self-healing for corrupt data.

Sources 3-way verify	Staleness <5s threshold	Self-Heal Auto-retry	Audit Data lineage
-------------------------	----------------------------	-------------------------	-----------------------

GOALS

- > Implement multi-source price verification: Tradier + Alpaca + yfinance must agree within 1%
- > Staleness detector: data older than 5 seconds = stale; cycle aborts and retries
- > Build DataAuditLog: every piece of data used in a decision logged with source, timestamp, value
- > Implement self-healing: on stale data, rotate to backup source automatically
- > Sanity bounds validation: stock price 0-100000, IV 0-500%, delta -1 to +1
- > Add data quality score to RunArtifact: % of data points that needed fallback or rejection

TASKS

- > Create services/data/data_guardian.py: DataGuardian.validate(symbol, data_type, value)
- > Implement price cross-check: fetch Tradier + Alpaca quotes, compare within tolerance
- > Implement freshness_check(data): raise StaleDataError if age > threshold
- > Add backup_source rotation: primary → secondary → tertiary → abort
- > DataAuditLog table: run_id, symbol, data_type, source, value, age_ms, validated_at
- > Add data_quality_score to RunArtifact: 1.0 = all data pristine, 0.8 = 20% from backup

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week24_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week24*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week24_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week24_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Price cross-check catches spoofed or erroneous quote (test case)
- > StaleDataError correctly aborts cycle and retries from backup source
- > DataAuditLog populated for every run
- > Data quality score displayed in run history view

AUTOPILOT LOGIC

- > Data guardian runs as a middleware layer — ALL data passes through it before reaching scoring
- > If data quality score < 0.7 → cycle skips execution entirely; logs WARNING
- > Data quality trends tracked weekly; degradation triggers alert

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: All 3 sources return bad data simultaneously → hard abort cycle; manual review
- > Risk: DataAuditLog grows too large → partition by month; archive after 90 days

DAY-BY-DAY BREAKDOWN

- > Mon: DataGuardian core; price cross-check
- > Tue: Freshness check; backup source rotation
- > Wed: DataAuditLog table and population
- > Thu: Data quality score in RunArtifact
- > Fri: Integration test all data guard scenarios

Week 25: Frontend Polish: Dark Theme, Animations & UX

Transform the dashboard from functional to premium: dark terminal aesthetics, micro-animations, and pro UX.

Theme Dark terminal	Animations 12 micro	Accessibility WCAG AA	Performance <100ms FCP
------------------------	------------------------	--------------------------	---------------------------

GOALS

- > Implement cohesive dark terminal theme: #0D1117 background, blue accent #2196F3, Fira Code monospace
- > Add 12 micro-animations: panel transitions, P&L; counters, status pulsing, chart fade-in
- > Performance: First Contentful Paint <100ms; no layout shifts; lazy-load heavy panels
- > Accessibility: keyboard navigation for all interactive elements; screen reader labels; focus rings
- > Mobile-responsive layout: dashboard works on tablet (1024px) without horizontal scroll
- > Add premium touches: animated gradient header, glassmorphism panels, smooth number transitions

TASKS

- > Create frontend/src/styles/theme.ts with all design tokens
- > Implement TailwindCSS dark mode; configure custom colors in tailwind.config.ts
- > Add Framer Motion animations to: panel open/close, P&L; increment/decrement, status change
- > Build animated status pulsing component for 'Autopilot Running' indicator
- > Add ARIA labels to all dashboard interactive elements
- > Performance: lazy-load VolSurfaceTile, Monte Carlo chart (heavy components)
- > Lighthouse audit: score >90 performance, >90 accessibility

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week25_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week25*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week25_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week25_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Dashboard looks like a premium Bloomberg-style terminal
- > All 12 micro-animations smooth at 60fps
- > Lighthouse scores: Performance 90+, Accessibility 90+

- > Mobile 1024px layout functional
- > No WCAG contrast failures

AUTOPILOT LOGIC

- > AIPanel redesign: prominent START/STOP button, animated status ring, live metrics
- > Status ring: pulsing green when running, static grey when idle, blinking red when killed
- > RunArtifact card shows animated number increment for candidates_considered and orders_placed

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Framer Motion increases bundle size → lazy-import animations; target <500KB gzipped
- > Risk: Animations cause jank on lower-end hardware → use CSS transforms only; avoid layout animations

DAY-BY-DAY BREAKDOWN

- > Mon: Design token system; implement dark theme throughout
- > Tue: Framer Motion animations for panels and status
- > Wed: Animated P&L; counter; chart fade-in
- > Thu: ARIA labels; keyboard navigation; focus management
- > Fri: Lighthouse audit; performance fixes; mobile layout

Week 26: Q2 Wrap-Up: Integration Testing, Hardening & v1.0 Tag

Full-stack integration test, production hardening, security audit, and tagging the v1.0 milestone release.

Release v1.0.0	E2E Tests 10 journeys	Security OWASP Top10	Uptime 99.9% target
-------------------	--------------------------	-------------------------	------------------------

GOALS

- > Run 10 complete E2E user journeys end-to-end with Playwright
- > Security audit: OWASP Top 10 review; fix any HIGH/CRITICAL findings
- > Performance load test: API handles 100 concurrent users with P99 < 200ms
- > Tag v1.0.0; create GitHub release with changelog, architecture diagram, setup guide
- > Deploy to production (or staging) environment; verify all services start cleanly
- > Write post-Q2 retrospective: what worked, what to improve in Q3

TASKS

- > Playwright E2E: run autopilot cycle, view run artifact, create price alert, kill switch, view reports
- > Run OWASP ZAP scan against API; fix any HIGH findings (injection, CSRF, auth bypass)
- > Locust load test: 100 VU hitting /cycle and /positions for 5 minutes
- > Fix all issues found by E2E, security, and load tests
- > git tag v1.0.0 and push with annotated tag message
- > Write GitHub release notes: features, known limitations, upgrade path

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week26_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week26*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week26_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week26_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > All 10 E2E journeys pass headlessly in CI
- > Zero HIGH/CRITICAL security findings

- > API P99 < 200ms under 100 concurrent users
- > v1.0.0 tagged and released on GitHub
- > Retrospective document in docs/retrospectives/q2.md

AUTOPILOT LOGIC

- > v1.0 marks: scheduler running, dual-LLM pipeline, backtesting, sentient monitoring — COMPLETE
- > First live paper trading week: track every cycle, every decision, every trade for 5 trading days
- > Document: did the autopilot actually work? What did it trade? What was the paper P&L;?

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Security scan finds credential leakage → audit all log outputs; mask API keys in logs
- > Risk: Load test reveals DB bottleneck → add SQLite WAL + connection pool; scale reads

DAY-BY-DAY BREAKDOWN

- > Mon: Write all 10 Playwright E2E scenarios
- > Tue: OWASP ZAP security scan; fix findings
- > Wed: Locust load test; performance fixes
- > Thu: Write GitHub release notes; tag v1.0.0
- > Fri: Deploy to staging; verify all services; Q2 retrospective

Week 27: Advanced LLM Prompt Engineering & Chain-of-Thought

Upgrade dual-LLM pipeline with CoT reasoning, structured outputs, and explainability traces.

CoT Steps 5-step	JSON Schema Enforced	Explainability Per-trade	Agreement Logged
---------------------	-------------------------	-----------------------------	---------------------

GOALS

- > Implement 5-step chain-of-thought prompts for Groq and Gemini
- > Enforce JSON schema output validation for every LLM response
- > Build explainability trace: why each candidate was selected/rejected with LLM reasoning
- > Log LLM agreement/disagreement patterns over time for meta-learning
- > Implement few-shot examples in prompts from best historical trades
- > Reduce LLM latency by caching prompt templates and parallel calls

TASKS

- > Upgrade groq_ranker.py: 5-step CoT prompt template
- > json_schema_validator: validate LLM output matches TradeDecision schema
- > Store llm_reasoning text per trade decision in DB
- > Parallel Groq+Gemini calls with asyncio.gather for speed
- > Few-shot prompt builder: inject top-3 historical trades as examples
- > GET /api/v1/runs/{id}/llm-trace → full reasoning for each decision

COMMANDS & FILES

- > python3 scripts/run_week27_feature.py --dry-run
 - > python3 -m pytest tests/test_week27*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week27_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week27_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > LLM responses always valid JSON matching schema
- > CoT reasoning visible in run artifact trace

- > Parallel LLM calls reduce pipeline latency by ~40%
- > LLM agreement rate tracked weekly

AUTOPILOT LOGIC

- > CoT forces LLM to reason about Greeks, IV, days-to-expiry explicitly before deciding
- > Disagreement logger: when Groq says YES but Gemini says NO, log the delta for review
- > Few-shot examples updated monthly from top 10 highest-scoring closed trades

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: CoT prompt too long → exceeds token limit; use compressed few-shot (2 examples max)
- > Risk: JSON parse failure on malformed response → retry once; then use deterministic fallback

DAY-BY-DAY BREAKDOWN

- > Mon: CoT prompt templates for Groq+Gemini
- > Tue: JSON schema validator with retry
- > Wed: Parallel LLM calls with `asyncio.gather`
- > Thu: Few-shot example builder from historical trades
- > Fri: LLM trace API; test agreement rate logging

Week 28: Portfolio Correlation & Sector Exposure Management

Prevent portfolio concentration risk with correlation filtering and sector exposure caps.

Correlation Max 0.7	Sectors Max 30%	Matrix Real-time	Blocked Auto
------------------------	--------------------	---------------------	-----------------

GOALS

- > Compute pairwise correlation matrix for all open positions
- > Block new trades that would create >0.7 correlation with existing position
- > Implement sector exposure cap: max 30% of portfolio in any single GICS sector
- > Show correlation heatmap in PortfolioView
- > Track correlation drift: alert when correlation rises post-entry (merger, index rebalancing)
- > Sector lookup table: map symbol → GICS sector for all 500 universe symbols

TASKS

- > Create services/portfolio/correlation.py: compute_correlation_matrix()
- > Add correlation_gate to ValidationGates: reject if pairwise_corr > 0.7
- > Build sector_exposure() from DB positions + sector_map CSV
- > Add sector_cap_gate: reject if adding symbol would put sector > 30%
- > CorrelationTile.tsx: heatmap using Chart.js matrix plugin
- > Weekly correlation drift alert job

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week28_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week28*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week28_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week28_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Correlation matrix computed from 30-day price returns for all positions
- > Correlation gate blocks correlated trade (test case passes)

- > Sector exposure bar chart in PortfolioView
- > Sector cap gate blocks concentrated sector trade

AUTOPILOT LOGIC

- > Correlation gate makes autopilot naturally diversified — no manual balancing needed
- > Sector exposure logged with every RunArtifact for trend analysis
- > Correlation drift alert: if two positions become >0.85 correlated post-entry → evaluate closing one

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in $< 500\text{ms}$ p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Correlation computation needs 30-day history → skip for new listings (<30 days old)
- > Risk: Correlation matrix too slow for 20+ positions → cache with 1-hour refresh

DAY-BY-DAY BREAKDOWN

- > Mon: Correlation matrix from returns DB
- > Tue: Correlation gate in validator
- > Wed: Sector map CSV + sector exposure cap
- > Thu: CorrelationTile.tsx heatmap
- > Fri: Correlation drift alert; E2E test

Week 29: Advanced Exit Logic: Trailing Stops & Gamma Scalping

Enhance exit logic with dynamic trailing stops, gamma scalping, and volatility-adjusted exit thresholds.

Trailing Stop Dynamic	Gamma Scalp Optional	Exit Score 5 factors	Override Manual OK
--------------------------	-------------------------	-------------------------	-----------------------

GOALS

- > Implement dynamic trailing stop: lock in 50% of max profit when position reaches 60% profit
- > Gamma scalping option: for delta-heavy positions, scalp deltas to reduce cost basis
- > Volatility-adjusted exits: tighten stop loss when VIX spikes (regime-aware exits)
- > Build exit score: 5-factor composite (DTE, P&L,%, IV change, delta drift, news score)
- > Manual exit override: UI button to immediately close position at market
- > Exit replay: show what exit trigger fired and at what exact time in run history

TASKS

- > Upgrade PositionMonitor: add trailing_stop tracking to each position
- > trailing_stop_update(): when P&L,% > 60%, set trailing = current - 50%*(max_profit - current)
- > VIX-adjusted stop: if VIX > 25, tighten stop to 30% loss (vs 50% default)
- > Gamma_scalp_hedge(): optional hedge for positions with |delta| > 0.3
- > ExitScore dataclass: compute from 5 factors; persist per monitoring event
- > Add 'Close Now' button to PositionTile with confirmation modal

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week29_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week29*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week29_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week29_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Trailing stop correctly locks in 50% profit after 60% threshold hit (tested on paper)
- > VIX spike triggers tighter stop loss (test with VIX=28)

- > Exit score computed every 5 minutes per position
- > Manual close executes immediately and logs exit_trigger=MANUAL

AUTOPILOT LOGIC

- > Trailing stop is the autopilot's most profitable exit enhancement — captures winners
- > Gamma scalping is optional (config.gamma_scalp_enabled) — disabled by default for simplicity
- > Exit replay makes every close transparent: user can see exactly WHY position was closed

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Trailing stop too aggressive → whipsaws on volatile days; add 15-min lockout after trigger
- > Risk: Manual close during fast market falls → confirm fill before updating position to CLOSED

DAY-BY-DAY BREAKDOWN

- > Mon: Trailing stop tracking per position
- > Tue: VIX-adjusted exit thresholds
- > Wed: Exit score 5-factor computation
- > Thu: Manual close UI button + confirmation
- > Fri: Exit replay in run history; test all scenarios

Week 30: n8n Workflow Orchestration (Docker Setup)

Deploy n8n in Docker for enterprise-grade workflow automation, replacing APScheduler for critical paths.

Workflows 8 active	Reliability Retry+alert	UI n8n visual	Trigger Webhook+cron
-----------------------	----------------------------	------------------	-------------------------

GOALS

- > Deploy n8n via Docker Compose alongside FastAPI and SQLite
- > Migrate 8 critical workflows from APScheduler to n8n: pre-market, scan, monitor, EOD, reports, backup, alert, recovery
- > Each workflow: retry on failure, send Slack alert on repeated failure, log execution in n8n history
- > Expose /api/v1/workflows/trigger endpoint for manual workflow execution from dashboard
- > n8n webhooks call FastAPI endpoints; FastAPI calls back with results via HTTP
- > WorkflowsView.tsx: list active workflows, last run status, trigger manually

TASKS

- > docker-compose.yml: add n8n service on port 5678 with volume mount
- > Configure n8n credentials: Alpaca, Tradier, Groq, SMTP
- > Create n8n workflows: JSON export for each of 8 workflows
- > FastAPI: POST /api/v1/workflows/trigger?name=pre-market → triggers n8n webhook
- > GET /api/v1/workflows → list workflows and last execution status from n8n API
- > WorkflowsView.tsx: workflow cards with status LEDs and trigger buttons

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week30_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week30*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week30_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week30_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > n8n running in Docker with all 8 workflows active
- > Pre-market workflow executes at 8:45 AM via n8n cron
- > Manual trigger from WorkflowsView calls n8n and returns execution ID
- > All workflows have retry logic and failure alerting

AUTOPILOT LOGIC

- > n8n replaces APScheduler for production reliability — n8n has built-in retry, history, visual debugging
- > n8n workflow execution IDs correlated with run_ids in FastAPI DB for full auditability
- > n8n admin dashboard (port 5678) accessible internally for debugging workflow issues

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Docker unavailable in WSL → fallback plan: keep APScheduler, n8n workflows as reference only
- > Risk: n8n credentials stored in n8n internal DB → ensure volume is outside container for persistence

DAY-BY-DAY BREAKDOWN

- > Mon: Docker Compose setup; n8n running locally
- > Tue: Configure credentials; first 3 workflows
- > Wed: Remaining 5 workflows; retry logic
- > Thu: FastAPI webhook endpoints; WorkflowsView
- > Fri: E2E test: n8n triggers pre-market → FastAPI executes → logs result

Week 31: Advanced Market Data: Tick Data & Level 2 Quotes

Integrate tick-level data and Level 2 quotes for better execution timing and microstructure analysis.

Tick Rate <100ms	L2 Depth 5 levels	Storage Compressed	Use Exec timing
---------------------	----------------------	-----------------------	--------------------

GOALS

- > Integrate real-time tick data via Alpaca WebSocket stream
- > Store compressed tick data for analysis (30-day rolling window, then archive)
- > Build Level 2 order book display for any symbol in the chart panel
- > Use bid/ask spread analysis to time option spread execution (execute when spread is tightest)
- > Build TickAnalyzer: detect unusual volume spikes before news (potential front-running signal)
- > Spread timing advisor: show optimal execution windows based on historical bid/ask patterns

TASKS

- > alpaca_ws.py: subscribe to trades and quotes feeds for universe symbols
- > tick_store.py: compressed HDF5 or parquet tick storage with 30-day TTL
- > GET /api/v1/market/ticks?symbol=AAPL&lookback;=60s
- > OrderBookTile.tsx: real-time 5-level L2 depth display
- > TickAnalyzer: volume_spike_detector() — flag when tick volume > 3× 5min MA
- > Spread timing: identify minutes of day when bid/ask spread is historically narrowest

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week31_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week31*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week31_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week31_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Alpaca tick stream connected and storing ticks
- > OrderBookTile renders live L2 depth

- > Volume spike alerts generated for test event
- > Spread timing advisor shows best execution windows

AUTOPILOT LOGIC

- > Execution engine waits for tight spread window before executing limit orders
- > Volume spike = potential news alert → autopilot flag for manual review before executing
- > Tick data used for next week's ML signal generation (foundation laid here)

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Tick data volume is massive → strict 30-day TTL; use parquet compression (10× compression)
- > Risk: WebSocket drops → reconnect with exponential backoff; buffer missed ticks with REST catch-up

DAY-BY-DAY BREAKDOWN

- > Mon: Alpaca WebSocket tick stream
- > Tue: Parquet tick storage with TTL
- > Wed: OrderBookTile with L2 depth
- > Thu: TickAnalyzer volume spike detector
- > Fri: Spread timing advisor; execution integration

Week 32: ML Feature Engineering & Predictive Signals

Build ML-ready feature pipeline and train first predictive models using historical trade outcomes.

Features 50+	Model XGBoost	AUC >0.65	Latency <100ms
-----------------	------------------	--------------	-------------------

GOALS

- > Engineer 50+ features from price, volume, Greeks, IV, sentiment, calendar
- > Train XGBoost classifier: P(trade wins) given feature vector
- > Use SHAP values to explain model predictions and select top features
- > Integrate ML signal as 8th factor in entry scorer
- > Walk-forward validation ensures no look-ahead bias in ML training
- > Model retraining pipeline: weekly retrain on last 90 days of trade outcomes

TASKS

- > Create services/ml/feature_store.py: build_feature_vector(candidate) → 50+ features
- > pip install xgboost shap scikit-learn
- > train_model.py: train XGBoost on historical trade outcomes
- > compute_shap(): identify top-15 features by importance
- > Add ml_signal_score to entry_scorer (weight: 12/100 points)
- > POST /api/v1/ml/retrain → trigger retraining; GET /api/v1/ml/model-info → current model stats
- > Weekly retrain scheduler job at Sunday midnight

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week32_feature.py --dry-run-> python3 -m pytest tests/test_week32*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week32_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week32_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > 50+ feature vector built for every candidate
- > XGBoost model trained on historical trades (AUC > 0.65)
- > SHAP values show top features (expected: IV_rank, DTE, delta, VIX)

-> ML signal integrated into entry scorer

AUTOPILOT LOGIC

- > ML signal is an 8th validation layer — adds information beyond rule-based scoring
- > SHAP explanations shown in run artifact trace: 'ML rejected this due to high gamma exposure'
- > Model performance tracked weekly: if AUC drops below 0.60, alert and switch to rule-based only

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Overfitting on small dataset — use 3-fold time-series CV; regularization $\alpha=0.5$
- > Risk: ML model too slow → cache model in memory; precompute feature vector during scan phase

DAY-BY-DAY BREAKDOWN

- > Mon: Feature store with 50 features
- > Tue: XGBoost training pipeline
- > Wed: SHAP values; feature importance analysis
- > Thu: ML signal integrated into entry scorer
- > Fri: Weekly retrain job; model info API; AUC tracking

Week 33: Real-Time P&L; Dashboard & Risk Metrics Display

Build a live trader's dashboard: real-time P&L;, drawdown meter, win-streak tracker, and daily risk status.

P&L; Update 5s	Risk Gauges 6	History 90 days	Alert Threshold-based
-------------------	------------------	--------------------	--------------------------

GOALS

- > Real-time unrealized P&L; for all open positions, updated every 5 seconds
- > Realized P&L; chart: daily bar chart for last 90 days
- > 6 live risk gauges: daily loss vs limit, portfolio delta, vega exposure, drawdown%, win rate, streak
- > Win/loss streak tracker with longest streak records
- > Daily VaR (Value at Risk) estimate using historical simulation
- > Alert when daily loss > 75% of daily limit (early warning)

TASKS

- > Expand positions WebSocket: push unrealized_pnl every 5s
- > Build PnlDashboardView.tsx: 90-day P&L; bar chart, open positions live table
- > 6 risk gauges using gauge chart (d3 or recharts)
- > VaR calculation: historical simulation on last 30 days trade returns
- > Alert on daily loss threshold: 75% warn, 100% block
- > Streak tracker: compute current streak from DB trade history

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week33_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week33*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week33_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week33_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Unrealized P&L; updates every 5s via WebSocket
- > 90-day P&L; chart renders correctly

- > All 6 risk gauges display live data
- > Win streak and VaR shown prominently on dashboard

AUTOPILOT LOGIC

- > VaR estimate shown as 'Expected worst day' — autopilot uses 95th-pct VaR for budget compliance
- > Consecutive loss counter → if 3 consecutive losses, autopilot pauses and sends alert for review
- > Dashboard is command center — shows everything needed to judge autopilot performance at a glance

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: 5s P&L; polling expensive with many positions → use DB snapshot; WebSocket pushes delta
- > Risk: VaR underestimates tail risk → supplement with CVaR (expected shortfall) for better risk view

DAY-BY-DAY BREAKDOWN

- > Mon: WebSocket P&L; streaming
- > Tue: 90-day P&L; bar chart
- > Wed: 6 risk gauges
- > Thu: VaR calculation + daily alert
- > Fri: Win streak tracker; dashboard layout polish

Week 34: Options Strategy Optimizer & Parameter Tuning

Auto-optimize strategy parameters (strikes, DTEs, widths) using backtest results as fitness function.

Parameters 8 tunable	Method Bayesian BO	Frequency Weekly	Improvement >5% Sharpe
-------------------------	-----------------------	---------------------	---------------------------

GOALS

- > Build StrategyOptimizer: tune 8 parameters (delta, DTE, width, profit target, stop, etc.)
- > Use Bayesian optimization (optuna) to find parameters that maximize OOS Sharpe
- > Run optimizer weekly on Sunday using last 60 days of walk-forward results
- > Implement parameter bounds: prevent optimizer from reaching unrealistic values
- > Show optimizer results: current vs recommended parameters with Sharpe delta
- > Apply new parameters after user approval (one-click in ParameterView)

TASKS

- > pip install optuna
- > Create services/backtest/optimizer.py: StrategyOptimizer using Optuna
- > Define 8 parameter search space: delta (0.1–0.35), DTE (14-45), width (1-5 strikes), etc.
- > Objective function: run walk-forward with given params, return negative OOS Sharpe
- > POST /api/v1/optimizer/run → async optimization job (30-min job)
- > GET /api/v1/optimizer/results → current + recommended params with comparison
- > ParameterView.tsx: parameter table with recommended changes and approve button

COMMANDS & FILES

- | | |
|---|---|
| <ul style="list-style-type: none"> -> python3 scripts/run_week34_feature.py --dry-run -> python3 -m pytest tests/test_week34*.py -v -> python3 scripts/run_autopilot_dry_run.py -> make lint && make test | <ul style="list-style-type: none"> -> services/core.py — core business logic -> services/router.py — FastAPI router for this feature -> tests/test_.py — unit + integration tests -> scripts/run_week34_feature.py — CLI to exercise the feature -> frontend/src/pages/Page.tsx — UI component -> docs/week34_design.md — design decisions + API contract |
|---|---|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|---|
| <ul style="list-style-type: none"> -> FEATURE_ENABLED=true -> FEATURE_TIMEOUT_SECONDS=30 -> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none"> -> Core: fastapi, sqlalchemy, pydantic -> Data: pandas, numpy -> Testing: pytest, httpx, playwright -> Feature-specific: see week title for specialized packages |
|---|---|

DELIVERABLES

- > Optuna optimizer runs and finds better parameters than defaults (tested)
- > Optimizer completes in <30 minutes for 8 parameters, 100 trials
- > ParameterView shows before/after Sharpe comparison

- > One-click parameter update applies new values to live config

AUTOPILOT LOGIC

- > Optimizer is the autopilot's self-improvement mechanism — gets better weekly
- > Parameter changes applied with full audit trail: who changed, from what, to what, why (optimizer trial ID)
- > If optimizer finds no improvement ($\Delta < 1\%$), keep current parameters and log result

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in $< 500\text{ms}$ p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Optimizer overfits to recent 60 days → use nested CV with held-out validation window
- > Risk: Optimizer applies harmful parameters without review → always require manual approval step

DAY-BY-DAY BREAKDOWN

- > Mon: Optuna integration; parameter space definition
- > Tue: Walk-forward as objective function
- > Wed: Optimizer API endpoint (async job)
- > Thu: ParameterView with comparison chart
- > Fri: Sunday optimizer scheduler; approval workflow

Week 35: Multi-Account & Paper-to-Live Transition Framework

Support multiple broker accounts and build a safe, gated process for transitioning from paper to live trading.

Accounts 3 supported	Gate 10-step checklist	Shadow 2 weeks	Rollback 1-click
-------------------------	---------------------------	-------------------	---------------------

GOALS

- > Support 3 account modes: paper (Alpaca), live (Alpaca), shadow (paper+live parallel)
- > Build 10-step go-live checklist: backtest passes, MC passes, 2-week paper profit, risk gates set, etc.
- > Shadow mode: execute on paper AND live simultaneously for 2 weeks for calibration
- > One-click rollback: instantly switch from live to paper if anomaly detected
- > Account selection UI: show current mode prominently; require confirmation to switch to live
- > Go-live readiness score: 0-100% based on checklist completion

TASKS

- > Refactor broker layer: BrokerConfig(mode='paper'|'live'|'shadow')
- > Shadow mode executor: sends orders to both accounts, tracks live vs paper P&L; divergence
- > Build go_live_checklist.py: 10 validation checks
- > GET /api/v1/accounts/readiness → readiness score + checklist status
- > AccountSwitcherModal.tsx: shows readiness score, checklist, requires 'I UNDERSTAND LIVE RISK' confirmation
- > Rollback endpoint: POST /api/v1/accounts/rollback → switch to paper immediately

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week35_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week35*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week35_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week35_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Paper/live/shadow modes switch correctly
- > Go-live readiness score computed from 10 checklist items

- > Shadow mode tracks live vs paper divergence
- > Rollback switches to paper in <5 seconds

AUTOPILOT LOGIC

- > Shadow mode is the FINAL test before live — run 2 weeks minimum
- > Divergence alert: if live P&L; diverges >10% from paper, trigger alert and review
- > Rollback is available 24/7 — even if autopilot is mid-cycle, emergency rollback aborts and switches

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Accidentally enabling live mode → require 3 confirmations + typed 'LIVE TRADING' phrase
- > Risk: Shadow mode executes double the trades cost → track paper fills separately; live fills are the alpha

DAY-BY-DAY BREAKDOWN

- > Mon: Multi-account broker abstraction
- > Tue: Go-live readiness checklist
- > Wed: Shadow mode executor
- > Thu: AccountSwitcherModal with readiness score
- > Fri: Rollback endpoint; divergence alerting

Week 36: Advanced Charting: Multi-Frame Analysis & Strategy Overlay

Upgrade the chart panel with multi-timeframe view, strategy overlays, and technical indicator suite.

Timeframes 6	Indicators 15+	Overlays Active trades	Performance 60fps
-----------------	-------------------	---------------------------	----------------------

GOALS

- > 6-timeframe chart: 1m, 5m, 15m, 1h, 4h, 1d all available via tab switcher
- > 15+ technical indicators: SMA, EMA, VWAP, Bollinger, RSI, MACD, ATR, IV overlay, etc.
- > Strategy overlay: show entry/exit points for all closed trades on historical chart
- > Entry signal overlay: mark where autopilot considered and rejected candidates
- > Volume profile panel: horizontal volume by price for current session
- > Chart alerts: user can click on chart to place price alert at that level

TASKS

- > Extend ChartPanel.tsx: timeframe switcher tabs
- > Indicator library: implement all 15 as overlays on lightweight-charts
- > Trade overlay: plot entry/exit arrows on chart for closed positions
- > Click-to-alert: chart click → opens 'Set Alert at \$X' modal
- > Volume profile: compute volume by price bucket for visible chart range
- > Performance: ensure 60fps on 6-month daily chart with all indicators enabled

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week36_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week36*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week36_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week36_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > All 6 timeframes switch without page reload
- > All 15 indicators render correctly

- > Trade entry/exit arrows visible on chart
- > Click-to-alert creates price alert from chart

AUTOPILOT LOGIC

- > Trade overlays make autopilot decisions transparent — see exactly where it entered/exited
- > RSI + MACD add a technical filter layer to complement the options Greeks-based scoring
- > IV overlay on price chart shows IV spikes at key price levels — critical context for options traders

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: 15 indicators slow down chart → compute indicators server-side; send pre-aggregated data
- > Risk: Clicking chart on mobile is imprecise → increase click target radius for price alerts

DAY-BY-DAY BREAKDOWN

- > Mon: Timeframe switcher + data fetching per timeframe
- > Tue: 8 indicator implementations
- > Wed: Remaining 7 indicators; performance optimization
- > Thu: Trade overlays; entry signal markers
- > Fri: Click-to-alert; volume profile panel

Week 37: Compliance Logging & Regulatory Audit Trail

Build a tamper-proof audit trail for all trading decisions, meeting retail broker compliance standards.

Immutability Write-once	Coverage 100% trades	Export CSV+PDF	Retention 7 years
----------------------------	-------------------------	-------------------	----------------------

GOALS

- > Implement immutable audit log: every trade decision, execution, and modification logged permanently
- > Audit records include: timestamp, decision_type, inputs, outputs, llm_reasoning, validator_results
- > Generate compliance report: exportable CSV and PDF for any date range
- > Implement 7-year retention policy: archive to cold storage after 1 year
- > Tamper detection: SHA-256 hash chain on audit records — any modification is detectable
- > Hide/anonymize sensitive data in exports (mask API keys, broker account numbers)

TASKS

- > Create compliance_audit_log table: append-only with SHA-256 hash chain
- > AuditLogger.log(event_type, payload) → hashes with previous record's hash
- > GET /api/v1/compliance/audit?start=&end;= → paginated audit records
- > POST /api/v1/compliance/export?format=csv|pdf → download compliance report
- > Implement archive_old_records(): move records >1yr to audit_archive table
- > Hash chain verifier: verify_chain() → confirms no tampering

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week37_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week37*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week37_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week37_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Audit log populated for every trade cycle (tested with 5 cycles)
- > Hash chain verifier detects tampered record (test case)
- > CSV export generates correct compliance report
- > Archive job moves records older than 1 year

AUTOPILOT LOGIC

- > Compliance log is the ultimate safety net — proves autopilot followed all rules
- > Every LLM decision logged with full reasoning → no black-box trades
- > Regulators (or user's accountant) can reconstruct any trade decision from audit trail alone

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Audit log grows very large → partition by year; compress archived partitions with zstd
- > Risk: Hash chain too slow for high-frequency logging → compute hash async; batch every 100 records

DAY-BY-DAY BREAKDOWN

- > Mon: Audit log table with hash chain
- > Tue: AuditLogger with all event types
- > Wed: Compliance report export (CSV + PDF)
- > Thu: Hash chain verifier; tamper test
- > Fri: Archive job; retention policy implementation

Week 38: Q3 Integration: Live Paper Trading Month

First full month of live paper trading: monitor, learn, and iterate on autopilot performance daily.

Cycles 22 trading days	Win Rate Target >60%	Avg P&L; >\$0 net	Incidents <3
---------------------------	-------------------------	----------------------	-----------------

GOALS

- > Run autopilot in paper mode for 22 trading days straight without manual intervention
- > Track every cycle: run_id, candidates_considered, candidates_selected, orders_placed, orders_filled
- > Daily review: compare actual vs expected outcomes; identify pattern discrepancies
- > Build LearningLog: capture surprises (trade that should have won but lost, etc.)
- > Adjust config parameters if win rate < 55% after 10 cycles
- > Reach 100 total paper trades milestone with full attribution analysis

TASKS

- > Enable APScheduler full-day operation for 4 weeks
- > Morning review ritual: check dashboard before market open, review overnight EOD report
- > LearningLog.md: daily journal of observations, surprises, and adaptations
- > Incident log: record any cycle failure, broker error, or unexpected behavior
- > After 10 cycles: run optimizer on data so far; update parameters if needed
- > After 22 cycles: comprehensive performance attribution by strategy, symbol, day-of-week

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week38_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week38*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week38_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week38_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 22 cycles run without critical failure
- > 100+ paper trades executed
- > Win rate, P&L, Sharpe computed for the month
- > LearningLog.md with daily observations

- > Performance attribution report

AUTOPILOT LOGIC

- > This week IS the autopilot — observing its behavior is the primary deliverable
- > LLM agreement rate for the month: track what % of cycles both LLMs agreed
- > Best and worst trades analyzed with full run artifact replay

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Autopilot makes many poor trades → immediately run optimizer; reduce position size
- > Risk: Broker API changes break execution → ensure broker abstraction catches and alerts on API errors

DAY-BY-DAY BREAKDOWN

- > Mon (rolling): Pre-market review; post-market analysis; update LearningLog
- > Tue: Focus on win rate vs expected
- > Wed: Mid-week parameter check
- > Thu: Strategy performance breakdown
- > Fri: Weekly summary; optimization if needed

Week 39: Sentiment Analysis 2.0: Social + Institutional Flow

Upgrade sentiment to include Reddit/Twitter social sentiment and options flow (unusual activity) signals.

Sources 5 sentiment	Dark Pool Yes	Social Reddit+X	Signal 0-100 composite
------------------------	------------------	--------------------	---------------------------

GOALS

- > Integrate Reddit WSB sentiment: use PRAW to fetch mentions + upvote velocity
- > Integrate Twitter/X sentiment via API Basic tier (or nitter scraping as fallback)
- > Options flow: integrate Unusual Whales or Tradier unusual activity endpoint
- > Dark pool print detector: large block trades as institutional signal
- > Composite sentiment score: 5-source weighted average (news 40%, social 20%, flow 30%, dark pool 10%)
- > Unusual activity alerts: when options flow is unusually bullish/bearish vs historical

TASKS

- > pip install praw
- > reddit_sentiment.py: fetch WSB mentions, run FinBERT, compute symbol sentiment
- > twitter_sentiment.py: search symbol mentions, FinBERT scoring
- > options_flow.py: Unusual Whales API or Tradier screener for unusual activity
- > darkpool.py: detect large block prints from tick data
- > Composite scorer: weighted combination of all 5 sources
- > GET /api/v1/sentiment/{symbol} → full sentiment breakdown by source

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week39_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week39*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week39_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week39_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Reddit sentiment computed for top 20 universe symbols

- > Options flow unusual activity flag working (test case)
- > Composite sentiment score integrates all 5 sources
- > Unusual activity alert fires when flow strongly bullish (test)

AUTOPILOT LOGIC

- > Social sentiment adds contrarian signal: extreme WSB bullishness = be cautious (fade retail)
- > Unusual options flow: strongly bullish flow + autopilot's bearish setup = SKIP trade
- > Institutional dark pool prints: large buy prints before earnings = avoid short premium near expiry

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Reddit API rate limits → use PRAW with OAuth; cache for 15 minutes per symbol
- > Risk: Social sentiment noisy → use only at high-conviction threshold (>80/100 score); low weight

DAY-BY-DAY BREAKDOWN

- > Mon: Reddit PRAW sentiment integration
- > Tue: Options flow unusual activity
- > Wed: Dark pool print detector
- > Thu: Composite scorer + flow alert
- > Fri: Sentiment dashboard UI; API endpoint

Week 40: Portfolio Heat Map & Sector Dashboard

Build visual portfolio heat map and sector breakdown for instant risk visualization.

Tiles All positions	Color P&L-based;	Sector GICS 11	Refresh 30s
------------------------	---------------------	-------------------	----------------

GOALS

- > Portfolio heat map: tiles sized by notional value, colored by unrealized P&L;
- > Sector donut chart: % of portfolio in each GICS sector, updated with each position change
- > Position summary table: sortable by P&L;, DTE, strategy type, sector
- > Quick filters: show only options, only stock, only positive P&L;
- > Export: positions table downloadable as CSV for tax/reporting
- > Watchlist integration: see symbols you're watching but not yet in

TASKS

- > HeatMapTile.tsx: responsive grid with variable-size tiles
- > Color gradient: red (loss >10%) → white (flat) → green (profit >10%)
- > SectorDonut.tsx: recharts pie chart with GICS colors
- > PositionsTable.tsx: sortable columns, filter dropdown
- > GET /api/v1/positions/export?format=csv
- > WatchlistPanel.tsx: manage 20-symbol watchlist with alerts

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week40_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week40*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week40_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week40_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Heat map renders all positions with correct sizing + coloring
- > Sector donut updates after each trade
- > Position table filterable and sortable
- > CSV export downloads correctly

AUTOPILOT LOGIC

- > Heat map at a glance: autopilot portfolio health visible in 2 seconds
- > Red tiles = positions autopilot should consider closing → used in exit prioritization
- > Sector donut auto-balances autopilot's new trade selection toward under-represented sectors

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Heat map slow with many positions → use CSS grid GPU-accelerated layout; avoid DOM re-renders

DAY-BY-DAY BREAKDOWN

- > Mon: HeatMapTile component
- > Tue: SectorDonut chart
- > Wed: PositionsTable with filters
- > Thu: CSV export; watchlist panel
- > Fri: Integration; 30s refresh via WebSocket

Week 41: Advanced Risk Metrics: VaR, CVaR & Stress Testing Dashboard

Implement institution-grade risk metrics available in a dedicated Risk Control Room view.

VaR 99% 1-day	CVaR Yes	Stress Tests 7	Dashboard Full view
------------------	-------------	-------------------	------------------------

GOALS

- > Daily VaR at 99% confidence using historical simulation on 252 days
- > CVaR (Conditional Value at Risk) = average loss in the worst 1% of days
- > Stress test portfolio against 7 scenarios: 2008 GFC, 2020 COVID, 2022 Rate Hike, etc.
- > Dollar-amount display: 'Worst expected day: -\$2,340' is more intuitive than percentages
- > RiskControlRoom.tsx: full-page risk dashboard with all metrics
- > Build risk score (0-100): aggregate measure of current portfolio risk level

TASKS

- > Extend monte_carlo.py: historical_var(), cvar(), stress_tests()
- > Stress test: modify returns distribution to match each historical crisis period
- > GET /api/v1/risk/dashboard → VaR, CVaR, stress results, risk score
- > RiskControlRoom.tsx: VaR gauge, CVaR card, 7 stress panels, risk score ring
- > Risk score formula: weighted combo of VaR%, max_drawdown%, greek_exposure, concentration
- > Route /risk in React Router; add to nav sidebar

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week41_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week41*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week41_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week41_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > VaR and CVaR computed correctly (verified on sample portfolio)
- > All 7 stress scenarios produce results

- > Risk score aligns with intuitive portfolio riskiness
- > RiskControlRoom view accessible from nav

AUTOPILOT LOGIC

- > Risk score gate: if risk score > 75, autopilot reduces position size to 50%
- > Stress test worst-case shown in pre-market briefing: 'If 2020-COVID repeat: -\$5,400'
- > CVaR is more conservative than VaR — used for position sizing decisions

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Stress test assumes stationary returns → add regime-specific stress overlay for accuracy
- > Risk: Risk score weighting arbitrary → document formula; allow config adjustment

DAY-BY-DAY BREAKDOWN

- > Mon: VaR + CVaR computation
- > Tue: 7 stress scenarios
- > Wed: Risk score formula
- > Thu: RiskControlRoom.tsx
- > Fri: VaR gate in autopilot position sizing

Week 42: SEC Filing & Earnings Call Transcript Analysis

Analyze 10-K, 10-Q filings and earnings call transcripts with LLM for fundamental risk signals.

Coverage S&P; 500 10-Qs	Analysis LLM-based	Latency <30s	Signals Qualitative risk flags
----------------------------	-----------------------	-----------------	-----------------------------------

GOALS

- > Fetch latest 10-Q and earnings call transcripts via SEC EDGAR API
- > Use Gemini (large context) to analyze filing for risk flags: liquidity, guidance cuts, lawsuits
- > Compute fundamental_risk_score per symbol: 0-100 (100=high risk)
- > High fundamental risk → reduce position size or skip entry
- > Cache filing analysis for 90 days (filings don't change)
- > Show filing analysis summary in SymbolDetail panel

TASKS

- > sec_edgar.py: fetch latest 10-Q text for any symbol via EDGAR full-text search
- > transcript_fetcher.py: earnings call transcript via Seeking Alpha scraper or Motley Fool RSS
- > gemini_analyst.py: 5-factor filing analysis prompt (liquidity, guidance, legal, management, competitive)
- > fundamental_risk_score: weighted average of 5 factors
- > Cache analysis in fundamentals table with 90-day TTL
- > SymbolDetailPanel.tsx: show filing analysis with risk flags

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week42_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week42*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week42_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week42_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 10-Q filing fetched for AAPL and MSFT
- > Gemini analysis returns risk flags correctly
- > Fundamental risk score integrated into entry scoring
- > Filing summary visible in symbol detail panel

AUTOPILOT LOGIC

- > High-fundamental-risk symbols (score >70) get position size reduced by 50%
- > Filing analysis runs quarterly — autopilot knows when each company last filed
- > LLM analysis of filings: one of the most sophisticated features of the system

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Gemini context limit for long filings → chunk filing into sections; summarize each section first
- > Risk: EDGAR API throttling → cache aggressively; respect rate limits with 0.5s delays

DAY-BY-DAY BREAKDOWN

- > Mon: SEC EDGAR filing fetcher
- > Tue: Earnings transcript fetcher
- > Wed: Gemini 5-factor analysis prompt
- > Thu: Fundamental risk score + cache
- > Fri: SymbolDetailPanel integration; entry scoring weight

Week 43: Options Term Structure & Roll Strategy Engine

Build automated roll detection and execution: roll losing positions to next month before expiry.

Roll Trigger DTE<7	Rollover Automated	Cost <\$50 per roll	Coverage All strategies
-----------------------	-----------------------	------------------------	----------------------------

GOALS

- > Term structure analysis: compare IV across expiry months to find optimal roll target
- > Roll trigger: when position DTE < 7, evaluate rolling vs closing
- > Automated roll: close current position, open new position 30 DTE in next month
- > Roll cost analysis: net debit/credit of rolling must be favorable
- > Roll simulation: backtest roll strategy vs close-and-reopen on historical data
- > RollView: upcoming rolls, roll candidates, roll history

TASKS

- > term_structure.py: build IV term structure curve from chain across expirations
- > roll_detector.py: scan positions for DTE<7; evaluate roll attractiveness
- > RollEvaluator: compute roll P&L; impact; recommend roll vs close
- > Automated roll executor: close → reopen in next expiry as atomic pair
- > Backtest roll strategy: compare rolling vs closing on last 2 years data
- > GET /api/v1/rolls/candidates → positions approaching roll threshold

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week43_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week43*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week43_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week43_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Roll trigger fires correctly when position DTE < 7
- > Automated roll executes close + reopen atomically
- > Term structure chart shows IV curve across expirations
- > Roll history tracked in DB with cost per roll

AUTOPILOT LOGIC

- > Rolling extends position life and reduces realized losses — key to long-term profitability
- > Roll calendar: autopilot plans rolls 5 days in advance — not a surprise
- > Roll costing: only roll if net credit or small debit (<\$20); otherwise close and redeploy

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Roll target month has bad IV → skip roll, close position instead
- > Risk: Execution slip during roll (close fills, open fails) → treat as partial roll; monitor open leg

DAY-BY-DAY BREAKDOWN

- > Mon: Term structure analysis + IV curve
- > Tue: Roll detector and evaluator
- > Wed: Automated roll executor (atomic)
- > Thu: Roll backtest comparison
- > Fri: RollView UI; roll scheduler job

Week 44: System Monitoring & Observability Stack

Add production-grade monitoring: Prometheus metrics, Grafana dashboard, health probes, and PagerDuty alerts.

Metrics 50+ exposed	Dashboards 3 Grafana	Uptime 99.9%	Alert PagerDuty
------------------------	-------------------------	-----------------	--------------------

GOALS

- > Expose 50+ Prometheus metrics: cycle duration, order fill rate, API latency, error rate, memory/CPU
- > Grafana dashboards: System Health, Autopilot Performance, Trading P&L;
- > Health probe: /health endpoint returns comprehensive status for all dependencies
- > PagerDuty integration: on-call alert for CRITICAL incidents (system down, kill switch activated)
- > Log aggregation: structured JSON logs → file rotation → optional ELK stack
- > SLA targets: API P99 < 200ms, cycle duration < 90s, uptime > 99.9%

TASKS

- > pip install prometheus-client
- > Add prometheus_client metrics to all service layers
- > prometheus_metrics.py: Histogram for latencies, Counter for orders, Gauge for positions/pnl
- > FastAPI middleware: auto-instrument all endpoints
- > /metrics endpoint: returns Prometheus scrape format
- > docker-compose: add prometheus + grafana services
- > 3 Grafana dashboard JSON files: system, autopilot, trading

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week44_feature.py --dry-run-> python3 -m pytest tests/test_week44*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week44_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week44_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > 50+ metrics exposed at /metrics
- > Grafana shows real-time system health
- > PagerDuty fires on CRITICAL alert (test case)

-> Structured JSON logs in all service modules

AUTOPILOT LOGIC

- > cycle_duration_seconds histogram → P99 visible in Grafana → catch slow cycles early
- > orders_placed_total counter → tracks autopilot activity over time
- > Grafana alert: if cycle_success_rate < 0.8 over 1 hour → PagerDuty page

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Prometheus/Grafana adds complexity → use single docker-compose service; document setup
- > Risk: PagerDuty billing → use free tier; limit to <5 alerts/month threshold

DAY-BY-DAY BREAKDOWN

- > Mon: Prometheus client + 30 core metrics
- > Tue: Remaining 20 metrics; /metrics endpoint
- > Wed: Docker Compose prometheus+grafana
- > Thu: 3 Grafana dashboards
- > Fri: PagerDuty integration; structured logging

Week 45: Multi-Strategy Portfolio: All 6 Strategy Types Live

Enable all 6 strategy types in production with intelligent allocation weighting by regime.

Strategies 6 types	Allocation Regime-based	Max/Type 30%	Concurrency 6 parallel scans
-----------------------	----------------------------	-----------------	---------------------------------

GOALS

- > Enable all 6 strategies: PutCreditSpread, CallCreditSpread, IronCondor, IronButterfly, CalendarSpread, DiagonalSpread
- > Regime-based allocation: allocate portfolio \$ to each strategy based on current regime
- > Run 6 parallel scans: each strategy type scanned concurrently using asyncio
- > Strategy performance tracking: individual P&L, win rate, Sharpe per strategy
- > Auto-deactivate strategy if: OOS backtest Sharpe < 0.5 for 30 days running
- > Portfolio max-per-strategy cap: 30% notional in any single strategy type

TASKS

- > Implement CalendarSpread and DiagonalSpread option chain builders + Greeks
- > 6-parallel scan: `asyncio.gather(*[scan(s,data) for s in strategies])`
- > AllocationManager: compute \$ allocation per strategy from regime config
- > Strategy health monitor: daily check of per-strategy rolling Sharpe
- > Auto-deactivate: if `strategy_health < threshold`, set `status=PAUSED` in config
- > StrategyAllocationView.tsx: donut chart + allocation table

COMMANDS & FILES

- | | |
|---|---|
| <ul style="list-style-type: none"> -> <code>python3 scripts/run_week45_feature.py --dry-run</code> -> <code>python3 -m pytest tests/test_week45*.py -v</code> -> <code>python3 scripts/run_autopilot_dry_run.py</code> -> <code>make lint && make test</code> | <ul style="list-style-type: none"> -> <code>services/core.py</code> — core business logic -> <code>services/router.py</code> — FastAPI router for this feature -> <code>tests/test_.py</code> — unit + integration tests -> <code>scripts/run_week45_feature.py</code> — CLI to exercise the feature -> <code>frontend/src/pages/Page.tsx</code> — UI component -> <code>docs/week45_design.md</code> — design decisions + API contract |
|---|---|

API ENDPOINTS TO BUILD

- > GET `/api/v1/` — list or status endpoint
- > POST `/api/v1/` — create or trigger endpoint
- > GET `/api/v1/{id}` — detail endpoint
- > DELETE `/api/v1/{id}` — delete or cancel endpoint

TESTS TO WRITE

- > `test_happy_path.py`: nominal flow works end to end
- > `test_edge_cases.py`: empty input, null values, boundary conditions
- > `test_error_handling.py`: invalid input returns 400 with clear message
- > `test_integration.py`: feature integrates with autopilot pipeline
- > `playwright/test_ui.py`: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|--|---|
| <ul style="list-style-type: none"> -> <code>FEATURE_ENABLED=true</code> -> <code>FEATURE_TIMEOUT_SECONDS=30</code> -> <code>FEATURE_MAX_RETRIES=3</code> | <ul style="list-style-type: none"> -> Core: fastapi, sqlalchemy, pydantic -> Data: pandas, numpy -> Testing: pytest, httpx, playwright -> Feature-specific: see week title for specialized packages |
|--|---|

DELIVERABLES

- > All 6 strategies execute correctly in paper mode
- > Regime-based allocation shifts correctly (test: Bull → 0% IronCondor, 40% PutCredit)
- > Parallel scan completes 6 strategies in < time of sequential scan
- > Strategy auto-deactivation works (test: force low Sharpe strategy)

AUTOPILOT LOGIC

- > 6 strategies = 6 alpha streams — diversified autopilot income sources
- > Regime detection now controls ALL allocation, not just strategy preference
- > Auto-deactivation: autopilot effectively fires underperforming strategies on its own

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Iron Butterfly hard to fill at mid → require tighter spread rules; use limit orders only
- > Risk: Calendar and diagonal need multiple expirations → validate both legs before submission

DAY-BY-DAY BREAKDOWN

- > Mon: Implement Calendar + Diagonal strategy builders
- > Tue: Parallel scan with `asyncio.gather`
- > Wed: AllocationManager + regime weights
- > Thu: Strategy health monitor + auto-deactivation
- > Fri: StrategyAllocationView; E2E 6-strategy test

Week 46: AI-Powered Market Briefing & Natural Language Reports

Generate natural language pre-market and EOD briefings using LLM, replacing raw data tables with narrative.

Briefing 2 daily	Sources 7 data feeds	Format NL + structured	Delivery Push+App
---------------------	-------------------------	---------------------------	----------------------

GOALS

- > Pre-market briefing (8:30 AM): NL summary of overnight news, economic events, vol regime, day's setup
- > EOD briefing (4:05 PM): narrative of what happened, what the autopilot did, P&L; for the day
- > Use Gemini for briefing generation: structured prompt with all 7 data sources
- > Briefing style: Bloomberg-style concise prose, not bullet points
- > Push to Slack, email, and show in BriefingView in dashboard
- > Store briefings in DB; searchable by date

TASKS

- > briefing_generator.py: pre_market_briefing() and eod_briefing() functions
- > 7 data sources: VIX, SPY regime, earnings calendar, economic calendar, positions, P&L;, sentiment
- > Gemini prompt: role='financial analyst'; structured data injection; output NL paragraph
- > BriefingView.tsx: styled text view with date navigator
- > Scheduler: pre-market job at 8:30 AM, EOD at 4:05 PM
- > slack_post_briefing(): format and post to Slack channel

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week46_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week46*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week46_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week46_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Pre-market briefing generated correctly for test date
- > EOD briefing references actual day's trades

- > BriefingView renders formatted briefing
- > Slack delivery confirmed working

AUTOPILOT LOGIC

- > Briefings make autopilot's decisions legible to non-technical users: 'Today I placed 2 put credit spreads on AAPL due to high IV rank and bullish regime'
- > Pre-market briefing shows confidence level for the coming day's session
- > EOD briefing includes 'Today's lesson' section: what worked, what didn't

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Gemini API unavailable at 8:30 AM → fallback to template-based structured briefing
- > Risk: Briefing contains false information from LLM → add disclaimer; user should verify key facts

DAY-BY-DAY BREAKDOWN

- > Mon: briefing_generator.py with 7 data sources
- > Tue: Gemini prompt engineering for NL quality
- > Wed: BriefingView.tsx
- > Thu: Slack + email delivery
- > Fri: Scheduler jobs; briefing DB storage

Week 47: API Rate Limiting, Caching & CDN Architecture

Harden the production API with multi-tier caching, rate limiting, and CDN delivery for the frontend.

Cache Hit Rate >70%	Rate Limit 100 req/min	CDN Frontend	TTL Strategy-based
------------------------	---------------------------	-----------------	-----------------------

GOALS

- > Multi-tier caching: in-memory (1s) → Redis (60s) → DB (permanent)
- > Tiered TTL strategy: stock quote 5s, option chain 30s, fundamentals 24h, reports 7d
- > API rate limiting: 100 req/min per client; 429 response with Retry-After header
- > CDN deployment: serve React build via Cloudflare Pages or Vercel
- > Cache invalidation: WebSocket events invalidate relevant cache keys
- > Cache hit rate monitoring: expose cache stats at /metrics

TASKS

- > pip install redis
- > redis_cache.py: get(), set(), invalidate() with TTL param
- > Decorate all market data endpoints with @cache(ttl=30)
- > FastAPI middleware: rate limiter using redis sliding window
- > Modify Vite build: CNAME + deploy script to Cloudflare Pages
- > Cache stats: expose cache_hit_total and cache_miss_total Prometheus metrics

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week47_feature.py --dry-run-> python3 -m pytest tests/test_week47*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week47_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week47_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > Redis cache reduces redundant Tradier API calls by >70%
- > Rate limiter returns 429 after 100 requests/min (tested)
- > Frontend served from CDN with <100ms global TTFB
- > Cache hit rate visible in Grafana

AUTOPILOT LOGIC

- > Caching makes the scan phase faster: option chains cached 30s, reducing Tradier calls
- > Cache invalidation on position update: ensure positions view always fresh
- > In paper mode: simulate redis with in-memory dict if Redis unavailable

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Redis unavailable → fallback to in-memory cache; warn in logs
- > Risk: Stale cache causes bad decisions → for real-time data (quotes), keep TTL very short (5s max)

DAY-BY-DAY BREAKDOWN

- > Mon: Redis client integration; basic get/set
- > Tue: Cache decorator + TTL strategy
- > Wed: Rate limiter middleware
- > Thu: Frontend CDN deployment
- > Fri: Cache stats in Prometheus; Grafana panel

Week 48: Automated Strategy Research & Alpha Discovery

Build a research mode where the system autonomously discovers and tests new strategy variants.

Variants 50/week	Auto-test Walk-forward	Promote If Sharpe>1.5	Report Weekly digest
---------------------	---------------------------	--------------------------	-------------------------

GOALS

- > Research mode: generate 50 strategy variants by perturbing parameters
- > Auto-test each variant with accelerated walk-forward backtest
- > Rank variants by OOS Sharpe; promote top-3 to staging for paper testing
- > Research report: weekly email digest of new strategy discoveries
- > Parameter mutation engine: systematic exploration of parameter space
- > Human review gate: all promoted strategies require user approval before paper testing

TASKS

- > research_mode.py: variant_generator() — produces 50 param combos per base strategy
- > Accelerated walk-forward: 5-year OOS on 6 months training (faster than full WF)
- > StrategyRegistry: promoted strategies tracked with status (research/staging/production)
- > Research report generator: summarize top-5 variants with backtest stats
- > POST /api/v1/research/run → trigger research cycle
- > ResearchView.tsx: variant ranking table; promote/dismiss buttons

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week48_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week48*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week48_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week48_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 50 variants generated and tested in <2 hours (Sunday night job)
- > Top-3 variants correctly identified and staged for paper testing
- > Research report email sent with top-5 discoveries
- > ResearchView shows all variant results

AUTOPILOT LOGIC

- > Research mode is the autopilot evolving itself — systematic alpha hunting
- > Each promoted variant runs in shadow mode (paper only) for 4 weeks before production consideration
- > Strategy mutation: not random — systematic grid around best current parameters

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Promote a variant based on lucky backtest period → require 3-year OOS minimum
- > Risk: Research report over-hypes findings → show confidence intervals; always caveat OOS

DAY-BY-DAY BREAKDOWN

- > Mon: Variant generator engine
- > Tue: Accelerated walk-forward for rapid testing
- > Wed: StrategyRegistry + promotion logic
- > Thu: ResearchView + research report email
- > Fri: Research scheduler (Sunday night); E2E test

Week 49: Position Sizing Engine: Kelly Criterion & Volatility Scaling

Implement advanced position sizing: fractional Kelly, volatility targeting, and dynamic sizing by regime.

Method Fractional Kelly	Vol Target 12% annual	Dynamic Per-regime	Risk-Adj Per trade
----------------------------	--------------------------	-----------------------	-----------------------

GOALS

- > Replace fixed position sizing with Kelly Criterion-based dynamic sizing
- > Fractional Kelly (0.25x) to reduce variance while capturing edge
- > Volatility targeting: scale position size up/down to maintain 12% annual portfolio vol
- > Regime-based sizing: larger positions in high-confidence regimes (Neutral, low VIX)
- > Per-trade risk budget: no single trade risks more than 2% of portfolio
- > Show sizing recommendation in trade approval flow: 'Recommended: 2 contracts'

TASKS

- > kelly_calculator.py: fractional_kelly(win_rate, avg_win, avg_loss) → position_size_fraction
- > vol_scaler.py: compute current portfolio vol; scale new positions to maintain target
- > regime_sizer.py: multiply Kelly by regime confidence (0.7x Bear, 1.0x Neutral, 0.5x HighVol)
- > Integrate into cycle: position_qty = min(kelly_qty, max_risk_qty, max_notional_qty)
- > Show sizing breakdown in RunArtifact: Kelly=X, VolAdj=Y, RegimeAdj=Z, Final=N
- > Test: verify sizing reduces during high vol periods

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week49_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week49*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week49_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week49_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Kelly calculator produces correct quantities for test cases
- > Vol-targeting adjusts sizing when portfolio vol exceeds 12% (verified)

- > Regime sizing reduces positions in Bear/HighVol regimes
- > Sizing breakdown visible in run artifact

AUTOPILOT LOGIC

- > Position sizing is one of the most important autopilot improvements — affects long-run P&L;
- > Fractional Kelly (0.25x) protects against estimation error in win rate
- > Vol targeting makes portfolio drawdowns more predictable — key for sleep-at-night comfort

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Kelly formula gives huge position with small sample → minimum 30 trades before Kelly is trusted
- > Risk: Vol spikes intraday → size based on 20-day realized vol, not intraday — more stable

DAY-BY-DAY BREAKDOWN

- > Mon: Kelly calculator; fractional sizing
- > Tue: Volatility targeting scaler
- > Wed: Regime-based sizing multiplier
- > Thu: Integration into cycle execution
- > Fri: Sizing breakdown in RunArtifact; E2E test

Week 50: Full Security Hardening & Penetration Test

Conduct full security audit, pen test the API, and harden all attack surfaces for production readiness.

OWASP All 10 checked	Findings 0 Critical	Auth JWT+2FA	Secrets Vault-ready
-------------------------	------------------------	-----------------	------------------------

GOALS

- > JWT authentication for all API endpoints (currently open for dev)
- > 2FA option for dashboard login using TOTP (Google Authenticator)
- > OWASP ZAP full scan: automated pen test of all API endpoints
- > Secrets management: move API keys from keys.env → HashiCorp Vault or AWS Secrets Manager
- > SQL injection prevention: verify all ORM queries use parameterized queries
- > CSP headers: prevent XSS on frontend; CORS tightened to production domain only

TASKS

- > Implement JWT auth: POST /api/v1/auth/login → token; all routes require Bearer
- > TOTP 2FA: pyotp library; QR code setup flow in Settings view
- > OWASP ZAP scan: run in CI; fail pipeline on HIGH severity finding
- > HashiCorp Vault dev mode setup; migrate API key reads to vault.read()
- > Audit all SQLAlchemy models: no raw SQL; use ORM queries only
- > Add security headers middleware: CSP, HSTS, X-Frame-Options, X-Content-Type

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week50_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week50*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week50_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week50_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > JWT auth working for all API endpoints
- > 2FA setup flow working in Settings
- > Zero HIGH/CRITICAL findings from ZAP scan
- > Security headers verified via securityheaders.com (grade A)

AUTOPILOT LOGIC

- > API auth ensures autopilot API is not publicly accessible — no unauthorized trigger of trades
- > Vault integration: API keys lifecycle-managed — rotate without code change
- > Security hardening makes the system deployable to any cloud without exposure risk

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: JWT secret leaked → rotate immediately; invalidate all tokens; re-deploy
- > Risk: TOTP QR code exposed → HTTPS only for setup; delete QR after first scan

DAY-BY-DAY BREAKDOWN

- > Mon: JWT auth for all API routes
- > Tue: TOTP 2FA setup flow
- > Wed: OWASP ZAP scan + fix HIGH findings
- > Thu: HashiCorp Vault integration
- > Fri: Security headers; CSP; final scan

Week 51: Documentation Suite & Interactive API Reference

Build comprehensive documentation: user guide, API reference, runbook, and interactive demo.

Docs Pages 50+	API Ref Auto-generated	Demo Interactive	Coverage 100% features
-------------------	---------------------------	---------------------	---------------------------

GOALS

- > User guide: 10 chapters covering every feature with screenshots
- > API reference: OpenAPI/Swagger auto-generated, hosted at /docs
- > Developer guide: setup, architecture, contributing, testing
- > Runbook: step-by-step recovery procedures for every failure mode
- > Interactive demo: Storybook component library for all React components
- > README overhaul: badges, architecture diagram, quick start in <5 commands

TASKS

- > FastAPI already generates OpenAPI; enhance with detailed descriptions and examples
- > Docusaurus setup in docs/ directory: user guide chapters
- > Write 10 user guide chapters: Introduction, Dashboard Tour, Autopilot, Positions, Reports, etc.
- > Storybook: npx storybook init; write stories for all 20 components
- > Generate architecture diagram as Mermaid diagram in README
- > Runbook: 15 failure scenarios with exact remediation steps

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week51_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week51*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week51_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week51_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Docusaurus docs site builds successfully
- > All 20 components have Storybook stories
- > README has architecture diagram and quick-start
- > Runbook covers all 15 failure scenarios

AUTOPILOT LOGIC

- > Documentation makes the autopilot auditable — anyone can read exactly how it makes decisions
- > Interactive demos let new users explore without touching live system
- > Architecture diagram shows data flow: Market → Scan → LLM → Validate → Execute → Monitor

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Docs fall out of date quickly → add docs-check CI step; fail if API endpoints undocumented
- > Risk: Storybook adds build time → build Storybook only in docs CI pipeline, not main CI

DAY-BY-DAY BREAKDOWN

- > Mon: Docusaurus setup; first 5 user guide chapters
- > Tue: Remaining 5 chapters + runbook
- > Wed: Storybook for all components
- > Thu: README architecture diagram; quick-start
- > Fri: API example enhancements; docs CI check

Week 52: Year 1 Wrap-Up: Performance Review & Year 2 Planning

Comprehensive Year 1 retrospective, performance analysis, and detailed Year 2 planning with new objectives.

Paper Trades 500+ target	Review Complete	Y2 Plan Approved	Milestone Year 1 Complete
-----------------------------	--------------------	---------------------	------------------------------

GOALS

- > Comprehensive Year 1 performance attribution: 52 weeks of development + paper trading results
- > Calculate all-time paper trading stats: total trades, win rate, Sharpe, max drawdown, best/worst strategy
- > Architecture review: what worked, what needs redesign in Y2
- > Technical debt audit: categorize and prioritize all known debt items
- > Produce Year 2 Roadmap document: key themes, milestones, technology upgrades
- > Celebrate: tag v2.0-rc, write blog post / Devpost update, share publicly

TASKS

- > Run comprehensive_year_review.py: query all DB tables for Year 1 stats
- > Year 1 report PDF: generated by ReportEngine; includes all key charts
- > Architecture retrospective: identify components that need rewrite (tech debt)
- > Year 2 Roadmap: AI agent autonomy, real money trading, institutional features, platform expansion
- > git tag v2.0-rc1 with full release notes
- > Blog/LinkedIn post: 'What I built in 52 weeks' with key architecture decisions

COMMANDS & FILES

- > python3 scripts/run_week52_feature.py --dry-run
 - > python3 -m pytest tests/test_week52*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week52_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week52_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > Year 1 performance report PDF
- > Architecture retrospective document

- > Year 2 Roadmap approved
- > v2.0-rc1 tagged on GitHub
- > Blog post / public update published

AUTOPILOT LOGIC

- > Year 1 autopilot: 26 weeks of operation, 500+ paper trades, dual-LLM, 6 strategies — COMPLETE
- > Year 2 autopilot theme: move from rule-based to fully adaptive AI agent
- > Key Year 1 lesson captured: what parameters worked best, what strategies underperformed

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Year 1 paper results disappointing → use year 2 to reboot with better parameters; don't panic
- > Risk: Tech debt too high to proceed → allocate first 4 weeks of Y2 to refactoring sprint

DAY-BY-DAY BREAKDOWN

- > Mon: Run Year 1 performance analysis scripts
- > Tue: Write Year 1 report PDF; architecture retrospective
- > Wed: Year 2 Roadmap document
- > Thu: Tag v2.0-rc1; write release notes
- > Fri: Blog post; share publicly; plan Week 53

Week 53: Autonomous AI Agent Architecture (LangChain)

Refactor autopilot into a true LangChain agent with tool-use, memory, and dynamic planning.

Framework LangChain	Tools 12	Memory Window Buffer	Fallback Deterministic
------------------------	-------------	-------------------------	---------------------------

GOALS

- > Replace hardcoded pipeline with LangChain AgentExecutor and 12 callable tools
- > Agent memory: ConversationBufferWindowMemory (20-message window)
- > Dynamic planning: agent decides tool call order based on market context
- > Log every tool call with agent's rationale in run_artifact.agent_trace
- > Fallback: if agent hangs >120s, revert to deterministic pipeline
- > Agent dry-run mode: complete full cycle without order execution

TASKS

- > pip install langchain langchain-groq langchain-google-genai
- > services/autopilot/agent.py: ApexAgent class with 12 tool wrappers
- > Tool descriptions explicitly require: validate before execute
- > AgentExecutor with verbose=True; max_iterations=25
- > Store agent_trace JSON field in RunArtifact
- > Fallback handler: asyncio timeout 120s → deterministic pipeline

COMMANDS & FILES

- > python3 scripts/run_week53_feature.py --dry-run
 - > python3 -m pytest tests/test_week53*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week53_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week53_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > LangChain agent completes full dry_run cycle
- > Agent uses >8 tools per cycle
- > Fallback activates on timeout
- > Agent trace shows reasoning per tool

AUTOPILOT LOGIC

- > Agent IS the autopilot — it decides its own workflow now
- > Dynamic strategy: high-VIX day → agent may check risk first before scanning
- > Agent memory: remembers morning briefing when making afternoon decisions

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Agent loops → strict max_iterations=25; hard timeout 120s
- > Risk: Agent execute without validate → tool descriptions require validate first

DAY-BY-DAY BREAKDOWN

- > Mon: LangChain setup; 12 tool wrappers
- > Tue: AgentExecutor + verbose trace
- > Wed: Memory integration
- > Thu: Agent trace in RunArtifact
- > Fri: Fallback handler; E2E dry-run

Week 54: Multi-Agent System: Researcher + Validator + Executor

Split single agent into specialized sub-agents that debate every trade before execution.

Sub-agents 3	Protocol Debate	Supervisor LangGraph	Transcript Saved
-----------------	--------------------	-------------------------	---------------------

GOALS

- > ResearcherAgent: scans universe, builds candidate list with context
- > ValidatorAgent: adversarial — challenges each trade; seeks to reject
- > ExecutorAgent: final decision after hearing both sides
- > SupervisorAgent: routes messages via LangGraph StateGraph
- > Full debate transcript saved in run_artifact.debate_transcript
- > Debate completes in <60s total

TASKS

- > agent_researcher.py, agent_validator.py, agent_executor.py — separate system prompts + tools
- > StateGraph: researcher→validator→executor routing
- > Validator prompt: 'Find every reason NOT to take this trade'
- > Executor reads debate transcript + weighs both perspectives
- > debate_transcript stored as JSON in RunArtifact

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week54_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week54*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week54_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week54_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 3-agent pipeline completes full dry-run
- > Validator argues against ≥1 candidate
- > Executor sometimes overrules with reasoning

-> Debate transcript saved

AUTOPILOT LOGIC

- > Multi-agent = adversarial self-review = fewer bad trades
- > Validator effectively acts as the system's risk manager

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Slower than single agent → parallelize Researcher + Validator
- > Risk: Agents disagree on everything → tune Validator to focus on material risks only

DAY-BY-DAY BREAKDOWN

- > Mon: ResearcherAgent
- > Tue: ValidatorAgent adversarial prompt
- > Wed: ExecutorAgent
- > Thu: LangGraph supervisor routing
- > Fri: Debate transcript in RunArtifact; E2E

Week 55: RAG-Powered Market Knowledge Base (ChromaDB)

Build a vector knowledge base from 500+ financial documents for LLM context retrieval.

Documents 500+	Vector DB ChromaDB	Retrieval Top-5 chunks	Latency <500ms
-------------------	-----------------------	---------------------------	-------------------

GOALS

- > Ingest 500+ docs: 10-K/10-Q, earnings transcripts, Fed minutes, research reports
- > Chunk and embed via sentence-transformers → store in ChromaDB
- > RAG query: ResearcherAgent queries KB before scoring each symbol
- > Hybrid retrieval: semantic similarity + keyword match
- > Weekly KB refresh: new filings + transcripts added automatically
- > RAG context visible in agent reasoning trace

TASKS

- > pip install chromadb langchain-community sentence-transformers
- > doc_ingestion.py: fetch → chunk (512 tokens, 50 overlap) → embed → store
- > KnowledgeBase.query(question, top_k=5) returns relevant chunks
- > Wire into ResearcherAgent: query KB per symbol before scoring
- > Weekly cron: re-index new 10-Q filings every Sunday

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week55_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week55*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week55_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week55_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 500+ docs indexed
- > KB query returns correct chunks for AAPL
- > RAG context in agent trace

- > Weekly refresh running

AUTOPILOT LOGIC

- > RAG gives agent memory of company history across all quarters
- > Agent can ask: 'What did AAPL say about margins last earnings?'

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: ChromaDB persistence → use PersistentClient with Docker volume
- > Risk: Embedding cost → use sentence-transformers (local, free)

DAY-BY-DAY BREAKDOWN

- > Mon: ChromaDB + ingestion pipeline
- > Tue: Embed 500 docs
- > Wed: Hybrid retrieval
- > Thu: Wire into ResearcherAgent
- > Fri: Weekly refresh scheduler

Week 56: Real-Time News Arbitrage & Event-Driven Alpha

Sub-second news processing to detect market-moving events and trigger reactive scans.

Feeds 6	Latency <2s detection	Classification ML	Actions Reactive scan+emergency exit
------------	--------------------------	----------------------	--

GOALS

- > 6 real-time feeds: Bloomberg RSS, Reuters, SEC EDGAR, Twitter, Reddit, Benzinga
- > ML event classifier (FinBERT): categorize as Earnings Beat/Miss, M&A, FDA, Legal, Macro
- > Estimate price impact for each event type based on historical reactions
- > Reactive scan: positive large event → immediate candidate scan on that symbol
- > Emergency exit evaluation: negative event for held position → close immediately?
- > Latency tracking: measure detection time vs first price tick after event

TASKS

- > newsfeeds.py: async readers for 6 sources; dedup by URL hash + 5-min freshness filter
- > event_classifier.py: FinBERT fine-tuned for 8 event categories
- > price_impact_estimator.py: regression from historical event→price impact pairs
- > Reactive scan trigger: POST /api/v1/autopilot/scan?trigger=news&symbol=X
- > Emergency exit: if negative_impact > 0.15 for held position → evaluate close

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week56_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week56*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week56_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week56_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > 6 feeds ingesting real-time
- > Event correctly classified (tested)
- > Emergency exit triggers on negative event test case
- > Latency < 2s measured

AUTOPILOT LOGIC

- > News arbitrage is the alpha layer — acts in seconds vs human minutes
- > Emergency exit is safety-first: protect capital before chasing alpha

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: False positives → require 2+ source confirmation for large trades
- > Risk: Stale reposted news → 5-minute freshness filter enforced

DAY-BY-DAY BREAKDOWN

- > Mon: 6 feed readers
- > Tue: FinBERT classifier
- > Wed: Impact estimator
- > Thu: Reactive scan trigger
- > Fri: Emergency exit; latency tracking

Week 57: Options Flow Intelligence & Dark Pool Analytics

Detect institutional money movement via unusual options flow and dark pool prints.

Flow Sources 3	Unusual 5× avg OI	Dark Pool Block prints	Score 0-100
-------------------	----------------------	---------------------------	----------------

GOALS

- > Unusual options activity: flag volume > 5× average open interest per strike
- > Sweep detector: aggressive multi-strike buying at ask → strong directional bet
- > Dark pool prints: large block trades >10,000 shares before regular hours
- > FlowScore 0-100: conviction based on size, direction, proximity to expiry, sweep flag
- > Flow alignment: if flow matches proposed trade → +10 bonus to entry score
- > Flow contradiction: if strong opposite flow → skip trade regardless

TASKS

- > options_flow.py: Unusual Whales API or Tradier screener
- > unusual_activity_detector(): volume vs 30-day avg OI per strike
- > sweep_detector(): multi-strike aggressive buying pattern
- > darkpool.py: large block prints from tick data inference
- > FlowScore weighted composite
- > GET /api/v1/flow/{symbol}

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week57_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week57*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week57_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week57_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Unusual activity detected for test case (>5× OI)
- > Sweep correctly identified
- > Flow score computed for 20 symbols
- > FlowView renders events

AUTOPILOT LOGIC

- > Flow alignment: institutional agreement = higher confidence entry
- > Flow contradiction = hard skip regardless of other signals

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Unusual Whales expensive → use Tradier flow as free fallback
- > Risk: Flow data lagged 15 min → context only, not sole signal

DAY-BY-DAY BREAKDOWN

- > Mon: Flow API integration
- > Tue: Unusual activity detector
- > Wed: Sweep detector
- > Thu: Dark pool prints
- > Fri: FlowView + score integration

Week 58: Fine-Tuning LLM on Own Trade History (LoRA PEFT)

Fine-tune Llama3-8B on 500+ paper trades to create a personalized trading judgment model.

Dataset 500+ trades	Base Model Llama3-8B	Method LoRA PEFT	Eval Win rate lift
------------------------	-------------------------	---------------------	-----------------------

GOALS

- > Build instruction-tuning dataset: trade features + outcome + LLM decision rationale
- > Format as {instruction, input, output} pairs for supervised fine-tuning
- > Fine-tune Llama3-8B with LoRA adapters (r=16) on Hugging Face PEFT
- > Evaluate fine-tuned vs base Groq on held-out 20% test trades
- > Deploy as 3rd LLM layer: Groq + Gemini + Fine-tuned = 3-judge consensus
- > Monthly retraining as more live data accumulates

TASKS

- > pip install transformers peft datasets accelerate
- > dataset_builder.py: generate instruction pairs from DB trade history
- > finetune.py: LoRA config + Trainer; evaluate on test set
- > Export adapter → load at inference alongside base model
- > fine_tuned_ranker.py: integration as 3rd judgment layer

COMMANDS & FILES

- > python3 scripts/run_week58_feature.py --dry-run
 - > python3 -m pytest tests/test_week58*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week58_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week58_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > 500-trade dataset built
- > LoRA trains without OOM
- > Fine-tuned beats base on test set

-> 3rd LLM layer integrated in pipeline

AUTOPILOT LOGIC

- > Fine-tuned model learns YOUR specific trading style
- > 3 judges: Groq + Gemini + Fine-tuned = higher confidence consensus

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: 500 trades too small → data augmentation + synthetic examples
- > Risk: Overfitting → evaluate on 20% held-out minimum

DAY-BY-DAY BREAKDOWN

- > Mon: Dataset builder
- > Tue: Instruction pair formatting
- > Wed: LoRA fine-tuning
- > Thu: Evaluation vs base
- > Fri: Deploy as 3rd LLM layer

Week 59: Regime-Adaptive RL Allocation Agent (PPO)

Train a PPO RL agent to dynamically adjust strategy allocation weights based on market regime.

Algorithm PPO	State 15 features	Action 6 weights	Reward Sharpe delta
------------------	----------------------	---------------------	------------------------

GOALS

- > RL env (gymnasium): state=15 market features, action=6 strategy allocation weights
- > Reward: Sharpe ratio improvement over prior period
- > Train PPO (Stable Baselines 3) on 5-year historical data, 1M steps
- > RL agent replaces static regime→allocation mapping from Week 19
- > OOS evaluation: RL vs static allocation on held-out 2-year period
- > Daily weight updates from RL policy; changes logged for transparency

TASKS

- > pip install stable-baselines3 gymnasium
- > TradingEnv(gymnasium.Env): state, action, reward, step() implementation
- > Historical simulation env over 5 years of daily regime data
- > PPO training: 1M steps with eval every 50k steps
- > Export trained policy → rl_allocation_policy.pkl
- > RLAllocator replaces AllocationManager; daily weight update job

COMMANDS & FILES

- > python3 scripts/run_week59_feature.py --dry-run
 - > python3 -m pytest tests/test_week59*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week59_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week59_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > RL env runs 5-year episode without error
- > PPO trains to positive reward
- > RL outperforms static allocation on OOS test
- > Daily weight updates logged

AUTOPILOT LOGIC

- > RL self-optimizes allocation without human tuning
- > Weight changes logged: 'RL reduced Iron Condor from 20% to 8% today'

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Pathological allocations → enforce min/max weight bounds per strategy
- > Risk: RL overfits → 3-year train / 2-year OOS split minimum

DAY-BY-DAY BREAKDOWN

- > Mon: TradingEnv implementation
- > Tue: PPO training
- > Wed: OOS evaluation
- > Thu: Policy deployment
- > Fri: Daily weight update + logging

Week 60: Go-Live: First Real Money Trades

Clear 10-step go-live checklist and execute first real-money trades at minimal 1-contract size.

Mode Live	Size 1 contract	Budget \$1,000	Monitor Manual daily
--------------	--------------------	-------------------	-------------------------

GOALS

- > Complete 10-item go-live checklist — every item green before flipping switch
- > Execute first real-money trade: 1 contract, max \$500 total risk
- > Monitor every cycle manually for first 2 weeks of live trading
- > Paper vs live P&L; daily comparison to measure execution slippage
- > Week 1 debrief document: surprises vs paper, lessons learned
- > Scale plan: 1 → 2 → 5 contracts over 8 weeks pending clean results

TASKS

- > Complete + document all 10 go-live checklist items
- > Switch `BROKER_MODE='live'; MAX_POSITION_CONTRACTS=1; MAX_DAILY_LOSS=200`
- > First trade: put credit spread on liquid ETF (SPY, QQQ, IWM)
- > Daily live vs paper comparison report
- > Week 1 debrief document

COMMANDS & FILES

- | | |
|---|---|
| -> <code>python3 scripts/run_week60_feature.py --dry-run</code> | -> <code>services/core.py</code> — core business logic |
| -> <code>python3 -m pytest tests/test_week60*.py -v</code> | -> <code>services/router.py</code> — FastAPI router for this feature |
| -> <code>python3 scripts/run_autopilot_dry_run.py</code> | -> <code>tests/test_.py</code> — unit + integration tests |
| -> <code>make lint</code> && <code>make test</code> | -> <code>scripts/run_week60_feature.py</code> — CLI to exercise the feature |
| | -> <code>frontend/src/pages/Page.tsx</code> — UI component |
| | -> <code>docs/week60_design.md</code> — design decisions + API contract |

API ENDPOINTS TO BUILD

- > `GET /api/v1/` — list or status endpoint
- > `POST /api/v1/` — create or trigger endpoint
- > `GET /api/v1/{id}` — detail endpoint
- > `DELETE /api/v1/{id}` — delete or cancel endpoint

TESTS TO WRITE

- > `test_happy_path.py`: nominal flow works end to end
- > `test_edge_cases.py`: empty input, null values, boundary conditions
- > `test_error_handling.py`: invalid input returns 400 with clear message
- > `test_integration.py`: feature integrates with autopilot pipeline
- > `playwright/test_ui.py`: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|--|--|
| -> <code>FEATURE_ENABLED=true</code> | -> Core: fastapi, sqlalchemy, pydantic |
| -> <code>FEATURE_TIMEOUT_SECONDS=30</code> | -> Data: pandas, numpy |
| -> <code>FEATURE_MAX_RETRIES=3</code> | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Checklist 100% green
- > First live trade executed + confirmed filled
- > Daily comparison report
- > Week 1 debrief written

AUTOPILOT LOGIC

- > Live trading is the 60-week culmination — the autopilot proven at real stakes
- > 1 contract = minimal risk; learning about live execution quality is the goal

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Bad fill on first trade → review order type (limit at mid price only)
- > Risk: Panic on first loss → commit to full 10-trade minimum before any config change

DAY-BY-DAY BREAKDOWN

- > Mon: Go-live checklist completion; switch to live
- > Tue: First live trade; monitor fill
- > Wed-Thu: 2nd and 3rd trades; slippage capture
- > Fri: Week 1 debrief document

Week 61: Scale-Up to 5 Contracts & Kelly Recalibration

Increase live position size to 5 contracts using live-calibrated Kelly after 2 clean weeks.

Max 5 contracts	Kelly Live-data	Daily Loss \$500	Slippage Tracked
--------------------	--------------------	---------------------	---------------------

GOALS

- > Scale to 5 contracts after 2 clean weeks of live trading
- > Recalibrate Kelly parameters using live trade win rate and P&L; data
- > Update daily loss limit: \$200 → \$500 proportionally
- > Track slippage: live fill vs theoretical mid price per trade
- > Volume impact: verify 5-contract orders don't worsen fill quality significantly

TASKS

- > Update config: MAX_POSITION_CONTRACTS=5, MAX_DAILY_LOSS=500
- > Kelly recalibration: live_trades = db.get_live_trades(last_n=50)
- > slippage_tracker.py: record expected_mid vs actual_fill per trade
- > Volume analysis: compare fill quality as contract count increases
- > Updated monitoring thresholds for 5-contract risk

COMMANDS & FILES

- > python3 scripts/run_week61_feature.py --dry-run
 - > python3 -m pytest tests/test_week61*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week61_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week61_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > 5-contract live trades executing cleanly
- > Kelly recalibrated on live data
- > Slippage per trade computed

AUTOPILOT LOGIC

- > Scale-up gated on clean execution — no incidents in first 2 weeks
- > Live Kelly recalibration makes sizing data-driven from actual fills

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Worse fills with 5 contracts → test 2, then 3, then 5 progressively
- > Risk: Overconfidence → keep daily loss limit strict; honor the kill switch

DAY-BY-DAY BREAKDOWN

- > Mon: Config update; first 5-contract trade
- > Tue: Kelly recalibration
- > Wed: Slippage tracking
- > Thu: Volume + fill quality analysis
- > Fri: Updated monitoring

Week 62: Multi-Broker Integration & Automatic Failover

Add TD Ameritrade/Schwab as second broker for redundancy and best-execution routing.

Brokers 2	Failover Automatic	Compare Commission + fill quality	SLA Zero outage
--------------	-----------------------	--------------------------------------	--------------------

GOALS

- > Integrate Schwab/TD Ameritrade API via schwab-py library
- > BrokerRouter: selects broker based on health + estimated fill quality
- > Automatic failover: if primary broker fails → auto-route to secondary in <5s
- > Commission tracker: per-trade commission by broker for cost comparison
- > Dual health check pre-market: both brokers must respond before cycle starts

TASKS

- > pip install schwab-py
- > schwab_broker.py implementing BrokerInterface
- > BrokerRouter.execute(): primary healthy → primary; else secondary
- > commission_tracker.py: log commission per trade per broker
- > Failover test: simulate primary outage → confirm secondary activates

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week62_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week62*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week62_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week62_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Schwab connected + placing paper orders
- > Failover test passes in <5s
- > Commission comparison report generated

AUTOPILOT LOGIC

- > Dual broker = zero single-point-of-failure in execution
- > Best-execution routing shifts to lower-cost broker over time

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Schwab API changes → use well-maintained schwab-py; monitor for deprecations

DAY-BY-DAY BREAKDOWN

- > Mon: Schwab API auth
- > Tue: schwab_broker.py
- > Wed: BrokerRouter failover
- > Thu: Commission tracker
- > Fri: Failover E2E test

Week 63: Futures & Index Hedging Integration

Add E-mini S&P; 500 futures and VIX derivatives for portfolio delta-neutral hedging.

Instruments ES, NQ, VIX	Hedge Delta-neutral	Auto Every 30 min	Cost Tracked separately
----------------------------	------------------------	----------------------	----------------------------

GOALS

- > E-mini ES futures via Interactive Brokers `ib_insync` for portfolio delta hedging
- > Hedge trigger: if portfolio delta drifts beyond ± 0.25 , neutralize with ES
- > VIX call hedge: buy cheap OTM VIX calls (insurance) when $VIX < 18$
- > Hedge sizing: exact contract count from portfolio delta / ES delta per contract
- > Auto-hedge job: check delta balance every 30 minutes intraday
- > Hedge P&L; tracked separately from options P&L;

TASKS

- > pip install `ib_insync`
- > `ib_broker.py`: ES mini execution on IB paper
- > `delta_hedger.py`: portfolio_delta → ES contracts needed
- > `vix_hedge.py`: buy OTM VIX calls when $VIX < 18$
- > Auto-hedge 30-min job; `HedgeView.tsx`

COMMANDS & FILES

- | | |
|---|---|
| -> <code>python3 scripts/run_week63_feature.py --dry-run</code> | -> <code>services/core.py</code> — core business logic |
| -> <code>python3 -m pytest tests/test_week63*.py -v</code> | -> <code>services/router.py</code> — FastAPI router for this feature |
| -> <code>python3 scripts/run_autopilot_dry_run.py</code> | -> <code>tests/test_.py</code> — unit + integration tests |
| -> <code>make lint</code> && <code>make test</code> | -> <code>scripts/run_week63_feature.py</code> — CLI to exercise the feature |
| | -> <code>frontend/src/pages/Page.tsx</code> — UI component |
| | -> <code>docs/week63_design.md</code> — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET `/api/v1/` — list or status endpoint
- > POST `/api/v1/` — create or trigger endpoint
- > GET `/api/v1/{id}` — detail endpoint
- > DELETE `/api/v1/{id}` — delete or cancel endpoint

TESTS TO WRITE

- > `test_happy_path.py`: nominal flow works end to end
- > `test_edge_cases.py`: empty input, null values, boundary conditions
- > `test_error_handling.py`: invalid input returns 400 with clear message
- > `test_integration.py`: feature integrates with autopilot pipeline
- > `playwright/test_ui.py`: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|--|---|
| -> <code>FEATURE_ENABLED=true</code> | -> Core: <code>fastapi</code> , <code>sqlalchemy</code> , <code>pydantic</code> |
| -> <code>FEATURE_TIMEOUT_SECONDS=30</code> | -> Data: <code>pandas</code> , <code>numpy</code> |
| -> <code>FEATURE_MAX_RETRIES=3</code> | -> Testing: <code>pytest</code> , <code>httpx</code> , <code>playwright</code> |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > ES mini orders on IB paper account
- > Delta hedge math verified (tested)
- > Auto-hedge fires every 30 minutes
- > `HedgeView` shows hedge positions + P&L;

AUTOPILOT LOGIC

- > Delta-neutral hedge removes market direction risk from portfolio
- > VIX insurance: affordable in calm markets, pays off in crashes

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Margin requirements → verify account margin before each hedge
- > Risk: Over-hedging (negative delta) → strict target ± 0.10 delta range

DAY-BY-DAY BREAKDOWN

- > Mon: IB API + ES paper trading
- > Tue: Delta hedger
- > Wed: VIX hedge logic
- > Thu: 30-min auto-hedge job
- > Fri: HedgeView + P&L; tracking

Week 64: Portfolio Rebalancing Engine & Capital Efficiency

Automate weekly rebalancing to maintain target allocation and maximize capital deployment.

Rebalance Weekly	Idle Capital <10%	Margin 50-70%	Turnover Minimize
------------------	-------------------	---------------	-------------------

GOALS

- > Weekly rebalance: assess allocation drift; generate rebalance orders if drift >5%
- > Capital efficiency: no more than 10% idle — deploy or sweep to T-bills
- > Margin utilization: 50-70% target — not too conservative, not too leveraged
- > Minimize turnover: prefer rolling over close-and-reopen
- > RebalanceView: current vs target allocation with action plan

TASKS

- > rebalancer.py: drift computation; rebalance order generation
- > capital_efficiency(): deployed% + idle recommendation
- > margin_monitor(): alert at 70% margin utilization
- > cash_sweep_manager(): broker cash management API
- > RebalanceView.tsx: side-by-side current vs target

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week64_feature.py --dry-run-> python3 -m pytest tests/test_week64*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week64_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week64_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > Rebalancer produces correct action for test drift
- > Capital efficiency metric computed
- > Margin alert at 70%
- > RebalanceView shows weekly plan

AUTOPILOT LOGIC

- > Rebalancing maintains intended risk profile over time

-> Idle capital detection removes drag from uninvested cash

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Rebalance too frequent → only execute if drift >5%
- > Risk: Cash sweep may not be available → integrate and verify with broker

DAY-BY-DAY BREAKDOWN

- > Mon: Rebalancer core
- > Tue: Capital efficiency + margin monitor
- > Wed: Cash sweep
- > Thu: RebalanceView.tsx
- > Fri: Weekly scheduler + E2E

Week 65: Institutional Reporting & FINRA Compliance

Build 13F-style reports, daily reconciliation, FINRA-ready audit trail, 7-year retention.

Status Y2 Mid-Phase	Priority High	Type Production	Detail 13F position report, daily rec
------------------------	------------------	--------------------	--

GOALS

- > 13F position report, daily reconciliation, trade confirms, 7-year tamper-proof archive.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Institutional Reporting & FINRA Compliance architecture
- > Implement core Institutional Reporting & FINRA Compliance service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week65_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week65*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week65_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week65_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Institutional Reporting & FINRA Compliance working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Institutional Reporting & FINRA Compliance enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 66: Tax Lot Accounting & Wash Sale Compliance

Track cost basis, holding periods, and wash sale rules automatically for tax reporting.

Status Y2 Mid-Phase	Priority High	Type Production	Detail FIFO/LIFO lot tracking, wash s
------------------------	------------------	--------------------	---

GOALS

- > FIFO/LIFO lot tracking, wash sale flag, annual tax report PDF, tax optimization hints.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Tax Lot Accounting & Wash Sale Compliance architecture
- > Implement core Tax Lot Accounting & Wash Sale Compliance service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week66_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week66*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week66_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week66_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Tax Lot Accounting & Wash Sale Compliance working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Tax Lot Accounting & Wash Sale Compliance enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 67: Crypto Correlation Signal Integration

Monitor BTC/ETH as risk-off signals affecting equity options strategy allocation.

Status Y2 Mid-Phase	Priority High	Type Production	Detail BTC correlation to SPY, crypto
------------------------	------------------	--------------------	---

GOALS

- > BTC correlation to SPY, crypto fear/greed index, regime modifier when crypto diverges.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Crypto Correlation Signal Integration architecture
- > Implement core Crypto Correlation Signal Integration service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week67_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week67*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week67_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week67_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Crypto Correlation Signal Integration working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Crypto Correlation Signal Integration enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 68: Macro Economic Indicator Dashboard

Build macro dashboard: CPI, NFP, PMI, Fed signals integrated into regime detection.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Economic calendar pull (FRED A)
------------------------	------------------	--------------------	--

GOALS

- > Economic calendar pull (FRED API), 12 macro indicators, regime overlay adjustments.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Macro Economic Indicator Dashboard architecture
- > Implement core Macro Economic Indicator Dashboard service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week68_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week68*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week68_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week68_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Macro Economic Indicator Dashboard working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Macro Economic Indicator Dashboard enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 69: IV Skew Trading Strategies

Build dedicated IV skew arbitrage: buy cheap wing, sell expensive ATM or vice versa.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Skew index per symbol, skew tr
------------------------	------------------	--------------------	---

GOALS

- > Skew index per symbol, skew trade builder, backtest on 2 years of vol surface data.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design IV Skew Trading Strategies architecture
- > Implement core IV Skew Trading Strategies service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week69_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week69*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week69_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week69_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > IV Skew Trading Strategies working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > IV Skew Trading Strategies enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 70: Live Earnings Play Strategies

Trade IV crush around earnings: structured calendar positions 5 days before, close day-of.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Earnings play scanner, pre-ear
------------------------	------------------	--------------------	---

GOALS

- > Earnings play scanner, pre-earnings entry, IV crush capture, post-earnings analysis.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Live Earnings Play Strategies architecture
- > Implement core Live Earnings Play Strategies service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week70_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week70*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week70_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week70_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Live Earnings Play Strategies working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Live Earnings Play Strategies enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 71: Portfolio Insurance & Tail Risk Hedging

Systematic tail hedging: OTM puts, VIX calls, correlation trades for crash protection.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Tail hedge budget 1% of NAV/mo
------------------------	------------------	--------------------	--

GOALS

- > Tail hedge budget 1% of NAV/month, auto-buy SPY puts in calm markets, track hedge drag.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Portfolio Insurance & Tail Risk Hedging architecture
- > Implement core Portfolio Insurance & Tail Risk Hedging service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week71_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week71*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week71_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week71_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Portfolio Insurance & Tail Risk Hedging working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Portfolio Insurance & Tail Risk Hedging enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 72: AI Agent Self-Evaluation & Improvement Loop

Agent evaluates its own weekly performance and proposes parameter improvements.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Weekly self-eval prompt, agent
------------------------	------------------	--------------------	--

GOALS

- > Weekly self-eval prompt, agent proposes changes, human approval gate, auto A/B test.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design AI Agent Self-Evaluation & Improvement Loop architecture
- > Implement core AI Agent Self-Evaluation & Improvement Loop service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week72_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week72*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week72_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week72_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > AI Agent Self-Evaluation & Improvement Loop working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > AI Agent Self-Evaluation & Improvement Loop enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 73: Multi-Timeframe Regime Analysis

Align short/medium/long-term regime signals for higher-conviction entries only.

Status Y2 Mid-Phase	Priority High	Type Production	Detail 3-timeframe regime (daily, wee
------------------------	------------------	--------------------	---

GOALS

- > 3-timeframe regime (daily, weekly, monthly), alignment score, skip on regime conflict.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Multi-Timeframe Regime Analysis architecture
- > Implement core Multi-Timeframe Regime Analysis service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week73_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week73*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week73_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week73_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Multi-Timeframe Regime Analysis working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Multi-Timeframe Regime Analysis enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 74: Client Reporting API & Partner Integration

RESTful API for third-party portfolio managers to access Apex Terminal performance data.

Status Y2 Mid-Phase	Priority High	Type Production	Detail OAuth2 API keys, GET /partner/
------------------------	------------------	--------------------	---

GOALS

- > OAuth2 API keys, GET /partner/performance, rate-limited, white-label report endpoint.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Client Reporting API & Partner Integration architecture
- > Implement core Client Reporting API & Partner Integration service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week74_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week74*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week74_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week74_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Client Reporting API & Partner Integration working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Client Reporting API & Partner Integration enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 75: Advanced Backtesting: Stress Period Deep-Dives

Deep-dive backtest in 5 stress periods (2008, 2011, 2018, 2020, 2022) with tick data.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Tick-level backtest engine, 5
------------------------	------------------	--------------------	---

GOALS

- > Tick-level backtest engine, 5 stress periods, regime-stratified Sharpe, tail loss audit.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Advanced Backtesting: Stress Period Deep-Dives architecture
- > Implement core Advanced Backtesting: Stress Period Deep-Dives service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week75_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week75*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week75_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week75_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Advanced Backtesting: Stress Period Deep-Dives working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Advanced Backtesting: Stress Period Deep-Dives enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 76: Volatility Forecasting with GARCH Models

Train GARCH(1,1) to forecast next-day IV; use forecast in entry scoring and position sizing.

Status Y2 Mid-Phase	Priority High	Type Production	Detail arch library GARCH(1,1), forec
------------------------	------------------	--------------------	---

GOALS

- > arch library GARCH(1,1), forecast vs realized calibration, IV forecast in entry scorer.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Volatility Forecasting with GARCH Models architecture
- > Implement core Volatility Forecasting with GARCH Models service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|---|---|
| <ul style="list-style-type: none"> -> python3 scripts/run_week76_feature.py --dry-run -> python3 -m pytest tests/test_week76*.py -v -> python3 scripts/run_autopilot_dry_run.py -> make lint && make test | <ul style="list-style-type: none"> -> services/core.py — core business logic -> services/router.py — FastAPI router for this feature -> tests/test_.py — unit + integration tests -> scripts/run_week76_feature.py — CLI to exercise the feature -> frontend/src/pages/Page.tsx — UI component -> docs/week76_design.md — design decisions + API contract |
|---|---|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|---|
| <ul style="list-style-type: none"> -> FEATURE_ENABLED=true -> FEATURE_TIMEOUT_SECONDS=30 -> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none"> -> Core: fastapi, sqlalchemy, pydantic -> Data: pandas, numpy -> Testing: pytest, httpx, playwright -> Feature-specific: see week title for specialized packages |
|---|---|

DELIVERABLES

- > Volatility Forecasting with GARCH Models working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Volatility Forecasting with GARCH Models enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 77: Options Market Making Simulation

Simulate basic market making: quote bid and ask; earn spread as alternative revenue stream.

Status Y2 Mid-Phase	Priority High	Type Production	Detail MM simulation engine, BBO-awar
------------------------	------------------	--------------------	---

GOALS

- > MM simulation engine, BBO-aware quoting, inventory hedging, simulated P&L; from spread capture.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Options Market Making Simulation architecture
- > Implement core Options Market Making Simulation service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week77_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week77*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week77_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week77_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Options Market Making Simulation working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Options Market Making Simulation enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 78: Q3 Performance Review & Y2H2 Planning

Mid-year live trading review and planning for the final 26 weeks of Year 2.

Status Y2 Mid-Phase	Priority High	Type Production	Detail Q3 attribution, live vs paper
------------------------	------------------	--------------------	---

GOALS

- > Q3 attribution, live vs paper comparison, top-3 strategy winners, Y2H2 roadmap approved.
- > Integrate with existing autopilot pipeline
- > Add comprehensive tests
- > Document in user guide

TASKS

- > Design Q3 Performance Review & Y2H2 Planning architecture
- > Implement core Q3 Performance Review & Y2H2 Planning service
- > Build API and frontend component
- > Write tests + CI integration
- > Document in user guide and runbook

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week78_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week78*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week78_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week78_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Q3 Performance Review & Y2H2 Planning working end-to-end
- > Tests passing in CI
- > Feature documented in user guide

AUTOPILOT LOGIC

- > Q3 Performance Review & Y2H2 Planning enhances autopilot decision quality and operational robustness
- > Results logged in RunArtifact and weekly performance reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Integration complexity → standalone service first, integrate after unit tests pass

DAY-BY-DAY BREAKDOWN

- > Mon: Design + architecture
- > Tue: Core implementation
- > Wed: API endpoints
- > Thu: Frontend component
- > Fri: Tests + documentation

Week 79: Mobile App: React Native Dashboard

iOS and Android app with real-time P&L;, push notifications, and biometric kill switch.

Platform iOS+Android	Push FCM+APNs	Auth Biometric	Size <30MB
-------------------------	------------------	-------------------	---------------

GOALS

- > React Native app: real-time P&L;, positions, alerts feed, kill switch
- > Push every trade execution, alert trigger, daily briefing
- > Biometric auth (FaceID/TouchID) for kill switch activation
- > Offline mode: cached last-known positions when no connection
- > Dark terminal theme matching desktop

TASKS

- > npx react-native init ApexMobile
- > JWT auth integration with FastAPI backend
- > WebSocket client for live P&L; streaming
- > FCM + APNs push setup
- > Biometric kill switch
- > TestFlight + Google Play internal beta

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week79_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week79*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week79_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week79_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > App builds for iOS + Android
- > P&L; updates via WebSocket
- > Push fires on test trade
- > Kill switch works with biometric

AUTOPILOT LOGIC

- > Mobile = always-on monitoring even away from desk
- > Biometric kill switch is the ultimate safety feature

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: WebSocket battery drain → background polling when app inactive
- > Risk: App store review delay → TestFlight for internal testing first

DAY-BY-DAY BREAKDOWN

- > Mon: RN setup; JWT auth
- > Tue: P&L; + positions screens
- > Wed: WebSocket
- > Thu: Push notifications
- > Fri: Biometric kill switch; TestFlight

Week 80: Web3 Integration & On-Chain Trade Audit Trail

Publish trade decision hashes to Polygon blockchain for tamper-proof immutable audit.

Chain Polygon	Cost <\$0.01/hash	Verify On-chain	DeFi Research phase
------------------	----------------------	--------------------	------------------------

GOALS

- > Publish SHA-256 hash of each trade decision to Polygon for immutable record
- > Verification API: compare DB record vs on-chain hash
- > Quarterly portfolio attestation published on-chain
- > Research Lyra DeFi options protocol on testnet
- > Batch hourly hashes to control gas costs

TASKS

- > pip install web3
- > blockchain.py: publish_hash(trade_id, hash) → tx_hash
- > Polygon Mumbai testnet → mainnet after testing
- > verify_trade(trade_id): compare DB vs on-chain
- > GET /api/v1/compliance/chain-verify/{trade_id}

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week80_feature.py --dry-run-> python3 -m pytest tests/test_week80*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week80_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week80_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > Hash published on Polygon for test trade
- > Verification endpoint works
- > Gas tracked at <\$0.01 per hash batch
- > Lyra research doc written

AUTOPILOT LOGIC

- > On-chain hashes = blockchain is the ultimate immutable audit

-> Quarterly attestation makes performance claims publicly verifiable

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Gas spikes → batch hourly rather than per-trade
- > Risk: Lyra liquidity thin → DeFi research only; no production trading yet

DAY-BY-DAY BREAKDOWN

- > Mon: Web3 + Polygon
- > Tue: Hash publisher
- > Wed: Verification API
- > Thu: Lyra testnet research
- > Fri: Quarterly attestation

Week 81: Quantitative Research Platform & Jupyter Integration

Advanced Year 2 strategic feature: *Quantitative Research Platform & Jupyter Integration*

Week 81/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Quantitative Research Platform & Jupyter Integration as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Quantitative Research Platform & Jupyter Integration design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week81_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week81*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week81_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week81_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Quantitative Research Platform & Jupyter Integration live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Quantitative Research Platform & Jupyter Integration adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 82: API Marketplace: Sell Data Feeds & Signals to Third Parties

Advanced Year 2 strategic feature: API Marketplace: Sell Data Feeds & Signals to Third Parties

Week 82/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement API Marketplace: Sell Data Feeds & Signals to Third Parties as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect API Marketplace: Sell Data Feeds & Signals to Third Parties design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week82_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week82*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week82_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week82_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > API Marketplace: Sell Data Feeds & Signals to Third Parties live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > API Marketplace: Sell Data Feeds & Signals to Third Parties adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 83: Enterprise Multi-Tenant Architecture for Multiple Users

Advanced Year 2 strategic feature: Enterprise Multi-Tenant Architecture for Multiple Users

Week 83/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Enterprise Multi-Tenant Architecture for Multiple Users as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Enterprise Multi-Tenant Architecture for Multiple Users design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week83_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week83*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week83_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week83_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Enterprise Multi-Tenant Architecture for Multiple Users live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Enterprise Multi-Tenant Architecture for Multiple Users adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 84: Black-Litterman Portfolio Optimization Model

Advanced Year 2 strategic feature: Black-Litterman Portfolio Optimization Model

Week 84/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Black-Litterman Portfolio Optimization Model as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Black-Litterman Portfolio Optimization Model design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week84_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week84*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week84_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week84_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Black-Litterman Portfolio Optimization Model live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Black-Litterman Portfolio Optimization Model adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 85: Regulatory Filing Automation (Form 4, 13F Preparation)

Advanced Year 2 strategic feature: Regulatory Filing Automation (Form 4, 13F Preparation)

Week 85/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Regulatory Filing Automation (Form 4, 13F Preparation) as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Regulatory Filing Automation (Form 4, 13F Preparation) design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week85_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week85*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week85_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week85_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Regulatory Filing Automation (Form 4, 13F Preparation) live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Regulatory Filing Automation (Form 4, 13F Preparation) adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 86: AI-Powered Strategy Narrative Auto-Generator

Advanced Year 2 strategic feature: AI-Powered Strategy Narrative Auto-Generator

Week 86/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement AI-Powered Strategy Narrative Auto-Generator as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect AI-Powered Strategy Narrative Auto-Generator design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week86_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week86*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week86_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week86_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > AI-Powered Strategy Narrative Auto-Generator live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > AI-Powered Strategy Narrative Auto-Generator adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 87: Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Forex)

Advanced Year 2 strategic feature: Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Fo

Week 87/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Forex) as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Forex) design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week87_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week87*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week87_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week87_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Forex) live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Cross-Asset Correlation Engine (Stocks + Bonds + Crypto + Forex) adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 88: Distributed Computing: Ray Cluster for Parallel Backtests

Advanced Year 2 strategic feature: Distributed Computing: Ray Cluster for Parallel Backtests

Week 88/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Distributed Computing: Ray Cluster for Parallel Backtests as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Distributed Computing: Ray Cluster for Parallel Backtests design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- > python3 scripts/run_week88_feature.py --dry-run
 - > python3 -m pytest tests/test_week88*.py -v
 - > python3 scripts/run_autopilot_dry_run.py
 - > make lint && make test
- > services/core.py — core business logic
 - > services/router.py — FastAPI router for this feature
 - > tests/test_.py — unit + integration tests
 - > scripts/run_week88_feature.py — CLI to exercise the feature
 - > frontend/src/pages/Page.tsx — UI component
 - > docs/week88_design.md — design decisions + API contract

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- > FEATURE_ENABLED=true
 - > FEATURE_TIMEOUT_SECONDS=30
 - > FEATURE_MAX_RETRIES=3
- > Core: fastapi, sqlalchemy, pydantic
 - > Data: pandas, numpy
 - > Testing: pytest, httpx, playwright
 - > Feature-specific: see week title for specialized packages

DELIVERABLES

- > Distributed Computing: Ray Cluster for Parallel Backtests live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Distributed Computing: Ray Cluster for Parallel Backtests adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 89: Automated A/B Testing Framework for Strategy Variants

Advanced Year 2 strategic feature: Automated A/B Testing Framework for Strategy Variants

Week 89/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Automated A/B Testing Framework for Strategy Variants as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Automated A/B Testing Framework for Strategy Variants design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week89_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week89*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week89_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week89_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Automated A/B Testing Framework for Strategy Variants live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Automated A/B Testing Framework for Strategy Variants adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 90: Production Hardening: Kubernetes, Helm Charts, Auto-Scaling

Advanced Year 2 strategic feature: Production Hardening: Kubernetes, Helm Charts, Auto-Scaling

Week 90/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Production Hardening: Kubernetes, Helm Charts, Auto-Scaling as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Production Hardening: Kubernetes, Helm Charts, Auto-Scaling design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week90_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week90*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week90_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week90_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Production Hardening: Kubernetes, Helm Charts, Auto-Scaling live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Production Hardening: Kubernetes, Helm Charts, Auto-Scaling adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 91: Community Platform: Strategy Sharing Marketplace

Advanced Year 2 strategic feature: Community Platform: Strategy Sharing Marketplace

Week 91/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Community Platform: Strategy Sharing Marketplace as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Community Platform: Strategy Sharing Marketplace design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week91_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week91*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week91_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week91_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Community Platform: Strategy Sharing Marketplace live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Community Platform: Strategy Sharing Marketplace adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 92: Beta Program: External User Onboarding (10 Beta Users)

Advanced Year 2 strategic feature: Beta Program: External User Onboarding (10 Beta Users)

Week 92/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Beta Program: External User Onboarding (10 Beta Users) as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Beta Program: External User Onboarding (10 Beta Users) design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week92_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week92*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week92_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week92_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Beta Program: External User Onboarding (10 Beta Users) live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Beta Program: External User Onboarding (10 Beta Users) adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 93: Public API v2.0 with GraphQL & WebSocket Subscriptions

Advanced Year 2 strategic feature: Public API v2.0 with GraphQL & WebSocket Subscriptions

Week 93/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Public API v2.0 with GraphQL & WebSocket Subscriptions as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Public API v2.0 with GraphQL & WebSocket Subscriptions design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week93_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week93*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week93_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week93_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Public API v2.0 with GraphQL & WebSocket Subscriptions live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Public API v2.0 with GraphQL & WebSocket Subscriptions adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 94: Advanced Options Analytics: Smile Arbitrage & Density Extraction

Advanced Year 2 strategic feature: Advanced Options Analytics: Smile Arbitrage & Density Extrac

Week 94/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Advanced Options Analytics: Smile Arbitrage & Density Extraction as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Advanced Options Analytics: Smile Arbitrage & Density Extraction design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week94_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week94*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week94_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week94_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Advanced Options Analytics: Smile Arbitrage & Density Extraction live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Advanced Options Analytics: Smile Arbitrage & Density Extraction adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 95: Institutional Data Integration: Bloomberg B-PIPE Alternative

Advanced Year 2 strategic feature: Institutional Data Integration: Bloomberg B-PIPE Alternative

Week 95/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Institutional Data Integration: Bloomberg B-PIPE Alternative as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Institutional Data Integration: Bloomberg B-PIPE Alternative design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week95_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week95*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week95_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week95_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Institutional Data Integration: Bloomberg B-PIPE Alternative live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Institutional Data Integration: Bloomberg B-PIPE Alternative adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 96: Automated Earnings Straddle & Strangle Strategy Suite

Advanced Year 2 strategic feature: Automated Earnings Straddle & Strangle Strategy Suite

Week 96/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Automated Earnings Straddle & Strangle Strategy Suite as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Automated Earnings Straddle & Strangle Strategy Suite design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week96_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week96*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week96_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week96_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Automated Earnings Straddle & Strangle Strategy Suite live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Automated Earnings Straddle & Strangle Strategy Suite adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 97: Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments

Advanced Year 2 strategic feature: Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments

Week 97/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week97_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week97*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week97_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week97_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Full CI/CD Pipeline: GitOps, ArgoCD, Blue-Green Deployments adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 98: Performance Attribution Engine: Brinson-Hood-Beebower Model

Advanced Year 2 strategic feature: Performance Attribution Engine: Brinson-Hood-Beebower Model

Week 98/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Performance Attribution Engine: Brinson-Hood-Beebower Model as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Performance Attribution Engine: Brinson-Hood-Beebower Model design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week98_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week98*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week98_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week98_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Performance Attribution Engine: Brinson-Hood-Beebower Model live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Performance Attribution Engine: Brinson-Hood-Beebower Model adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 99: Predictive Analytics Dashboard: ML-Powered Forward Metrics

Advanced Year 2 strategic feature: Predictive Analytics Dashboard: ML-Powered Forward Metrics

Week 99/104	Phase Y2 Final	Status Production	Priority Strategic
----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Predictive Analytics Dashboard: ML-Powered Forward Metrics as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Predictive Analytics Dashboard: ML-Powered Forward Metrics design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|--|--|
| -> python3 scripts/run_week99_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week99*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week99_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week99_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Predictive Analytics Dashboard: ML-Powered Forward Metrics live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Predictive Analytics Dashboard: ML-Powered Forward Metrics adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 100: Two-Year Milestone: 10,000 Paper Trades Analysis & Insights

Advanced Year 2 strategic feature: Two-Year Milestone: 10,000 Paper Trades Analysis & Insights

Week 100/104	Phase Y2 Final	Status Production	Priority Strategic
-----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Two-Year Milestone: 10,000 Paper Trades Analysis & Insights as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Two-Year Milestone: 10,000 Paper Trades Analysis & Insights design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|---|---|
| -> python3 scripts/run_week100_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week100*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week100_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week100_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Two-Year Milestone: 10,000 Paper Trades Analysis & Insights live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Two-Year Milestone: 10,000 Paper Trades Analysis & Insights adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 101: Open Source Launch: Apache 2.0 License + GitHub Stars Campaign

Advanced Year 2 strategic feature: Open Source Launch: Apache 2.0 License + GitHub Stars Campai

Week 101/104	Phase Y2 Final	Status Production	Priority Strategic
-----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Open Source Launch: Apache 2.0 License + GitHub Stars Campaign as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Open Source Launch: Apache 2.0 License + GitHub Stars Campaign design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|---|---|
| -> python3 scripts/run_week101_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week101*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week101_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week101_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Open Source Launch: Apache 2.0 License + GitHub Stars Campaign live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Open Source Launch: Apache 2.0 License + GitHub Stars Campaign adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 102: Premium Tier: SaaS Model Design & Stripe Billing Integration

Advanced Year 2 strategic feature: Premium Tier: SaaS Model Design & Stripe Billing Integration

Week 102/104	Phase Y2 Final	Status Production	Priority Strategic
-----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Premium Tier: SaaS Model Design & Stripe Billing Integration as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Premium Tier: SaaS Model Design & Stripe Billing Integration design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|---|---|
| -> python3 scripts/run_week102_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week102*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week102_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week102_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Premium Tier: SaaS Model Design & Stripe Billing Integration live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Premium Tier: SaaS Model Design & Stripe Billing Integration adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 103: Final Integration Testing & Two-Year Performance Certification

Advanced Year 2 strategic feature: Final Integration Testing & Two-Year Performance Certification

Week 103/104	Phase Y2 Final	Status Production	Priority Strategic
-----------------	-------------------	----------------------	-----------------------

GOALS

- > Implement Final Integration Testing & Two-Year Performance Certification as production-grade capability
- > Integrate with core autopilot ecosystem
- > Test in staging environment before production rollout
- > Document and release with comprehensive changelog

TASKS

- > Architect Final Integration Testing & Two-Year Performance Certification design
- > Implement core service + API
- > Build frontend component
- > Write unit + integration tests
- > Documentation + CI integration

COMMANDS & FILES

- | | |
|---|---|
| -> python3 scripts/run_week103_feature.py --dry-run | -> services/core.py — core business logic |
| -> python3 -m pytest tests/test_week103*.py -v | -> services/router.py — FastAPI router for this feature |
| -> python3 scripts/run_autopilot_dry_run.py | -> tests/test_.py — unit + integration tests |
| -> make lint && make test | -> scripts/run_week103_feature.py — CLI to exercise the feature |
| | -> frontend/src/pages/Page.tsx — UI component |
| | -> docs/week103_design.md — design decisions + API contract |

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|-------------------------------|--|
| -> FEATURE_ENABLED=true | -> Core: fastapi, sqlalchemy, pydantic |
| -> FEATURE_TIMEOUT_SECONDS=30 | -> Data: pandas, numpy |
| -> FEATURE_MAX_RETRIES=3 | -> Testing: pytest, httpx, playwright |
| | -> Feature-specific: see week title for specialized packages |

DELIVERABLES

- > Final Integration Testing & Two-Year Performance Certification live in production
- > Tests passing in CI
- > User documentation updated
- > Released with changelog

AUTOPILOT LOGIC

- > Final Integration Testing & Two-Year Performance Certification adds strategic depth to the autonomous trading ecosystem
- > Performance impact tracked in weekly attribution reports

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: Scope underestimated → timebox to 1 week; defer excess to post-v3.0 roadmap

DAY-BY-DAY BREAKDOWN

- > Mon: Design + kickoff
- > Tue: Core implementation
- > Wed: API + frontend
- > Thu: Testing + integration
- > Fri: Documentation + deployment

Week 104: Two-Year Completion: Open Source Launch, 4-Year Roadmap & Celebration

Final week: 2-year retrospective, open source the core, publish the definitive technical article, plan Year 3+.

Live Trades 2,000+ target	Sharpe Target >1.5	Open Source GitHub	Release v3.0
------------------------------	-----------------------	-----------------------	-----------------

GOALS

- > Comprehensive 2-year performance attribution: all strategies, all regimes
- > Calculate full live trading P&L; target Sharpe >1.5, max drawdown <15%
- > Open-source the core framework (minus proprietary alpha signals) on GitHub
- > Publish definitive technical article: 'Building an Autonomous Options Autopilot in 2 Years'
- > Year 3-4 roadmap: hedge fund structure, institutional capital, full platform
- > Tag v3.0; celebrate the two-year milestone

TASKS

- > two_year_review.py: full performance attribution from DB
- > 2-year report PDF via ReportEngine
- > Open-source prep: audit private signals; write contribution guide + LICENSE
- > Publish technical article on Substack / HN / LinkedIn
- > Year 3-4 Roadmap.md: investor-deck format with track record
- > git tag v3.0 with full annotated changelog

COMMANDS & FILES

- | | |
|--|--|
| <ul style="list-style-type: none">-> python3 scripts/run_week104_feature.py --dry-run-> python3 -m pytest tests/test_week104*.py -v-> python3 scripts/run_autopilot_dry_run.py-> make lint && make test | <ul style="list-style-type: none">-> services/core.py — core business logic-> services/router.py — FastAPI router for this feature-> tests/test_.py — unit + integration tests-> scripts/run_week104_feature.py — CLI to exercise the feature-> frontend/src/pages/Page.tsx — UI component-> docs/week104_design.md — design decisions + API contract |
|--|--|

API ENDPOINTS TO BUILD

- > GET /api/v1/ — list or status endpoint
- > POST /api/v1/ — create or trigger endpoint
- > GET /api/v1/{id} — detail endpoint
- > DELETE /api/v1/{id} — delete or cancel endpoint

TESTS TO WRITE

- > test_happy_path.py: nominal flow works end to end
- > test_edge_cases.py: empty input, null values, boundary conditions
- > test_error_handling.py: invalid input returns 400 with clear message
- > test_integration.py: feature integrates with autopilot pipeline
- > playwright/test_ui.py: UI page renders without console errors

ENV VARS & DEPENDENCIES

- | | |
|---|--|
| <ul style="list-style-type: none">-> FEATURE_ENABLED=true-> FEATURE_TIMEOUT_SECONDS=30-> FEATURE_MAX_RETRIES=3 | <ul style="list-style-type: none">-> Core: fastapi, sqlalchemy, pydantic-> Data: pandas, numpy-> Testing: pytest, httpx, playwright-> Feature-specific: see week title for specialized packages |
|---|--|

DELIVERABLES

- > 2-year performance report PDF
- > Open-source repo live with README

- > Technical article published
- > Year 3-4 roadmap finalized
- > v3.0 tagged on GitHub

AUTOPILOT LOGIC

- > Two years of autonomous operation — the autopilot has proven itself at real stakes
- > Open-source core: give back to the trading community that provided the tools
- > Track record = foundation for Year 3 institutional conversations

PITFALLS & HOW TO AVOID

- > Build the service layer first, wire to API second — easier to test
- > Mock all external APIs in tests — no live calls in CI
- > Log every decision with enough context to debug 3 months later
- > Add the feature to the autopilot RunArtifact output for full traceability
- > Write the test BEFORE the implementation for clearer requirements

SUCCESS METRICS

- > Feature endpoint responds in < 500ms p95
- > All tests pass in CI with no flaky failures
- > Feature logged in RunArtifact for every autopilot cycle
- > No regressions in existing test suite after integration
- > UI page Lighthouse score ≥ 75

RISK

- > Risk: 2-year results below target → honest retrospective; document what to change for Y3
- > Risk: Open-source exposes proprietary alpha → carefully audit what stays private

DAY-BY-DAY BREAKDOWN

- > Mon: 2-year analysis + performance report PDF
- > Tue: Open-source prep + contribution guide
- > Wed: Technical article draft + publish
- > Thu: Year 3-4 roadmap + v3.0 release notes
- > Fri: Tag v3.0; publish article; celebrate 104 weeks of building